

Architettura e Gestione delle Basi di Dati

Appunti - Teoria della Concorrenza e Sistemi Distribuiti

Tutor Accademico

3 maggio 2026

Prefazione

Questo documento raccoglie gli appunti avanzati strutturati in logica Zettelkasten riguardanti i sistemi transazionali, la teoria della concorrenza, la serializzabilità (CSR e VSR), i protocolli di locking (2PL e S2PL), il controllo tramite Timestamp e Multiversione (MVCC), fino ad arrivare alla gestione delle transazioni distribuite e al protocollo Two-Phase Commit (2PC).

Indice

1	Gestione delle Transazioni	1
1.1	Introduzione all’Affidabilità e alla Gestione delle Basi di Dati	1
1.2	Condivisione delle Risorse e Controllo della Concorrenza	2
1.3	Architettura e Moduli di un DBMS	3
1.4	Transazione, Processo e Programma Applicativo	3
1.5	Gestione Operativa delle Transazioni in JDBC e SQL	4
1.5.1	Transazioni in JDBC	4
1.5.2	Transazioni nel linguaggio SQL	5
1.5.3	Esempi in codice SQL	5
1.6	Le Proprietà Acide (ACID) delle Transazioni	6
1.6.1	Atomicità	6
1.6.2	Consistenza	9
1.6.3	Isolamento	11
1.6.4	Durabilità (Persistenza)	13
1.7	Transazioni Relazionate ai Moduli di un DBMS	13
2	Gestione dell’Affidabilità: Atomicità e Durabilità	15
2.1	Gerarchia e Persistenza delle Memorie	15
2.2	Classificazione dei Guasti	16
2.2.1	Il Modello "Fail-Stop"	16
2.3	Il Gestore dell’Affidabilità e il Log	17
2.3.1	Metafore del Log: Arianna, Hansel e Gretel	18
2.4	Architettura del Controllore dell’Affidabilità	18
2.5	Modello di Riferimento delle Operazioni	19
2.6	Composizione e Scrittura del Log	19
2.6.1	L’Esito e l’Irrevocabilità del Commit	20
2.7	Meccanismi di Ripristino: Undo e Redo	21
2.8	Le Regole Aurea e le Modalità di Scrittura	21
2.8.1	Modalità Immediata	22
2.8.2	Modalità Differita	22
2.8.3	Modalità Mista	23
2.9	Algoritmo di Rollback (Annullamento Isolato)	23
3	Il Concetto di Recovery: Ripristino e Salvaguardia dei Dati	25
3.1	Obiettivi e Necessità del Recovery (Ripristino a Caldo)	25
3.2	Algoritmo di Recovery Base: Approccio Teorico	26

3.2.1	Ottimizzazione Architetture dell'Algoritmo	26
3.3	Il Costo del Recovery e il Checkpoint Quiescente	27
3.3.1	Meccanica del Checkpoint Quiescente	28
3.3.2	L'Algoritmo di Recovery Ottimizzato (con Ckpt Quiescente)	28
3.4	Il Checkpoint Non Quiescente (Dinamico)	29
3.4.1	Modalità Esecutiva del Ckpt Non Quiescente	29
3.4.2	Algoritmo di Recovery Evoluto (con Ckpt Non Quiescente)	30
3.5	Esempio Pratico e Dettagliato di Ripresa a Caldo	30
3.5.1	Esecuzione della Fase 1: UNDO ed Estrapolazione Insiemei	31
3.5.2	Esecuzione della Fase 2: Applicazione del REDO	32
3.6	Dump, Cold Restart e Ripresa a Freddo	32
3.6.1	Meccanica del Cold Restart	33
4	Gestione della Concorrenza	35
4.1	Architettura e Ruolo del Controllo di Concorrenza	35
4.2	Tassonomia delle Anomalie di Concorrenza	36
4.2.1	Perdita di Aggiornamento (Lost Update)	36
4.2.2	Lettura Sporca (Dirty Read o Lettura prima del Commit)	37
4.2.3	Lettura Inconsistente e Aggiornamento Fantasma (Ghost Update)	38
4.2.4	Inserimento Fantasma (Phantom)	39
4.3	Riepilogo e Classificazione	39
5	Il Controllo di Concorrenza: Modelli e Anomalie Avanzate	41
5.1	Riepilogo delle Anomalie Concorrenziali	41
5.2	Livelli di Isolamento e Implicazioni Pratiche (SQL e JDBC)	41
5.2.1	Esempi Applicativi dei Livelli di Isolamento (Caso d'esame)	43
5.3	Teoria e Formazione dello Schedule	44
5.4	Obiettivi dello Scheduler: Serialità e Serializzabilità	45
5.5	Modello VSR: La View-Serializzabilità	45
5.5.1	Definizione e Verifica dell'Equivalenza VSR	46
5.5.2	Il Dramma della Complessità e l'Inutilizzabilità Pratica	47
5.6	Il Compromesso Architetture e i Limiti Superiori	47
5.7	Il Modello CSR: La Conflict-Serializzabilità	48
5.7.1	Definizione e Verifica dell'Equivalenza CSR	49
5.7.2	La Relazione Gerarchica: CSR e VSR a confronto	49
6	Formalizzazione della Concorrenza: Dai Grafi ai Lock	51
6.1	Dimostrazione: Conflict-Serializzabilità implica View-Serializzabilità	51
6.2	Il Grafo dei Conflitti e l'Aciclicità	52
6.2.1	Dimostrazione del Teorema dell'Aciclicità	53
6.3	Limiti Operativi della Conflict-Serializzabilità nella Pratica DBMS	54
6.4	Il Paradigma Fisico: Locking e Incompatibilità	55
6.4.1	Tavola dei Conflitti e Gestione delle Attese (Lock Manager)	56
6.5	Il Fallimento del Lock Puro e la Genesi del Protocollo 2PL	57
6.5.1	2PL vs CSR: L'Inclusione Insiemeistica	58
6.5.2	Il Limite Mortale: 2PL e Stallo (Deadlock)	59
6.6	Locking a Due Fasi Stretto (S2PL) e Fine dell'Utopia	60

6.7	Integrazione Complessa: Granularità, Isolamento Reale e Lock di Predicato	61
6.7.1	Implementazione Pratica dell'Isolamento e Inserimento Fantasma	62
7	Controllo di Concorrenza Avanzato: Timestamp e Multiversione	65
7.1	Il Controllo di Concorrenza basato su Timestamp (TS)	65
7.1.1	Meccanica ed Etichettatura degli Oggetti	65
7.1.2	La Variante "Thomas Write Rule" (Scritture Ignorate)	67
7.1.3	Esempio di Esecuzione TS	67
7.2	Confronto di Architettura: 2PL vs TS	67
7.3	Controllo Multiversione (MVCC)	68
7.3.1	Meccanica della Variante Multiversione Pura	69
7.4	Multiversione MVCC in PostgreSQL (Analisi e Anomalie)	70
7.4.1	Livelli di Isolamento PostgreSQL implementati su Multiversion	70
7.4.2	Esercitazioni e Dinamica Postgres VS 2PL (Scenari a confronto)	71
7.4.3	Esercizio d'Esame 27/05/2019: Le Inconsistenze Occulte	73
8	Gestione delle Transazioni nelle Basi di Dati Distribuite	75
8.1	Architetture e Tipologie di Transazioni Distribuite	75
8.2	Tecnologia, Consistenza e Durabilità nei Sistemi Distribuiti	76
8.3	Controllo di Concorrenza su Nodi Plurimi	77
8.4	Guasti Sistemici e l'Atomicità Globale	77
8.5	Il Protocollo di Commit a Due Fasi (2PC)	78
8.5.1	Dizionari di Log: TM vs RM	79
8.5.2	Evoluzione Completa delle Due Fasi	79
8.6	Stati Anomali: La Finestra di Incertezza e il Recovery Distribuito	80
8.6.1	Recupero Post-Crash (Recovery del TM e dell'RM)	80

Capitolo 1

Gestione delle Transazioni

1.1 Introduzione all’Affidabilità e alla Gestione delle Basi di Dati

Le basi di dati costituiscono una risorsa fondamentale e critica per chi le possiede, rappresentando spesso il nucleo informativo essenziale per il funzionamento di aziende e organizzazioni. Per tale motivo, esse devono essere intrinsecamente affidabili e i dati in esse contenuti devono essere rigorosamente conservati, preservandone l’integrità anche e soprattutto in presenza di malfunzionamenti hardware o software, cadute di sistema (crash) o interruzioni anomale dell’alimentazione.

Un esempio classico che illustra la necessità critica di questa affidabilità è rappresentato dal trasferimento di fondi tra due conti correnti bancari. Questa operazione logica si scompone fisicamente in almeno due operazioni distinte: il prelievo dal primo conto e il successivo versamento sul secondo. Qualora si verificasse un guasto del sistema esattamente a metà dell’esecuzione, ovvero dopo il prelievo ma prima del versamento, la base di dati verrebbe a trovarsi in uno stato inconsistente, con una perdita netta di denaro.

Per prevenire simili anomalie, è strettamente necessario che i Sistemi di Gestione delle Basi di Dati (DBMS) forniscano un supporto nativo alla gestione di transazioni che godano di due proprietà fondamentali:

- **Atomicità:** Le transazioni devono essere eseguite seguendo la logica del "tutto o niente" (all-or-nothing). Questo implica che le operazioni che compongono una transazione non possono essere applicate in modo parziale; o tutte le modifiche vengono riflesse nel database, oppure nessuna di esse deve avere alcun effetto, riportando il sistema allo stato antecedente l’inizio della transazione.
- **Definitività:** Le transazioni completate con successo devono produrre effetti definitivi. Dopo la loro naturale e corretta conclusione, le modifiche apportate divengono permanenti e il sistema non può in alcun modo "dimenticarle", a prescindere dai guasti futuri che potrebbero verificarsi.

Il raggiungimento e il mantenimento di tale affidabilità rappresentano una sfida tecnologica particolarmente impegnativa per l’architettura di un DBMS. Le basi di dati, infatti, non sono entità statiche, ma subiscono aggiornamenti continui e

frequenti. In aggiunta, per questioni di efficienza e latenza, le operazioni di lettura e scrittura non avvengono direttamente sul disco fisico, ma passano attraverso un'area di memoria volatile, gestita dal modulo del sistema noto come gestore del buffer (buffer manager). La presenza di questo buffer introduce una complessità notevole: poiché i dati modificati nel buffer potrebbero non essere ancora stati trascritti fisicamente (flushed) sulla memoria secondaria al momento di un guasto, il sistema deve disporre di meccanismi avanzati per garantire che nessuna transazione confermata vada perduta e che nessuna transazione incompleta lasci tracce permanenti.

1.2 Condivisione delle Risorse e Controllo della Concorrenza

Oltre all'affidabilità, una caratteristica peculiare delle basi di dati è la loro natura profondamente condivisa. Una base di dati si configura come una risorsa integrata, destinata a essere fruita contemporaneamente da svariate applicazioni e utenti. Questa architettura multi-utente e multi-applicazione genera due ordini di conseguenze cruciali:

1. **Meccanismi di autorizzazione:** Poiché vi sono attività diverse che insistono su dati parzialmente condivisi, è imperativo implementare sistemi di controllo degli accessi. Il DBMS deve assicurare che ogni utente o applicazione operi esclusivamente all'interno dei privilegi e dei permessi a esso accordati.
2. **Controllo della concorrenza:** L'esecuzione simultanea di attività multi-utente su dati condivisi impone la necessità di orchestrare gli accessi concorrenti per evitare interferenze distruttive.

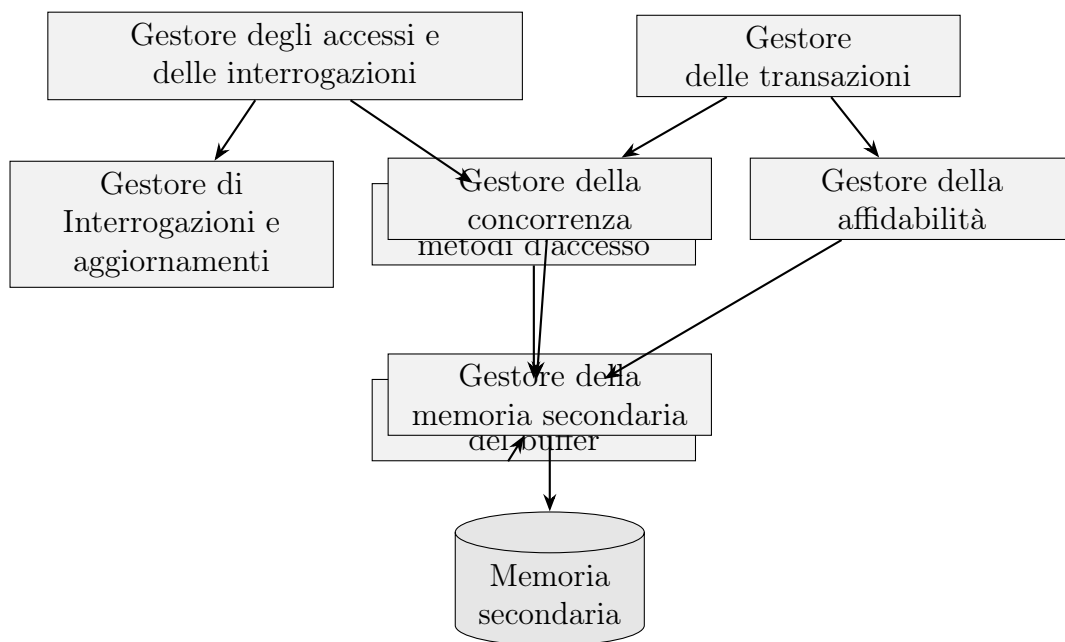
Per comprendere la criticità del controllo della concorrenza, si considerino le problematiche derivanti da aggiornamenti simultanei su basi di dati condivise. Un primo esempio emblematico riguarda l'esecuzione di due prelevamenti che avvengono in modo (quasi) contemporaneo sul medesimo conto corrente da due sportelli bancomat differenti. Se il sistema leggesse per entrambe le operazioni il saldo iniziale prima di applicare la rispettiva detrazione, il saldo finale rifletterebbe l'aggiornamento di una sola transazione, "sovrascrivendo" e vanificando di fatto l'altra (fenomeno noto come *lost update*). Un secondo esempio è fornito dai sistemi di biglietteria elettronica, dove due prenotazioni (quasi) contemporanee potrebbero tentare di riservare lo stesso identico posto su un volo o per uno spettacolo.

Intuitivamente, la soluzione più banale per garantire la correttezza assoluta delle transazioni consisterebbe nell'imporre una esecuzione strettamente seriale: elaborare la prima transazione per intero e, solo alla sua conclusione, avviare la seconda. Tuttavia, nei sistemi reali operanti su larga scala, forzare un'esecuzione puramente seriale penalizzerebbe l'efficienza e il throughput (il numero di transazioni eseguite per unità di tempo) in maniera del tutto inaccettabile, causando lunghi tempi di attesa. Pertanto, i DBMS implementano sofisticati meccanismi di controllo della concorrenza che permettono un ragionevole compromesso: essi consentono un'esecuzione "intrecciata" (interleaved) delle operazioni, ma esclusivamente quando questa

sovrapposizione è sicura, garantendo che il risultato finale sia sempre logicamente equivalente a quello che si otterrebbe con una qualche esecuzione seriale.

1.3 Architettura e Moduli di un DBMS

La gestione delle transazioni coinvolge un ecosistema di moduli all'interno dell'architettura di un Sistema di Gestione di Basi di Dati. Il diagramma seguente illustra la scomposizione logica e l'interazione tra i gestori che si occupano dell'accesso ai dati, delle interrogazioni e dell'affidabilità transazionale.



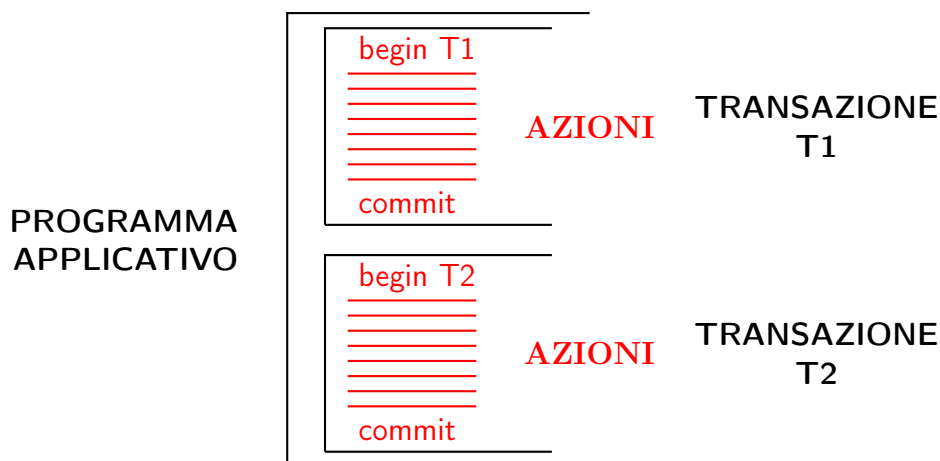
Come si evince dall'architettura, le richieste provenienti dalle applicazioni si dividono tra logica di accesso (interrogazioni) e logica transazionale. Il *Gestore delle transazioni* orchestra due componenti nevralgiche: il *Gestore della concorrenza*, che regola gli accessi simultanei per evitare conflitti, e il *Gestore dell'affidabilità*, che cura la durabilità e l'atomicità interagendo strettamente con il *Gestore del buffer*. Solo attraverso la memoria temporanea del buffer le operazioni si traducono in vere e proprie scritture e letture su memoria secondaria.

1.4 Transazione, Processo e Programma Applicativo

Per comprendere appieno le dinamiche del DBMS, è vitale distinguere concettualmente tra transazione, processo e programma. Un processo è un'entità dinamica in esecuzione le cui azioni devono essere percepite dal sistema come indivisibili: i loro effetti "vengono eseguiti" rigorosamente tutti o, in caso di anomalia, nessuno. Una transazione è essenzialmente una porzione specifica di un programma applicativo che specifica e delimita le istruzioni relative a tale processo. Essa possiede dei confini ben definiti:

- Un inizio, formalizzato dal comando `begin-transaction` (o equivalenti, come `start transaction`). Spesso questo confine iniziale non è esplicitato dal programmatore ma inferito implicitamente dal sistema all'atto della prima istruzione utile.
- Una fine, teoricamente identificabile con `end-transaction`, anche se nella pratica industriale questo comando è quasi mai esplicitato direttamente.
- Una conclusione logica cogente: all'interno dei confini della transazione deve essere eseguito una e una sola volta uno dei due seguenti comandi terminali:
 - `commit [work]`: impartito per terminare correttamente la transazione, confermando in modo irreversibile tutte le operazioni.
 - `rollback [work]`: impartito per abortire la transazione, richiedendo al sistema di annullare sistematicamente qualsiasi effetto parziale prodotto dal processo fino a quel momento.

La differenza strutturale tra il programma nel suo complesso e le singole transazioni in esso incapsulate è schematizzata nella figura seguente, dove un unico contenitore applicativo esegue due transazioni sequenziali separate dai blocchi di `begin` e `commit`.



1.5 Gestione Operativa delle Transazioni in JDBC e SQL

La traduzione dei concetti transazionali nella pratica della programmazione avviene attraverso apposite API e sintassi del linguaggio di interrogazione.

1.5.1 Transazioni in JDBC

Nell'interfaccia Java Database Connectivity (JDBC), la scelta della modalità transazionale è demandata a un metodo specifico definito all'interno dell'interfaccia `Connection`: il metodo `setAutoCommit(boolean autoCommit)`.

- Impostando `con.setAutoCommit(true)`, che rappresenta il comportamento di default dei driver JDBC, si attiva la modalità *"autocommit"*. In questo scenario, il DBMS tratta ogni singola operazione SQL come se fosse una transazione autonoma a sé stante, implicitamente avviata ed immediatamente confermata al termine dell'istruzione.
- Impostando `con.setAutoCommit(false)`, si disabilita l'automatismo e si passa alla gestione esplicita delle transazioni via codice (da programma). In questa modalità si adoperano i metodi `con.commit()` per validare il blocco e `con.rollback()` per annullarlo.

È degno di nota che in JDBC non esiste un corrispettivo esplicito del comando di inizio transazione (nessuno `start transaction`). Il sistema adotta una convenzione implicita: una nuova transazione viene aperta automaticamente in due specifici momenti logici, ovvero all'atto dell'apertura iniziale della connessione verso il database e contestualmente all'esecuzione di un `commit` o `rollback` che ha sancito la conclusione della transazione immediatamente precedente.

1.5.2 Transazioni nel linguaggio SQL

Similmente a JDBC, nel puro linguaggio SQL una transazione inizia automaticamente al momento del primo comando SQL inoltrato dopo aver stabilito la "connessione" alla base di dati, oppure immediatamente dopo la conclusione della transazione precedente. Benché lo standard internazionale SQL indichi formalmente l'esistenza di un comando `start transaction`, questo è storicamente catalogato come opzionale. Ne consegue che molti motori relazionali non ne prevedano un utilizzo strettamente obbligatorio, sebbene RDBMS moderni ed estesi come PostgreSQL lo supportino e lo implementino regolarmente.

Per la conclusione della transazione, lo standard SQL impone:

- `commit [work]`: impone al motore relazionale che tutte le operazioni specificate, conteggiate a partire dal momento esatto dell'inizio della transazione, vengano materialmente ed effettivamente eseguite sulla base di dati, divenendo visibili agli altri utenti e permanenti.
- `rollback [work]`: impone che si rinunci categoricamente all'esecuzione di tutte le operazioni specificate e accumulate dopo l'inizio della transazione. Qualsiasi alterazione pendente viene istantaneamente scartata.

Allo stesso modo di JDBC, molti sistemi relazionali offrono a livello di console o driver nativo la modalità *autocommit*, in cui ciascuna istruzione isolata forma ed esaurisce l'intero ciclo di vita di una transazione.

1.5.3 Esempi in codice SQL

Per rendere evidente la sintassi descritta, si esamini un caso base di trasferimento di 10 unità monetarie da un conto a un altro.

```
start transaction;  -- (opzionale a seconda del DBMS)
update ContoCorrente
    set Saldo = Saldo + 10 where NumConto = 12202;
update ContoCorrente
    set Saldo = Saldo - 10 where NumConto = 42177;
commit;
```

Le transazioni non sono semplici elenchi sequenziali di operazioni; esse incapsulano anche complessi rami condizionali. Si supponga di voler condizionare l'aggiornamento a un controllo sulla capienza residua del conto. Una volta decurtati i 10 dal conto origine ($\text{NumConto} = 42177$), il programma estrarrà il nuovo saldo transitorio all'interno della medesima transazione, per prendere una decisione (decision-making intrinsecamente atomico):

```
start transaction;
update ContoCorrente
    set Saldo = Saldo + 10 where NumConto = 12202;
update ContoCorrente
    set Saldo = Saldo - 10 where NumConto = 42177;

select Saldo into A
from ContoCorrente
where NumConto = 42177;

if (A >= 0) then commit
else rollback;
```

La potenza del paradigma si esplica nell'istruzione condizionale: solo se il saldo post-operazione risulta non negativo ($A \geq 0$) le azioni vengono consolidamente persistite (`commit`); viceversa, invocando il `rollback`, il DBMS provvederà in autonomia a ripristinare sia il conto destinatario (12202) che quello mittente (42177) allo stato esatto che detenevano prima del comando `start transaction`.

1.6 Le Proprietà Acide (ACID) delle Transazioni

L'efficacia concettuale di una transazione risiede nel suo configurarsi come un'unità di elaborazione che gode invariabilmente delle cosiddette proprietà "ACIDE". L'acronimo, universalmente accettato in letteratura informatica, riassume i concetti di Atomicità, Consistenza, Isolamento e Durabilità (o persistenza). Ognuna di queste proprietà risponde a un'esigenza specifica dell'elaborazione di dati in ambienti critici e condivisi.

1.6.1 Atomicità

L'atomicità asserisce che una transazione rappresenta un'entità di elaborazione atomica, governata dalla legge del "tutto o niente". Questo principio si traduce nel

fatto che se viene invocato ed eseguito il **commit**, allora *tutte* le azioni programmate all'interno della transazione hanno effetto irrevocabile sullo stato finale della base di dati. Al contrario, se per qualsiasi ragione interviene un **abort** (o **rollback**), *nessuna* azione sortisce effetto alcuno sul database persistito. Fondamentale per l'atomicità è che per il sistema non è logicamente e fisicamente ammissibile una terza possibilità: non può esistere in maniera visibile alcuno stato intermedio, ibrido o parzialmente aggiornato.

L'esito di una transazione può dipendere da varie cause, categorizzabili metaforicamente in:

- **Commit**: costituisce la conclusione definita "naturale" e corretta del processo, in cui tutte le logiche si sono dipanate senza errori semantici o impedimenti di sistema.
- **Abort (Rollback)**: interrompe l'esecuzione e inverte gli effetti. Viene pittorescamente distinto in:
 - **Suicidio**: l'abort richiesto esplicitamente dal codice dell'applicazione stessa (es. il controllo del saldo in rosso menzionato in precedenza). L'applicativo deduce un'anomalia logica e innesca autonomamente la rinuncia alle operazioni.
 - **Omicidio**: l'abort imposto dall'alto, richiesto d'imperio dal sistema relazionale per preservare l'integrità. Questo fallimento indotto dal DBMS accade a seguito di circostanze non imputabili al flusso di istruzioni dell'utente: una rottura fisica (crash) del nodo, una violazione dei vincoli di integrità del modello relazionale, la risoluzione di un vicolo cieco causato dal controllo della concorrenza (deadlock), oppure uno stato di incertezza irreversibile in caso di fallimenti distribuiti.

Per assicurare meccanicamente la garanzia del "tutto o niente", l'architettura di un RDBMS deve possedere la facoltà di cancellare a ritroso eventuali scritture provvisorie già annotate (operazione di **undo**) o di reinserirle post-crash (operazione di **redo**). Questi complessi recuperi di stato sono gestiti internamente dal DBMS e dai suoi moduli, sollevando integralmente l'utente e il programmatore dalla responsabilità di scriverne la logica nei propri applicativi.

Esempio di Atomicità in PostgreSQL. Per chiarire empiricamente questi esiti, si propone una casistica in ambiente PostgreSQL (da testare idealmente mediante l'utilizzo in parallelo di due sessioni client distinte per percepirne gli effetti in tempo reale). Si predisponga un rudimentale schema relazionale con due tabelle **r1** e **r2**, legate tra loro da un vincolo di chiave esterna.

```
drop table if exists r1 cascade;
drop table if exists r2 cascade;
create table r1 (a integer primary key, b integer, c integer);
create table r2 (d integer primary key, e integer, f integer);
alter table r1 add constraint r1_c_fk
    foreign key (c) references r2 (d);
```

- **Operazione in autocommit:** Eseguendo una singola direttiva DML:

```
insert into r2 values (0,0,0);
```

Il sistema avvia l'operazione in autocommit, e senza bisogno di delimitatori la riga viene persistentemente salvata nel dizionario di `r2`.

- **Transazione che abortisce:** Se si avvia esplicitamente il blocco con lo scopo di annullarlo:

```
start transaction;
insert into r2 values (1,1,1);
select * from r2;    -- La riga [1,1,1] risulterà visibile limitatamente
                    -- allo scope di questo blocco isolato.

rollback;
```

Al termine del `rollback`, la tabella ritornerà istantaneamente priva del record contenente i valori (1, 1, 1). Il tentativo di tracciamento risulta estirpato senza lasciare alcuna traccia.

- **Transazione che va in commit:** Per terminare in modo coerente e definitivo una catena d'azione:

```
start transaction;
insert into r2 values (1,1,1);
insert into r1 values (1,1,1);
commit;
```

Al comando finale, il blocco di due istruzioni (corretto rispetto alla dipendenza gerarchica) viene solidificato nella base di dati a vantaggio di tutti gli attori che vi accedono.

- **Transazione incompleta (effetto di un crash):** Un'operazione intrapresa senza poterla concludere:

```
start transaction;
insert into r2 values (2,2,2);
```

Se in questo momento esatto si procede ad "uccidere `pgadmin`" (simulando un severo crash dell'applicativo o della connessione senza inviare abort espliciti), la transazione risulta spezzata e diviene orfana. Il DBMS interpreterà il disallineamento come fallimentare e provvederà autonomamente tramite procedure di recupero (recovery) ad abbattere lo stato provvisorio, applicando di fatto un abort (omicidio di sistema).

1.6.2 Consistenza

La proprietà di Consistenza stabilisce che la transazione deve rigorosamente rispettare e validare i vincoli di integrità del modello dei dati associati alle relazioni. Questa correttezza impone che, dato uno stato iniziale corretto prima dell'invocazione della transazione (prerequisito assunto come vero), anche lo stato finale, al termine della procedura transazionale, debba risultare inoppugnabilmente corretto e congruente rispetto allo schema logico. Se lo stato finale computato non rispetta in toto le restrizioni definite sul database, l'intero blocco fallisce inesorabilmente causando un'eccezione abortiva.

La peculiarità architetturale della Consistenza risiede nel fatto che, durante l'esecuzione vera e propria delle istruzioni, il sistema può ammettere la violazione di un vincolo. Nelle basi di dati transazionali, i vincoli di integrità sono di norma analizzati e controllati sistematicamente per ciascuna operazione isolata DML. Viene tuttavia concessa a livello DDL la possibilità architetturale di definire il concetto di "vincolo differito" (deferred constraint): tale meccanismo sospende la verifica tassativa del requisito referenziale o di entità fino all'attimo conclusivo di emissione del commit.

Il problema del vincolo rigido (Immediato). Osserviamo nuovamente PostgreSQL, utilizzando l'impianto base descritto in precedenza per la definizione logica:

```
drop table if exists r1; drop table if exists r2;
create table r1 (a integer primary key, b integer, c integer);
create table r2 (d integer primary key, e integer, f integer);
alter table r1 add constraint r1_c_fk
    foreign key (c) references r2 (d);
```

Tentando di popolare il dataset, si simula l'apertura di un pacchetto di esecuzione:

```
start transaction;
insert into r1 values (3,3,3);
```

Questa singola istruzione fallisce repentinamente. Il comportamento è dettato dalla natura immediata del vincolo: per memorizzare il valore $c = 3$ in **r1**, il relazionale esige pedissequamente che la tupla con $d = 3$ esista già in **r2**. Il vincolo viene valutato immediatamente, rigettando prematuramente la tupla.

Vincolo Differito (Deferred constraint). Si modifichi il DDL del database introducendo esplicitamente la direttiva di procrastinazione:

```
rollback;
drop table if exists r1; drop table if exists r2;
create table r1 (a integer primary key, b integer, c integer);
create table r2 (d integer primary key, e integer, f integer);
alter table r1 add constraint r1_c_fk
    foreign key (c) references r2 (d)
    deferrable initially deferred;
```

Eseguendo ora la casistica difettosa mostrata poc'anzi:

```
start transaction;
insert into r1 values (3,3,3);
commit;
```

In questo scenario, il DBMS accetta formalmente e processa la prima istruzione `insert` (anche se essa genera una temporanea orfanità logica), senza lanciare eccezioni. Tuttavia, nel momento esatto in cui viene impartito il `commit`, il *Gestore dell'integrità a tempo di esecuzione* entra in scena controllando tutti gli arretrati; constatando la definitiva mancanza del nodo primario su `r2`, la transazione fallisce rovinosamente proprio nell'attimo del salvataggio.

Per concretizzare una chiusura consistente occorrerà fornire i dati in maniera olisticamente completa:

```
start transaction;
insert into r1 values (3,3,3);
insert into r2 values (3,3,3);
commit;
```

Questo secondo flusso restituisce esito "*Ok!*": alla richiesta di chiusura differita, il motore rileva che, seppur momentaneamente mancante dopo la prima `insert`, il record genitore $d = 3$ in `r2` è stato infine materializzato tramite la seconda `insert`. Lo stato finale complessivo è concorde con i domini del DBMS.

Perché differire i vincoli? Il caso necessario. Esistono circostanze complesse, specificamente nei **cicli di vincoli di integrità referenziale**, in cui la procrastinazione del controllo costituisce l'unica via tecnicamente percorribile. Si supponga un paradigma bidirezionale puro (chicken-and-egg problem) dove le entità si richiamano in modo circolare:

```
drop table if exists r1; drop table if exists r2;
create table r1 (a integer primary key, b integer, c integer);
create table r2 (d integer primary key, e integer, f integer);

alter table r1 add constraint r1_c_fk
    foreign key (c) references r2(d);
alter table r2 add constraint r2_c_fk
    foreign key (f) references r1(a);
```

Tentando un inserimento standard:

```
start transaction;
insert into r1 values (3,3,3);
insert into r2 values (3,3,3);
commit;
```

Questo approccio produrrà inesorabilmente un errore al primo rigo, rendendo impensabile qualsiasi sequenza inseritiva: non posso caricare dati in **r1** se manca l'aggancio in **r2**, ma in via speculare, non posso caricare neppure su **r2** in assenza del relativo punto d'appoggio primario in **r1**. I due vincoli sono intrinsecamente paralizzanti. L'adozione della clausola **deferrable initially deferred** su entrambi i constraints consente di sospendere artificialmente la semantica relazionale: si inietta la tupla in **r1**, successivamente quella di chiusura in **r2**, e, una volta invocato il controllo unico al **commit**, il grafo si presenterà simmetricamente ed integralmente consistente ("Ok!").

1.6.3 Isolamento

L'isolamento prescrive che l'esecuzione di una determinata transazione non deve in alcun caso risentire degli effetti scaturiti dallo svolgimento di altre transazioni concorrenti presenti nel medesimo sistema. Detto in altri termini, l'esecuzione concorrente e intrecciata (interleaved) di una vasta collezione di transazioni, governata dallo scheduler del motore relazionale, deve poter produrre un risultato logicamente identico a quello che si sarebbe materializzato se si fosse forzata una rigidissima esecuzione sequenziale (seriale). Le operazioni devono verificarsi *come se*, per un qualsivoglia processo transazionale in corso d'opera, tutte le altre elaborazioni fossero virtualmente inesistenti o sospese.

I due postulati fondamentali che sorreggono l'isolamento sono:

1. La protezione da corruzioni dati quali incroci tra due operazioni quasi contestuali (es. due prelievi asincroni descritti nell'introduzione).
2. La protezione e occultamento totale dei risultati intermedi. I dati temporaneamente calcolati o scritti in seno a una transazione viva non debbono per nessuna ragione poter essere visibili agli altri client connessi prima della conclusione della medesima. Qualora l'infrastruttura disattendesse questo principio vitale (permettendo la cosiddetta "dirty read" o lettura sporca), le transazioni terze potrebbero utilizzare e veicolare all'esterno informazioni sbagliate. Di conseguenza, qualora la transazione originaria dovesse andare in abort volontario o cadere per crash, occorrerebbe annullare forzatamente anche tutte le transazioni che nel frattempo hanno consumato quei dati temporanei, generando una disastrosa propagazione di invalidamenti sistemici, tristemente nota nella teoria dei database come "**effetto domino**" (cascading rollbacks).

Meccanica dell'isolamento e blocchi. Il sistema preserva l'integrità introducendo meccanismi di blocco temporaneo dei record (locking). Prendiamo in esame una contesa esplicita simulabile ricorrendo a due Client disgiunti e concorrenti:

- *Stato Iniziale:*

```
drop table if exists r2 cascade;  
create table r2 (d integer primary key, e integer, f integer);  
insert into r2 values (1,1,1);
```

- *Evoluzione del Client 1:* Il primo utente apre le danze transazionali richiedendo una prima alterazione in lettura-scrittura sul dominio per $d = 1$:

```
start transaction;
update r2 set e = e + 1 where d = 1;
select * from r2;
```

In questo esatto frangente il Client 1 riceverà a video il dato proiettato temporaneamente nel buffer, in cui il campo e vale 2. La riga soggiacente nel disco è stata acquisita in modalità esclusiva (lock in scrittura).

- *Evoluzione concorrente del Client 2:* Contemporaneamente, da terminale parallelo, l'utente due avvia la propria procedura di modifica:

```
start transaction;
select * from r2;
update r2 set e = e + 10 where d = 1;
```

Il Client 2 eseguirà indenne la pura istruzione `select` prima del lock (visualizzando l'originale $e = 1$), ma l'esecuzione dell'istruzione `update` non si compirà immantinente: l'isolamento del sistema provocherà l'entrata in uno stato di attesa silente e indefinita (sospensione o blocco). Il Client 2 non è distrutto, ma temporaneamente paralizzato poiché la risorsa associata al predicato $d = 1$ risulta contesa ed in possesso del Client 1, il quale non ha ancora certificato se le proprie variazioni si tramuteranno o meno in una modifica globale.

Questo stallo concorrenziale permane finché il Client 1 non assume una determinazione sulla propria chiusura:

- **Scenario di Commit:** Il Client 1 invia un `commit`. L'aggiornamento transitorio si trasforma in uno stato di consistenza formale, il lucchetto di locking decade ed il DBMS risveglia il Client 2, permettendo al suo applicativo bloccato di terminare regolarmente la propria `update` (sommando +10 su quello che ora diviene il nuovo valore confermato di $e = 2$, giungendo quindi a $e = 12$). Anche il Client 2 concluderà a sua volta con un `commit`.
- **Scenario di Rollback:** Alternativamente, il Client 1 propende per scartare il progresso inserendo `rollback`. Nell'istante esatto dell'annullamento, il sistema "protegge" la risorsa cancellando il transitorio e ristabilendo il record alle condizioni antecedenti ($e = 1$). Il lucchetto viene rilasciato. Immediatamente dopo, il Client 2, precedentemente sospeso, si sveglia e porta finalmente a termine il calcolo per cui era rimasto in pendenza applicando il suo differenziale alcolabile di +10 alla base $e = 1$, stabilizzando logicamente e coerentemente la nuova riga a $e = 11$ a seguito di proprio `commit`.

In entrambe le fattispecie l'approccio pessimistico di immobilizzazione condizionale garantisce che l'esito formale corrisponda in tutto e per tutto ad un incasellamento strettamente sequenziale delle manovre.

1.6.4 Durabilità (Persistenza)

La Durabilità costituisce il suggello finale dell'architettura: essa decreta che tutti gli effetti prodotti dai mutamenti operati da una transazione, una volta andata esplicitamente a buon fine ed elaborata dal comando di `commit`, non andranno perduti sotto alcuna circostanza. Tali effetti permarranno registrati ("durano per sempre") ed incorruttibili dinanzi ad ogni futuro problema strutturale e ad una vastità di guasti. Nel gergo dell'amministrazione DB, il verbo "commit" perde l'accezione gergale per riappropriarsi del proprio rigore etimologico: esso attesta un severo e insindacabile "*impegno*".

Nella prassi l'insidia più temuta per la preservazione permanente è celata nell'esistenza del *buffer* ramificato. Avendo il motore elaborativo condotto calcoli unicamente in memoria temporanea ed appoggiandosi pesantemente su processi di scaricamento differito sul disco fisso (I/O logici e fisici), la manifestazione inattesa di un evento bloccante disastroso (un black out sistemico ad alta criticità) esporrebbe teoricamente l'infrastruttura al rischio catastrofico di dispersione irreversibile dei record che non sono ancora materialmente andati incontro al processo di immagazzinamento nel piatto ("scaricati" su disco). Onde prevenire in radice la probabilità di simili danni la durabilità fa ampio ricorso ai registri crittografici (log di affidabilità o log files) che trascrivono in maniera rigidamente sincrona le alterazioni validate transazionalmente prima di accreditare lo stato del `commit` applicativo medesimo.

1.7 Transazioni Relazionate ai Moduli di un DBMS

È opportuno chiudere il compendio mappando i criteri cardinali appena illustrati in riferimento alle sub-architetture e al diagramma descritti nei precedenti capitoli introduttivi. Ognuna delle proprietà imposte dal paradigma ACID si incarna materialmente nel lavoro invisibile dei motori che governano il core del DBMS:

- Le proprietà di **Atomicità** e **Durabilità** si collocano sotto l'egida esclusiva del **Gestore dell'affidabilità (Reliability manager)**. Ad esso spetta redigere ed interrogare i log, implementare le sofisticate e costose manovre di *undo/redo* nonchè sovrintendere alle policy del flush verso memoria secondaria in sinergia col gestore del buffer.
- La proprietà dell'**Isolamento** rientra storicamente nell'ambito e nelle competenze applicative del **Gestore della concorrenza**. Il compito immane di monitorare e dispiegare istanti per istante lucchetti relazionali e semafori sui diversi livelli di granularità risponde ad una sua esclusiva prerogativa schedulatrice.
- La proprietà di **Consistenza** è presidiata integralmente dal **Gestore dell'integrità a tempo di esecuzione**. Operando alle dipendenze dirette dei metadati creati preventivamente in ausilio con il costrutto ed il supporto vitale del compilatore del vocabolario strutturale DDL, esso valuta attivamente scostamenti, vincoli incrociati e, non di meno, posticipazioni temporali (i meccanismi *deferrable*) in totale trasparenza e autonomia.

Capitolo 2

Gestione dell’Affidabilità: Atomicità e Durabilità

2.1 Gerarchia e Persistenza delle Memorie

La gestione dell’affidabilità in un Sistema di Gestione di Basi di Dati (DBMS) si fonda intrinsecamente sulle caratteristiche fisiche e logiche dei supporti di memorizzazione su cui il sistema opera. L’architettura hardware di riferimento può essere astratta in tre livelli fondamentali di memoria, ciascuno caratterizzato da un diverso grado di persistenza e vulnerabilità:

- **Memoria centrale (RAM):** Costituisce il supporto di lavoro primario per l’elaborazione dei dati e ospita il buffer del DBMS. Per sua natura tecnologica, è una memoria volatile, il che significa che non è persistente: al venir meno dell’alimentazione elettrica o in caso di crash del sistema operativo, l’intero contenuto informativo in essa immagazzinato viene istantaneamente perduto.
- **Memoria di massa (o secondaria, es. Dischi fissi, SSD):** Rappresenta il supporto di archiviazione principale della base di dati. Essa è persistente, garantendo la conservazione dei dati anche in assenza di alimentazione. Tuttavia, non è infallibile: i dispositivi fisici possono danneggiarsi (guasti meccanici, corruzione magnetica o usura elettronica), portando alla perdita parziale o totale delle informazioni ivi contenute.
- **Memoria stabile:** È un’astrazione logica "ideale". Si definisce come una memoria teorica che non può mai danneggiarsi o subire perdite di dati. Sebbene un dispositivo fisico singolarmente infallibile non esista in natura, l’astrazione della memoria stabile viene perseguita e approssimata ingegneristicamente attraverso sofisticate tecniche di ridondanza. L’utilizzo di configurazioni RAID (dischi replicati), la dislocazione geografica dei dati e l’ausilio di nastri magnetici o archivi di backup garantiscono che la probabilità di perdita simultanea di tutte le copie rasenti lo zero.

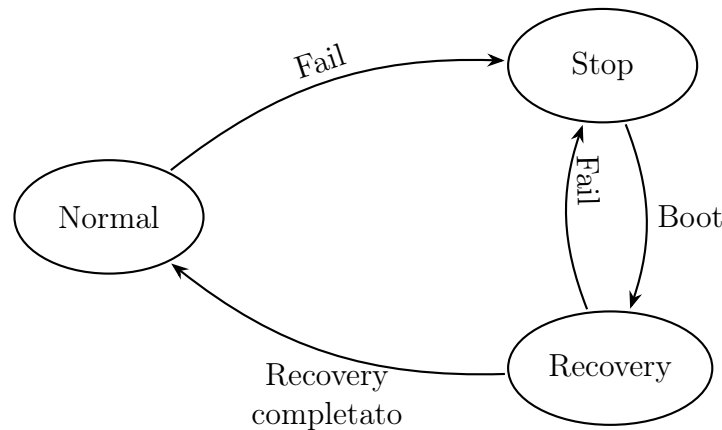
2.2 Classificazione dei Guasti

Per strutturare adeguati meccanismi di ripristino, la teoria dell’affidabilità classifica le anomalie in tre categorie di severità crescente:

1. **Guasti "soft" (Flessibili o di sistema):** Includono errori generati da un programma (es. divisioni per zero), crash del sistema operativo o repentine cadute di tensione elettrica (blackout). In queste circostanze, si perde irrimediabilmente il contenuto della memoria centrale (buffer), ma il contenuto salvato sulla memoria secondaria (dischi) resta intatto e non viene corrotto a livello hardware. La procedura di ripristino applicabile in questo scenario è definita *warm restart* (o recovery a caldo), e consiste nel riallineare il database partendo dalle tracce stabili.
2. **Guasti "hard" (Fisici o di supporto):** Coinvolgono un danneggiamento diretto dei dispositivi di memoria secondaria, come la rottura meccanica della testina di un disco rigido. In questo caso, si perde non solo la memoria centrale, ma anche la memoria secondaria stessa. Tuttavia, i dati preservati nei supporti di memoria stabile (le copie di backup o i sistemi ridondati) rimangono inalterati. Il ripristino, molto più oneroso in termini di tempo, prende il nome di *cold restart* (ripresa a freddo), richiedendo il ripopolamento fisico dei dischi danneggiati a partire dai salvataggi.
3. **Catastrofe:** Rappresenta lo scenario apocalittico in cui si verifica la perdita simultanea non solo dei supporti primari e secondari, ma anche della memoria stabile (es. la distruzione fisica del data center e del sito di backup contemporaneamente). Nell’ingegneria del software transazionale, si pone l’assunto teorico che tale eventualità non si possa verificare, in quanto il DBMS non disporrebbe più di alcun dato sorgente da cui operare un ripristino.

2.2.1 Il Modello "Fail-Stop"

Il comportamento dinamico del sistema in relazione ai guasti può essere modellato attraverso un automa a stati finiti noto come modello "fail-stop". Questo paradigma assume che il sistema non compia operazioni arbitrarie o bizantine durante un malfunzionamento, ma si blocchi immediatamente smettendo di erogare servizi, prevenendo così la corruzione attiva dei dati.



Il sistema opera ordinariamente nello stato *Normal*. L'insorgere di un guasto (*Fail*) lo fa precipitare immediatamente nello stato *Stop*, paralizzando ogni operazione utente. A seguito di un riavvio (*Boot*), il sistema transita in uno stato di *Recovery*, dove esegue le manovre di ripristino. È cruciale notare che anche durante il recovery può verificarsi un nuovo guasto (*Fail*), riportando il sistema allo stato di *Stop*. Solo quando la procedura di ripristino giunge a termine con successo, il sistema rientra nello stato *Normal*, riaprendo l'accesso alle applicazioni.

2.3 Il Gestore dell’Affidabilità e il Log

Il nucleo architetturale demandato alla salvaguardia di atomicità e durabilità è il *Gestore dell'affidabilità*. Esso ha il duplice compito di supervisionare l'esecuzione ordinaria dei comandi transazionali — ossia le fasi di **start transaction** (indicata con *B*, begin), **commit work** (*C*) e **rollback work** (o abort, *A*) — e di orchestrare le complesse e delicate operazioni di ripristino post-guasto (sia *warm restart* che *cold restart*).

Per assolvere a queste funzioni critiche, il Gestore dell'affidabilità fa un impiego intensivo di informazioni ridondanti. Gli strumenti principali a sua disposizione sono due:

1. **Il Log (o "giornale" / "diario di bordo")**: È l'elemento cardine dell'intera architettura. Consiste in un archivio permanente, rigidamente collocato in memoria stabile, che registra la cronologia di ogni singola operazione svolta. La sua struttura è quella di un file rigorosamente sequenziale: riporta le azioni esattamente nell'ordine temporale in cui si verificano. Anche se, per motivi di latenza, i record non vengono fisicamente scritti sul disco in tempo reale al millisecondo (esistono code nel buffer del log), essi vi approdano sempre e comunque in ordine strettamente cronologico.
2. **Il Dump**: Rappresenta una copia di riserva statica (backup) dell'intera base di dati, anch'essa salvata in memoria stabile. Funge da "punto di partenza" globale in caso di disastri fisici (*cold restart*), su cui successivamente riapplicare le operazioni estratte dal log sequenziale.

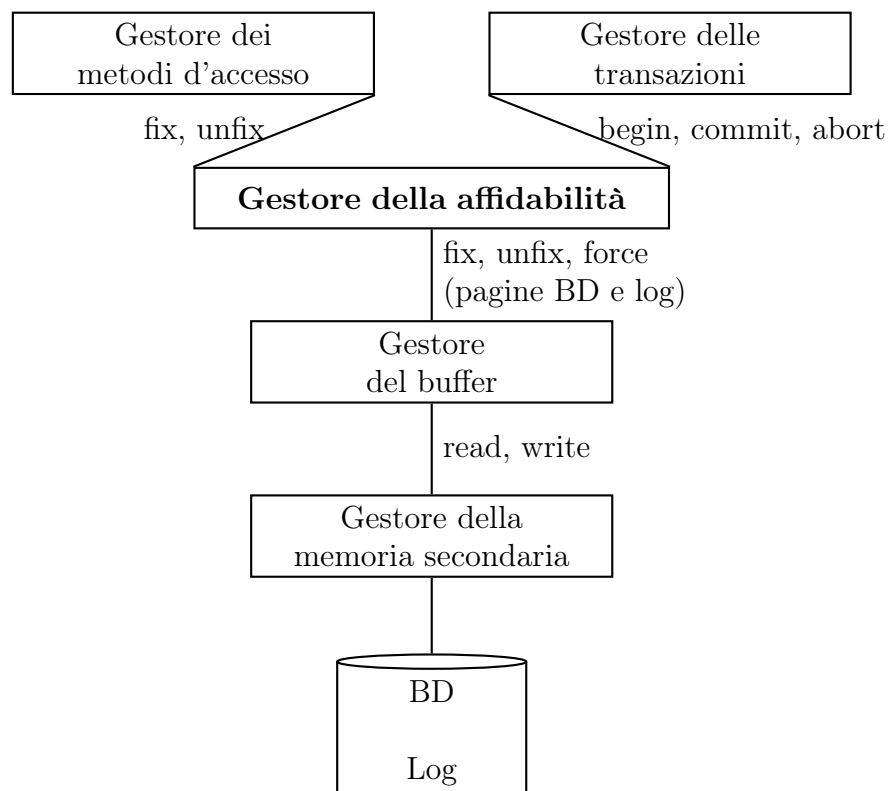
2.3.1 Metafore del Log: Arianna, Hansel e Gretel

Il ruolo concettuale del log può essere compreso appieno attraverso due celebri allegorie della narrativa e del mito:

- **Il filo di Arianna:** Nel labirinto informatico delle transazioni intrecciate, il log agisce come il filo srotolato da Teseo. Consente al sistema, in caso di vicolo cieco logico (un `abort`) o di interruzione anomala, di tornare esattamente sui propri passi ripercorrendo a ritroso il tragitto, disfacendo progressivamente le operazioni fino al punto di ingresso iniziale ($B(T)$), e uscendone in modo sicuro e pulito.
- **I sassolini e le briciole di Hansel e Gretel:** Questa metafora illustra la fondamentale differenza tra una memoria volatile e una stabile. Se le operazioni vengono tracciate in memoria centrale, è come se venissero segnate con delle briciole di pane: al verificarsi di un guasto (l’arrivo degli uccelli nel bosco), le tracce svaniscono, rendendo impossibile il ripristino. Scrivere il log in memoria stabile, invece, equivale a tracciare il sentiero con pietre e sassolini: essi resistono agli eventi avversi esterni, rimanendo permanentemente ancorati al suolo e permettendo sempre di ritrovare la strada di casa (la consistenza del database).

2.4 Architettura del Controllore dell’Affidabilità

Il posizionamento del Gestore dell’affidabilità nello stack interno del DBMS richiede un’interazione continua con i livelli superiori e inferiori.



Come evidenziato dall'architettura, il Gestore dell'affidabilità riceve dal gestore delle transazioni le primitive decisionali (**begin**, **commit**, **abort**) e dal gestore dei metodi di accesso le richieste di interazione con i dati. A sua volta, comanda il Gestore del buffer per ancorare temporaneamente le pagine in memoria (tramite le chiamate **fix** e **unfix**) e impone al buffer di scaricare in via perentoria sul supporto fisico i dati vitali tramite il comando di **force**. Quest'ultimo è un ordine tassativo di scrittura sincrona (bypassando le logiche di rinvio del buffer) applicato alle pagine della base di dati e, soprattutto, alle pagine del log, che poi verranno lette/scritte fisicamente (**read**, **write**) dal gestore della memoria secondaria.

2.5 Modello di Riferimento delle Operazioni

Al livello fisico-logico in cui opera il gestore dell'affidabilità, la transazione perde la sua semantica applicativa di alto livello. Il sistema non ragiona più in termini di "trasferimento bancario" o di "incremento di stipendio". Una transazione viene declassata e considerata unicamente come una nuda e cruda sequenza di operazioni di input-output su "oggetti" astratti (che possono coincidere fisicamente con un record della tabella o con un intero blocco/pagina di memoria).

Le operazioni atomiche riconosciute su un oggetto O da una transazione T sono essenzialmente tre:

- $I(O)$: inserimento (insert) di un nuovo oggetto.
- $D(O)$: eliminazione (delete) di un oggetto O esistente.
- $U(O)$: aggiornamento (update) dello stato dell'oggetto O .

Il gestore dell'affidabilità ignora completamente le formule logiche del programma. Per esempio, se il programma impartisce un $x := x + 1$, al gestore non interessa l'operatore aritmetico. Esso si limita a fotografare lo stato binario delle memorie, registrando esclusivamente:

- Il valore posseduto dall'oggetto "prima" dell'esecuzione dell'operazione, denominato **Before State (BS)**. È rilevante ed esiste solo per le operazioni di eliminazione e di aggiornamento.
- Il valore assunto dall'oggetto "dopo" l'esecuzione dell'operazione, denominato **After State (AS)**. È rilevante ed esiste solo per le operazioni di inserimento e di aggiornamento.

Nell'esempio $x := x + 1$, il gestore si preoccuperà solo di annotare che il BS era 3 e che l'AS è divenuto 4.

2.6 Composizione e Scrittura del Log

Alla luce del modello a stati fisici descritto, il file sequenziale del log si popola di specifici *record* altamente strutturati. Da un lato, troviamo i **record delle transazioni**, formattati specificando l'identificatore della transazione T , l'oggetto O e gli stati pertinenti:

- $B(T)$: segnala il *begin*, ovvero l’avvio formale della transazione.
- $I(T, O, AS)$: traccia un inserimento. L’oggetto nasce ora, quindi non possiede un before state, ma solo il nuovo after state.
- $D(T, O, BS)$: traccia una cancellazione. L’oggetto viene distrutto, si annota solo il before state prima della scomparsa, per permetterne l’eventuale resurrezione in caso di annullamento.
- $U(T, O, BS, AS)$: traccia un aggiornamento. È l’operazione più ricca in quanto prevede la cattura contemporanea di come l’oggetto era (BS) e di come l’oggetto è diventato (AS).
- $C(T)$ e $A(T)$: sanciscono l’esito finale. $C(T)$ rappresenta il *commit*, $A(T)$ l’avvenuto *abort*.

Dall’altro lato, nel log confluiscono i **record di sistema**, necessari a scopi organizzativi e di calibrazione delle riprese, come i marker di *dump* e i *checkpoint* (punti di sincronizzazione periodica tra buffer e disco).

2.6.1 L’Esito e l’Irrevocabilità del Commit

Il punto di non ritorno logico di una transazione si ha in un istante temporale precisissimo: una transazione si intende conclusa positivamente solo ed esclusivamente quando il record di commit $C(T)$ viene fisicamente scritto e consolidato nel log in memoria stabile. In quel millisecondo esatto, l’esito acquisisce la natura di atto irrevocabile.

Da questo postulato fondamentale derivano delle ferree direttive di ingegnerizzazione: il comando di commit non può mai rischiare di andare perduto a causa di latenze del buffer. Pertanto, il record di commit $C(T)$ va scritto subito, forzando la mano al sistema operativo attraverso l’operazione di **force** verso la memoria stabile. Per contro, l’annotazione dei record di abort $A(T)$ può avvenire in modalità asincrona e più rilassata, poiché se il sistema dovesse schiantarsi prima di scrivere su disco un **abort**, al riavvio, non trovando alcun **commit**, dedurrebbe implicitamente l’incompiutezza della transazione e la abortirebbe comunque, arrivando al medesimo risultato finale.

A causa della netta demarcazione imposta dal record $C(T)$, un guasto del sistema divide le transazioni in due grandi insiemi comportamentali:

- Un guasto sopravvenuto *prima* del compimento del **commit** interrompe il processo e lascia tracce parziali. Occorre estirpare questi dati "sporchi" per ricostruire fedelmente lo stato originario della base di dati antecedente l’inizio.
- Un guasto sopravvenuto *successivamente* al compimento del **commit** non deve in alcun modo avere conseguenze. Se, a causa dell’uso del buffer, lo stato finale promesso dalla transazione non è stato fisicamente scaricato sui dischi della base di dati prima del blackout, il sistema è contrattualmente obbligato a ricostruirlo deducendolo dal log.

Nasce dunque l’esigenza vitale di dotare il DBMS di meccanismi meccanici simmetrici per *annullare* gli errori (Undo) e per *ricostruire* i dati garantiti (Redo).

2.7 Meccanismi di Ripristino: Undo e Redo

Le funzioni di Undo (disfare) e Redo (rifare) operano manipolando le immagini di stato presenti nel log.

Undo di un'azione su un oggetto O:

- Se l'azione era un **update** o un **delete**, l'undo preleva rigorosamente dal log il valore del Before State (BS) e lo ricopia fisicamente sopra l'oggetto *O* nel database, ripristinando il passato.
- Se l'azione originaria era un **insert**, l'undo consiste semplicemente nell'eliminare l'oggetto *O* appena creato (se ancora presente a livello fisico).

Redo di un'azione su un oggetto O:

- Se l'azione era un **insert** o un **update**, il redo preleva dal log il valore dell'After State (AS) e lo ricopia a forza sull'oggetto *O*, riaffermando l'avvenuta modifica.
- Se l'azione era un **delete**, il redo sopprime nuovamente l'oggetto *O* (qualora fosse impropriamente riapparso o rimasto pendente).

Una proprietà matematica e strutturale imprescindibile di queste due primitive è la loro **idempotenza**. Un'operazione è idempotente se la sua applicazione multipla e ripetuta su un dato restituisce esattamente lo stesso identico risultato della sua singola applicazione iniziale. In formule matematiche:

$$\text{undo}(\text{undo}(A)) = \text{undo}(A)$$

$$\text{redo}(\text{redo}(A)) = \text{redo}(A)$$

L'idempotenza riveste un'importanza cardinale per l'affidabilità: se durante l'esecuzione di una fase di recovery interviene drammaticamente un ulteriore guasto (si veda l'anello *Recovery* $\rightarrow$ *Fail* nel modello fail-stop), il sistema può, al riavvio successivo, riapplicare ciecamente e da capo tutte le procedure di Undo e Redo partendo dalle direttive del log, senza il timore di corrompere lo stato calcolando la logica due o più volte. Scrivere l'After State in modo assoluto e non relativo (es. porre $x = 4$ anziché istruire di fare $+1$) è ciò che garantisce questa assenza di effetti collaterali distruttivi nelle esecuzioni multiple.

2.8 Le Regole Aurea e le Modalità di Scrittura

Il funzionamento sicuro delle riprese dipende dall'osservanza di due regole sequenziali inviolabili. Il concetto base, che riecheggia la saggezza popolare del "prendete nota della password prima di procedere al reset", impone che si debba tassativamente scrivere nel file di log l'intenzione prima di apportare qualsiasi alterazione e prima di convalidarla.

1. **Write-Ahead-Log (WAL):** Onde potersi riservare la garanzia di "disfare" un’azione in corso (Undo), il sistema impone che venga fisicamente forzata la scrittura del *Before State (BS)* sul log in memoria stabile *prima* ancora che avvenga l’equivalente scrittura del dato modificato nelle tabelle fisiche della base di dati.
2. **Regola della Commit-Precedenza:** Onde potersi riservare la garanzia di "rifare" un’azione legittimata dal commit (Redo), il sistema impone che venga fisicamente forzata la scrittura dell’*After State (AS)* sul log in memoria stabile *prima* di poter formalizzare sul log medesimo il record terminale di **commit** $C(T)$. Come insegna la metafora: "scrivetevi la nuova password su un taccuino indelebile prima di farvene sostituire una vecchia".

Rispettando questi due severi vincoli cronologici a carico del log, il gestore del buffer mantiene tuttavia una certa libertà logistica su *quando* materialmente far scaricare le pagine modificate dei dati veri e propri sulla memoria secondaria del database. Questa libertà determina l’esistenza di tre distinte politiche di scrittura: **immediata**, **differita** e **mista**.

Le tre architetture di scrittura mostrano comportamenti antitetici e determinano approcci differenti nell’affrontare le transazioni pendenti al momento dello schianto di sistema ("Crash"). A scopo esplicativo, supponiamo una linea temporale incrociata dalla barriera fatale del Crash. Sulla linea operano cinque transazioni: T_1, T_2 e T_3 che giungono al commit prima del guasto, mentre T_4 e T_5 vengono falciate dal crash in piena operatività, divenendo "uncommitted".

2.8.1 Modalità Immediata

Nella modalità immediata, il DBMS è autorizzato a scaricare liberamente le modifiche sui dischi della base di dati man mano che la transazione avanza, a patto di rispettare solo il Write-Ahead-Log. Pertanto, nel momento del crash, il DB fisico contiene sia i valori modificati e legittimati dalle transazioni completate (T_1, T_2, T_3), sia i residui altamente compromettenti delle transazioni rimaste a metà (T_4, T_5).

- **Effetto Recovery:** Le transazioni andate in commit non richiedono alcuna attenzione protettiva (esito "*Niente*"), in quanto si ha la certezza matematica che i loro esiti siano già trascritti e permanenti. Al contrario, i residui di T_4 e T_5 costituiscono un grave pericolo e debbono essere rimossi retroattivamente leggendo i BS dal log.
- **Paradigma:** Questa politica definisce un’architettura **UNDO-only (No-REDO)**.

2.8.2 Modalità Differita

All’estremo opposto, la politica differita impone un vincolo di cautela ferreo al buffer: a nessuna transazione è concesso di alterare materialmente le pagine della base di dati residente su disco fino all’istante esatto in cui non ha certificato il proprio **commit**. Al sopraggiungere del guasto, il DB è assolutamente immacolato per quanto riguarda

i valori delle transazioni uncommitted ($T4, T5$), ma al tempo stesso non vi è più alcuna certezza ingegneristica che le transazioni regolarmente concluse e commitate ($T1, T2, T3$) abbiano avuto il tempo materiale di essere processate e fisicamente riversate dalla memoria volatile ai piatti del disco.

- **Effetto Recovery:** Le transazioni interrotte $T4$ e $T5$ vengono puramente ignorate (esito "*Niente*"), in quanto non hanno mai contaminato il disco fisico. Per $T1, T2$ e $T3$, si procede a una forzata ricostruzione leggendo in massa gli AS dal log stabile e reinstallandoli nel disco.
- **Paradigma:** Questa politica definisce un'architettura **REDO-only (No-UNDO)**.

2.8.3 Modalità Mista

Nella pratica tecnologica contemporanea, la rigidità delle prime due metodologie si rivela penalizzante. Si adotta perciò una terza variante, la modalità mista. In questo scenario la scrittura verso la memoria di massa può essere pilotata in modo del tutto asincrono e svincolato: il buffer può decidere di scrivere in maniera precoce (immediata) per liberare spazio ramificato, oppure procrastinare a lungo le scritture per accorpare blocchi (differita). Al verificarsi del cataclisma, il disco verserà in uno stato di caos indifferenziato: vi saranno porzioni salvate di transazioni compiute ($T1 \dots T3$), assieme a tracce di transazioni interrotte precocemente ($T4, T5$).

- **Effetto Recovery:** L'algoritmo di ripristino deve farsi carico di entrambe le manovre. Per le transazioni incomplete ($T4, T5$) deve attuare un severo *Undo*; per quelle garantite dal commit ($T1 \dots T3$) deve innescare un *Redo* per esser certo della presenza dei dati.
- **Paradigma:** L'architettura assume forma biunivoca: **REDO - UNDO**.

Quale conviene implementare? La risposta, nonostante l'apparente appesantimento in fase di recovery (dove il sistema deve attuare sia undo che redo), verte indiscutibilmente a favore della modalità mista. Essa delega la totale indipendenza decisionale al Gestore del buffer, permettendogli di basare la calendarizzazione del flush su euristiche di bilanciamento del carico hardware. Si sceglie deliberatamente di pagare un prezzo computazionale nettamente maggiore nelle (rare) ipotesi in cui si verifichi un guasto severo, pur di privilegiare, abbattere le latenze e massimizzare la resa delle prestazioni durante l'ordinaria e continua operatività dell'infrastruttura di database (il "salvo il caso di guasto").

2.9 Algoritmo di Rollback (Annullamento Isolato)

I meccanismi di Undo studiati per contrastare i guasti generalizzati di sistema trovano la loro più elementare ed elegante applicazione nella procedura di rollback isolato. Si tratta dell'operazione di annullamento chirurgico mirato di un'unica e specifica transazione T (presumibilmente abortita su richiesta suicida dell'applicativo o su

omicidio logico del motore relazionale) che non sia riuscita a giungere alla milestone del commit.

L’algoritmo di **rollback** opera secondo la seguente meccanica procedurale:

1. Il sistema inizia a scandire il file di log partendo rigorosamente dalla fine (dal record temporale più recente) e procedendo a ritroso, riga dopo riga, fino a giungere in prossimità del record tracciato come **begin** ($B(T)$) relativo alla specifica transazione da annullare.
2. Durante questa risalita lungo il "filo di Arianna", per ogni record intercettato che si riferisce in modo incontrovertibile a una manipolazione di stato (aggiornamento, cancellazione o inserimento) effettuata da quella medesima transazione T , il controllore esegue meccanicamente la corrispettiva istruzione formale di *undo*. Le modifiche vengono fisicamente scartate restituendo la cella di memoria al suo Before State certificato.
3. Una volta azzerata la cronologia della transazione infausta, il sistema cristallizza lo stato annotando perennemente, aggiungendolo in coda al termine della lista sequenziale del log, un esplicito record strutturale formattato come **rollback** (o $A(T)$).

Le motivazioni logiche dietro questa prassi risiedono nell’ottimizzazione del recupero: Innanzitutto, perché si procede leggendo la cronologia tassativamente a ritroso? Il motivo è duplice. In prima battuta, la scansione inversa (backward scan) è computazionalmente più economica, dato che le transazioni aperte e abortite recentemente si trovano allocate di norma nella parte terminale della coda del file sequenziale. In secondo luogo, ed è la motivazione architettturalmente più profonda, procedere a ritroso garantisce la coerenza nel sovrascrivere. Qualora una transazione avesse imperversato operando iterativamente modifiche plurime sullo *stesso identico* oggetto (ad esempio aggiornando una cella in rapida successione prima $3 \rightarrow 4$, poi $4 \rightarrow 5$, ecc.), disfando le azioni partendo dall’ultimo passo e scendendo per strati temporali, il sistema si garantisce che, giunto al termine dell’operazione di annullamento, l’oggetto regredirà esibendo come esito finale e inossidabile il valore originario di partenza, evitando corruzioni o falsi positivi intermedi.

Infine, per quale scopo si esige l’apposizione persistente di un record terminale di **rollback** in fondo al diario di bordo? Tale contrassegno funge da epitaffio dichiarativo: occorre fornire una prova indelebile per il motore relazionale affinché esso (e soprattutto qualsiasi esecuzione futura o ripresa post-disastro del gestore dell’affidabilità) "sappia" legalmente e permanentemente che quella specifica catena di tentativi è morta, è stata esplicitamente annullata, e che nessuna manovra futura dovrà mai azzardarsi a prenderne in considerazione il consolidamento logico.

Capitolo 3

Il Concetto di Recovery: Ripristino e Salvaguardia dei Dati

3.1 Obiettivi e Necessità del Recovery (Ripristino a Caldo)

Il concetto focale e la vera essenza della gestione dell'affidabilità si sostanziano nella procedura di **recovery** (o warm restart, ripresa a caldo). Lo scopo primario, imprescindibile e fondamentale del gestore dell'affidabilità è quello di garantire meccanicamente che il contenuto complessivo della base di dati sia logicamente e fisicamente corretto ogniqualvolta si verifichi un evento traumatico (come un guasto software, un crash o una caduta di tensione elettrica). Questa garanzia di correttezza è richiesta anche durante le fasi ordinarie di accensione del sistema (startup) a seguito di un normale spegnimento (shut down).

La correttezza dello stato della base di dati post-guasto (o allo startup) si articola in due corollari speculari:

1. Lo stato finale archiviato nei dischi **non deve** contenere alcuna traccia o alterazione che sia stata prodotta da transazioni lasciate in sospeso o interrotte a metà dell'opera.
2. Tutte le transazioni che, prima del crash, erano regolarmente andate a buon fine (registrando il proprio *commit* in memoria stabile) debbono essere state portate a definitivo compimento e i loro effetti devono risultare incontrovertibilmente impressi nella base di dati.

La complessità sorge dal fatto che le modalità di conclusione di una sessione di lavoro non sono sempre predicibili o "controllabili" a priori. Per il motore del database, una chiusura improvvisa determinata da un crash di sistema è indistinguibile da un arresto forzato. Per via di tale intrinseca incertezza, il paradigma architetturale impone che, all'atto di ogni riavvio o startup, il DBMS sia programmato per dubitare dello stato in cui versa la memoria di massa ed effettui, di default, una rigorosa verifica esplorativa per controllare l'eventuale presenza di disallineamenti.

3.2 Algoritmo di Recovery Base: Approccio Teorico

In via teorica, si potrebbe immaginare un metodo risolutivo ingenuo e fortemente inefficiente. Scorrendo l'intera storia del database sin dalle sue origini, per ogni singola transazione mai registrata:

- Se la transazione è andata in *commit* (il che significa che esiste in modo inequivocabile un record di commit associato sul file di log), il sistema esegue forzatamente e ciecamente il *redo* (rifacimento) di tutte le azioni di quella transazione, assicurandosi che i dati finali siano stati riversati su disco.
- Se la transazione è stata esplicitamente annullata tramite un comando di *rollback* (abort), il sistema ignora il blocco e non fa assolutamente niente, confidando che l'utente o il gestore avesse già rimosso le tracce al momento del suicidio.
- In ogni altro caso alternativo (altrimenti), ovvero in presenza di transazioni pendenti per le quali non vi è né commit né rollback, il sistema esegue tassativamente l'*undo* (disfacimento) di tutte le rispettive azioni, bonificando il disco dai frammenti.

Questo approccio intuitivo si formalizza nel classico **Algoritmo di Recovery Base**, che si snoda tipicamente in tre macro-passaggi sequenziali (identificati come passi 0, 1 e 2):

Fase 0 (Fase di Analisi e Classificazione): Si percorre integralmente il file di log andando all'indietro (a ritroso, dall'evento più recente verso il passato). L'obiettivo è individuare le transazioni e smistarle in due insiemi ben definiti: l'insieme UNDO (transazioni da disfare) e l'insieme REDO (transazioni da rifare). Se durante la lettura a ritroso si incontra un record di *commit*, l'ID della transazione viene eletto per l'inserimento nell'insieme REDO. Se, viceversa, si riscontrano tracce di azioni (insert/update/delete) per una determinata transazione di cui non si è trovato traccia né di *commit* né di *rollback* esplicito, quella transazione viene declassata e inserita nell'insieme UNDO.

Fase 1 (Fase di UNDO): Procedendo rigorosamente a ritroso nel log, il sistema esegue materialmente il comando di "disfa" su tutte le azioni che appartengono alle transazioni precedentemente accatastate nell'insieme UNDO.

Fase 2 (Fase di REDO): Una volta ripristinato il passato, si inverte il senso di marcia. Procedendo in avanti (dal punto più antico verso la fine), il sistema "rifà" o sovrascrive tutte le azioni delle transazioni presenti nell'insieme REDO, riaffermando gli aggiornamenti validi.

3.2.1 Ottimizzazione Architetture dell'Algoritmo

Dal punto di vista dell'ingegneria del software, eseguire la fase 0 come passaggio isolato rappresenta uno spreco computazionale. È prassi consolidata fondere il Passo 0 direttamente all'interno del Passo 1 (lettura a ritroso), arrivando a una procedura bifase ottimizzata:

1. **Scansione a ritroso (dalla fine verso l'inizio):** Per ogni record analizzato nel log:
 - Se è un *commit*, si aggiunge dinamicamente l'identificativo della transazione all'insieme REDO.
 - Se è un *rollback*, si aggiunge l'identificativo all'insieme temporaneo RB (Rollback).
 - Se si incontra un record operativo (un'azione su oggetti) e l'ID della transazione responsabile non figura ancora né nell'insieme REDO né nell'insieme RB, significa che è un'operazione orfana: si procede in tempo reale a disfare l'azione applicando il valore di Before State.
2. **Scansione in avanti (dall'inizio verso la fine):** Per ogni record analizzato nel log:
 - Se il record è un'azione appartenente a una transazione che era stata etichettata e inclusa nell'insieme REDO, si procede a rifare attivamente l'azione applicando il valore di After State.

Nota Architettureale: Si potrebbe obiettare che rifare e disfare azioni temporalmente sfasate crei apparenti incoerenze formali se più transazioni diverse operano sullo stesso medesimo dato. Tali conflitti (dirty writes) sono categoricamente evitati e prevenuti a monte dalle ferree barriere di isolamento erette dal modulo del controllo di concorrenza, che assicura che una risorsa in transizione sia protetta da lucchetti (lock) e non possa generare sovrascritture anomale.

3.3 Il Costo del Recovery e il Checkpoint Quiescente

L'algoritmo di base discusso presenta un "costo" drammaticamente insostenibile nei sistemi reali. Esso obbliga a compiere la lettura integrale di tutto l'archivio sequenziale del log (per ben due volte) e, soprattutto, a scorrere all'indietro fino alle origini primordiali del database, comportando la ripetizione e l'annullamento di una moltitudine sterminata e inutile di operazioni storiche ormai consolidate.

L'interrogativo ingegneristico diviene quindi: *Come si può rendere questa ripresa molto più efficiente?* La risposta risiede nell'evitare di andare "troppo indietro" con la lettura del file. Occorre un marker, un punto di riferimento sicuro che garantisca al gestore di potersi fermare a ritroso, qualora si abbia la certezza inossidabile che, da quel punto all'indietro, non ci siano più buffer o pagine sporche non salvate fisicamente sul disco e, simultaneamente, non ci siano più vecchie transazioni vaganti o lasciate a metà.

Questa necessità si formalizza nel meccanismo del **Checkpoint inattivo (o "quiescente")**.

Il checkpoint inattivo funge da barriera temporale artificiale. La sua attuazione periodica può essere paragonata alla manovra estrema di una "chiusura dei conti" di fine anno in un'amministrazione pubblica o aziendale. In questa metafora, per effettuare una revisione pulita, a partire da un mese come fine novembre, gli uffici

smettono di accettare qualsiasi nuova richiesta dai cittadini e dedicano tutto il mese di dicembre per smaltire e concludere tutte quelle pratiche che erano state avviate in precedenza, prima di poter finalmente riaprire gli sportelli e accettarne di nuove a metà gennaio.

3.3.1 Meccanica del Checkpoint Quiescente

L'attuazione di un checkpoint quiescente nel DBMS è un processo bloccante e radicale, che si svolge attraverso i seguenti passaggi tassativi:

1. Si interrompe bruscamente e totalmente l'accettazione di ogni nuova transazione in ingresso da parte degli applicativi.
2. Si aspetta passivamente e pazientemente la naturale conclusione logica di tutte le transazioni che erano già attive e in esecuzione nel momento del blocco.
3. Raggiunto lo svuotamento delle attività, il gestore del buffer forza d'imperio la scrittura sincrona su disco fisico di tutte le "pagine sporche" (dirty pages) transitate nel buffer, cristallizzando lo stato.
4. Si inserisce a sigillo un record speciale di *checkpoint* in fondo al log e se ne forza la scrittura in memoria stabile per sancire la conclusione della sincronizzazione.
5. Terminata la manovra, si sblocca il sistema e si riprende l'ordinaria accettazione di nuove transazioni.

3.3.2 L'Algoritmo di Recovery Ottimizzato (con Ckpt Quiescente)

L'impiego del checkpoint quiescente rivoluziona le prestazioni della ripresa a caldo limitando drasticamente la profondità della scansione.

1. **Ricerca a ritroso interrotta:** Si procede dalla fine (l'ultimo record generato) andando a ritroso accumulando transazioni in REDO o RB e applicando gli UNDO. La grandissima differenza è che la corsa a ritroso **si arresta non appena viene incontrato il primo record di checkpoint quiescente**. Poiché quel checkpoint garantisce per definizione che al suo scoccare non vi fossero transazioni "a cavallo", spingersi oltre nel passato diviene superfluo.
2. **Ricerca in avanti ridotta:** Si ripercorre il log in avanti partendo **esattamente dal punto del checkpoint** fino alla fine del log, rifacendo ciecamente e solo le azioni pertinenti all'insieme REDO.

Commenti e Criticità: L'evidente vantaggio del metodo quiescente consiste nel circoscrivere brutalmente il raggio di indagine: il recovery non deve mai più risalire all'inizio della storia del log, fermandosi confortevolmente all'ultimo checkpoint incontrato procedendo dalla fine. Pertanto, il salvataggio va indotto e scatenato abbastanza di frequente (ad intervalli regolari) allo scopo di ridurre enormemente la lunghezza temporale e computazionale delle operazioni di ripristino post-guasto.

Risulta utile sottolineare che il marker di checkpoint viene tipicamente apposto in automatico dopo che un recovery ha concluso con successo le sue funzioni (per segnalare alle future esecuzioni che la base di dati, a quel punto preciso nel tempo, è pulita e corretta). Poiché lo startup, come già evidenziato, deve sempre avviare una routine di recovery esplorativa per cautela contro chiusure scorrette, in pratica la fase di startup sfocia sempre nell'emissione di un nuovo checkpoint di garanzia. Per prassi virtuosa, eseguire esplicitamente un checkpoint volontario anche prima del normale *shutdown* agevola e semplifica infinitamente il compito del recovery allo startup successivo.

Il difetto monumentale del checkpoint quiescente risiede tuttavia nel suo pesante impatto prestazionale: il blocco operativo derivante dal rifiuto di nuove transazioni e l'attesa dello svuotamento delle code attive introduce inaccettabili inefficienze in sistemi ad alta disponibilità (H24). Per applicazioni bancarie o di telecomunicazioni, un simile congelamento è impraticabile. Si è resa quindi necessaria una soluzione architetturale evoluta: "si può fare di meglio".

3.4 Il Checkpoint Non Quiescente (Dinamico)

Il superamento dei limiti di blocco si ottiene adottando il **Checkpoint Non Quiescente**. A differenza del predecessore, esso non impone un blocco prolungato per l'esaurimento delle transazioni, ma si configura come una sorta di rapida "fotografia istantanea". Lo scopo non è azzerare il carico, bensì "fare il punto" logico della situazione, registrando meticolosamente in un preciso istante T_0 un censimento delle transazioni correntemente attive e pendenti nel sistema. Dualmente, la mera non presenza di un ID transazionale all'interno della lista del checkpoint conferma tacitamente che quella specifica transazione o non è ancora mai iniziata (nascerà dopo T_0) oppure è già defunta e terminata prima di T_0 .

3.4.1 Modalità Esecutiva del Ckpt Non Quiescente

La modalità base e più lineare per operare questa fotografia istantanea prevede pochissimi passi ad altissima reattività:

1. Si applica un'interruzione di servizio microscopica: si sospende istantaneamente l'accettazione di ogni tipo di richiesta DML e transazionale (nuovi begin, scritture, letture, commit e abort).
2. Si rilevano, leggendo dalla tabella interna di sistema, gli ID di tutte le transazioni classificate come "in corso" (attive in quel nanosecondo).
3. Si invia un segnale di svuotamento (flush) per forzare la scrittura su disco di tutte le pagine sporche accatastate nel buffer. (In varianti ottimizzate per architetture No-UNDO, il sistema si limiterà a salvare selettivamente solo le pagine sporche pertinenti alle transazioni storicamente andate in commit).
4. Si compila e inserisce formalmente nel file di log il record speciale di checkpoint, arricchito strutturalmente con la lista (array) degli identificatori delle transazioni appena rilevate come attive. Si forza l'I/O verso il disco stabile.

5. Terminata l'operazione di trascrizione del censimento, si riaprono le porte e si riprende tempestivamente l'accettazione di nuove richieste.

3.4.2 Algoritmo di Recovery Evoluto (con Ckpt Non Quiescente)

Il fatto di aver scattato una fotografia asincrona senza obbligare il sistema ad attendere la fine delle operazioni comporta l'inevitabile esistenza di transazioni che "scavalcano" e rimangono a cavallo del record di checkpoint. Questo innalza la complessità dell'algoritmo di recovery in tre fasi:

Fase 1 (Ricerca e Classificazione a ritroso): Si scandisce il log dalla fine fino all'incontro del primo record di checkpoint, distribuendo normalmente in REDO o RB, e contemporaneamente eseguendo le azioni di *disfa* per le operazioni orfane (UNDO).

Fase 1-bis (Ricerca Estesa a ritroso): Giunti al record di checkpoint e letta la lista dei censiti, ci si rende conto che alcune operazioni appartenenti a tali transazioni pendenti risalgono a tempi ben antecedenti il checkpoint. Occorre quindi proseguire l'indagine. L'estensione della scansione a ritroso non è infinita, ma si arresta chirurgicamente non appena viene incrociato il record di *start* (begin) associato alla transazione anagraficamente "più vecchia" e longeva elencata nella lista del checkpoint. Durante questo scorrimento suppletivo, se l'operazione intercettata fa capo a una delle transazioni censite ma prive di garanzia REDO, le sue azioni continuano ad essere doverosamente disfatte (UNDO).

Fase 2 (Ricerca in Avanti per REDO): Una volta ripristinato ogni frammento sporco, si inverte la rotta. Si ripercorre il file di log procedendo rigorosamente in avanti, ma a differenza del metodo quiescente, non si parte dall'antico *start* più vecchio, bensì si può comodamente ripartire dal punto logico ove si era esteso il processo 1bis o (se le policy di salvataggio del buffer lo consentono) addirittura dal marker di checkpoint stesso. L'obiettivo resta invariato: scorrere in avanti e applicare il comando *rifai* su tutte e solo le azioni riconducibili all'elenco delle transazioni assicurate nell'insieme REDO.

3.5 Esempio Pratico e Dettagliato di Ripresa a Caldo

Per padroneggiare compiutamente la dinamica dell'algoritmo su un checkpoint non quiescente, è imperativo analizzare un log tracciato, simulando le fasi manuali del motore di un DBMS a fronte di un crash distruttivo.

Si consideri un log storico contenente 5 transazioni (da $T1$ a $T5$) e interagente su vari oggetti ($O1 \dots O6$), fino allo spezzarsi della catena per via di un fatale *Crash*.

Cronologia Lineare degli Eventi (dall'alto in basso): 1. $B(T1)$ e $B(T2)$ inizializzano le elaborazioni. 2. $U(T2, O1, B1, A1)$ e $I(T1, O2, A2)$ eseguono manovre su $T2$ e $T1$. 3. $B(T3)$ si unisce; subito dopo $T1$ esegue con successo il proprio $C(T1)$ (Commit di $T1$). $T1$ cessa di esistere formalmente. 4. $B(T4)$ inizia. Seguono variazioni: $U(T3, O2, B3, A3)$ e $U(T4, O3, B4, A4)$. 5. **Evento Cardine:** Il sistema

lancia un checkpoint non quiescente: $CK(T2, T3, T4)$. Il censimento registra che $T1$ è morta, mentre $T2, T3$ e $T4$ sono quelle attive. 6. Poco dopo, $T4$ va in commit: $C(T4)$. 7. $B(T5)$ si iscrive. Seguono svariate modifiche su $T3$ e $T5$: $U(T3, O3, B5, A5)$, $U(T5, O4, B6, A6)$ e un delete $D(T3, O5, B7)$. 8. **Evento anomalo:** $T3$ impone un Abort manuale, registrando $A(T3)$. 9. $T5$ va in commit felicemente: $C(T5)$. 10. $T2$ accenna un inserimento: $I(T2, O6, A8)$. 11. **CRASH:** Il sistema cade prima che $T2$ possa pronunciare la sua chiusura.

3.5.1 Esecuzione della Fase 1: UNDO ed Estrapolazione Insiemei

Il gestore del recovery, riavviato post-crash, inizializza le strutture dati vuote: $UNDO = \{\}$, $REDO = \{\}$. L'algoritmo inizia la risalita (leggendo dal basso verso l'alto).

Passo 1: Legge $I(T2, O6, A8)$. La transazione $T2$ non ha sancito la fine. L'identificativo $T2$ finisce in UNDO. Poiché si tratta di un'operazione *Insert*, l'undo consiste nel rimuovere fisicamente il nodo dal disco: **Azione:** $D(O6)$.

Passo 2: Legge il commit $C(T5)$. Si inserisce a pieno titolo $T5$ in REDO: $UNDO = \{T2\}$, $REDO = \{T5\}$.

Passo 3: Legge l'abort $A(T3)$. Pur non generando REDO, il sistema sa di dover sradicare $T3$. La inserisce precauzionalmente in UNDO: $UNDO = \{T2, T3\}$, $REDO = \{T5\}$.

Passo 4: Legge $D(T3, O5, B7)$. Poiché $T3$ è in UNDO, si disfa l'eliminazione recuperando dal log il Before State originario e riscrivendolo materialmente: **Azione:** **Assegna all'oggetto $O5$ il valore $B7$.**

Passo 5: Legge $U(T5, O4, \dots)$. $T5$ fa parte dei buoni (REDO), quindi al momento la bypassa inesorabilmente. Subito sopra trova $U(T3, O3, B5, A5)$. Essendo $T3$ cattiva (UNDO), ripristina lo stato pre-modifica: **Azione:** **Assegna all'oggetto $O3$ il valore $B5$.**

Passo 6: Procedendo si imbatte in $B(T5)$ (viene snobbata) e poi in $C(T4)$. L'identificativo $T4$ fa il suo trionfale ingresso in REDO: $UNDO = \{T2, T3\}$, $REDO = \{T4, T5\}$.

Incrocio del Checkpoint: Salendo ancora si incontra il divisorio $CK(T2, T3, T4)$. Il sistema legge la matrice dei censiti. Conclude che per ripulire completamente $T2, T3$ e $T4$ dovrà scavare a ritroso nel log ben oltre il muro del checkpoint. $T1$ non è censito, quindi le manovre di $T1$ sono considerate storiche e pure.

Passo 7 (Fase 1-bis): Si continua la risalita. Incontra $U(T4, \dots)$ ma $T4$ è in REDO, dunque si salta. Subito dopo $U(T3, O2, B3, A3)$. $T3$ fa parte di UNDO, si applica chirurgicamente il ripristino di quel frammento storico: **Azione:** **Assegna all'oggetto $O2$ il valore $B3$.** Salendo si bypassano i righe per $B(T4), C(T1)$ e $B(T3)$.

Passo 8: Si intercetta $I(T1, \dots)$. Si salta perché $T1$ non è un problema (era già morta e in commit prima del Ckpt). Si scavalca fino all'aggiornamento critico $U(T2, O1, B1, A1)$. Essendo la sfortunata $T2$ elencata tra i cattivi da epurare in UNDO, va ripulito il danno remoto: **Azione:** **Assegna all'oggetto $O1$ il valore $B1$.**

La scansione all'indietro si infrange infine sui blocchi operativi primordiali di $B(T2)$ e $B(T1)$. Avendo epurato tutti i frammenti sparsi causati da $T2$ e $T3$, il disco locale si ritrova ora magicamente immacolato da ogni potenziale corruzione temporanea. Gli insiemi stabili computati per la prosecuzione sono fermamente ratificati in: $UNDO = \{T2, T3\}$, $REDO = \{T4, T5\}$.

3.5.2 Esecuzione della Fase 2: Applicazione del REDO

Forte della classificazione assodata, il gestore dei dati ribalta l'esplorazione del log, procedendo in una frenetica corsa verso valle, dal passato in direzione del crash.

L'algoritmo valuterà unicamente le istruzioni legate a $T4$ e $T5$. Durante la veloce discesa in direzione della fine, tralascerà totalmente i comandi impartiti da $T2$ e $T3$ (già annullati).

Passo 9: Quando l'algoritmo sfreccia incrociando l'istruzione in riga 9: $U(T4, O3, B4, A4)$, riconosce che $T4$ appartiene ai legittimati. Onde evitare la dispersione causata dalle modalità Differite (o miste Redo-Undo), reimprime a forza sul disco lo stato calcolato della modifica: **Azione di REDO: Assegna rigorosamente a $O3$ il differenziale imposto di $A4$.**

Passo 10: Scendendo fino al rigo 10: $U(T5, O4, B6, A6)$, si convalida la promessa del commit per $T5$: **Azione di REDO: Assegna formalmente e prepotentemente ad $O4$ il differenziale predetto in $A6$.**

Raggiunto il punto fatale di demarcazione del Crash, la base di dati è matematicamente salva. Epurata dal disordine delle rovine, e rafforzata dalle transazioni giurate, è ufficialmente pronta per sbloccare i lucchetti e riprendere a ricevere le richieste degli utenti in stato *Normal*.

3.6 Dump, Cold Restart e Ripresa a Freddo

Le logiche meticolose di ripresa a caldo falliscono clamorosamente qualora il guasto non sia di tipo "soft" ma degeneri in un **guasto "hard"**, ossia in una rottura meccanica e fisica distruttiva a carico dei dispositivi di memoria secondaria o dei piatti dischi di lavoro. In tale nefasto scenario (come l'allagamento o l'incendio di un server rack), la base di dati e tutte le tracce presenti sul supporto fisico polverizzato sono irrimediabilmente annientate.

Non potendo agire su una base di partenza, le tecniche descritte poc'anzi sono inapplicabili. Diviene strettamente necessario avvalersi di un **Dump**. Il dump non è altro che una copia totale, completa, olistica ("di riserva," o backup pesante) dell'intera immensità del database in uno specifico punto del passato.

Date le dimensioni titaniche di un simile salvataggio globale, la fotografia dump è solitamente prodotta e calendarizzata dall'amministratore di sistema in specifiche finestre notturne o festive in cui il sistema è dichiaratamente non operativo. Questo gigantesco monolite viene fisicamente e strategicamente salvato su dischi remoti o su un supporto in memoria stabile. Nell'esatto istante dell'avvio della colata di dati, viene furbescamente inserito nel file di log sequenziale un record speciale, detto *record di dump*, che attesta formalmente la data cronologica in cui il dump è stato cristallizzato, specificando tutti i puntatori vitali e i dettagli logistico-pratici

dell'operazione (struttura file, dispositivo di destinazione, ecc.). Per evidenti limiti legati all'enorme overhead e ai costi di stoccaggio generati, le manovre di dump globale vengono eseguite molto, ma molto più di rado in proporzione a quanto accade per l'infinita serie giornaliera dei marcatori di checkpoint inattivi o non quiescenti.

3.6.1 Meccanica del Cold Restart

All'avverarsi del guasto rovinoso e constatata la perdita della memoria di massa (ma potendo ancora contare sull'archivio log rimasto saldamente intonso in memoria stabile), si decreta un **Cold Restart (ripresa a freddo)**, procedura drastica articolata in tre fasi:

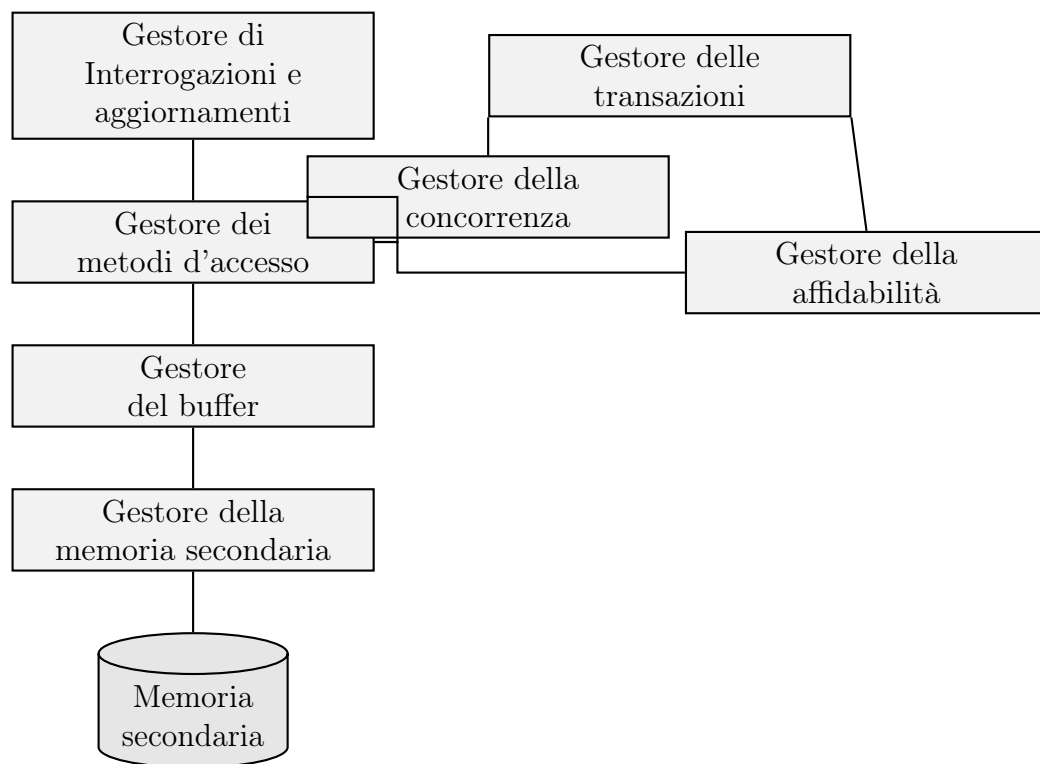
1. **Re-installazione delle fondamenta:** Un operatore sostituisce i dischi guasti con dell'hardware vergine. Il motore DBMS inizializza il restore forzato procedendo a ripristinare i dati massivi della base a partire dall'ultimo backup colossale (dump) estrapolato dal caveau o dal nastro.
2. **Roll-forward cromatico:** Raggiunta la consistenza di base legata al periodo della fotografia (es. i dati di tre notti prima), il DBMS non può arrestarsi. Procederà allora in avanti leggendo metodicamente il filo d'Arianna e si prenderà il mostruoso carico computazionale di **rieseguire integralmente** tutte le operazioni chirurgiche, transazione per transazione, rigo dopo rigo, registrate minuziosamente sul file di log partendo dal record di dump e procedendo fino al secondo esatto che ha preceduto lo schianto disastroso ("fino all'istante del guasto").
3. **Riallineamento transazionale finale:** Poiché l'istante di crash ha tranciato arbitrariamente i lavori, la base rigenerata conterrà le medesime alterazioni, imperfezioni e code "sporche" di transazioni pendenti che esistevano nel secondo esatto dell'esplosione. Come ultimo salvifico passaggio, il DBMS eseguirà sulla base dei dati appena risorta la normalissima routine di ripresa a caldo (Warm Restart con checkpoint) illustrata nei capitoli precedenti, epurando il falso stato tramite gli algoritmi di Undo e consolidando gli ultimi frammenti promessi di Redo.

Capitolo 4

Gestione della Concorrenza

4.1 Architettura e Ruolo del Controllo di Concorrenza

Nel contesto dell'architettura interna di un Sistema di Gestione di Basi di Dati (DBMS), la corretta orchestrazione degli accessi simultanei ai dati è delegata a un modulo specifico: il **Gestore della concorrenza**. Questo componente non opera in isolamento, ma si inserisce in una fitta rete di comunicazioni con gli altri moduli vitali del sistema.



Il diagramma illustra come il *Gestore delle transazioni* si interfacci in maniera bidirezionale con il *Gestore della concorrenza* e il *Gestore dell'affidabilità*. A sua volta,

il controllore della concorrenza deve dialogare con il *Gestore dei metodi d'accesso* per porre i necessari blocchi logici (lock) sui dati prima che questi vengano richiesti al *Gestore del buffer*.

La necessità di un simile apparato nasce dal fatto che la concorrenza è un requisito fondamentale e ineludibile per le basi di dati moderne. Nei sistemi informativi reali (quali gli istituti di credito, i complessi sistemi di prenotazione aerea o i server dei social network), il throughput richiesto impone l'elaborazione di decine o centinaia di transazioni al secondo. In questi scenari ad alto traffico, le transazioni non possono essere processate in modo puramente seriale (eseguendo cioè ogni processo per intero dal suo inizio alla sua fine, senza permettere azioni intercalate di altre transazioni), poiché ciò causerebbe colli di bottiglia catastrofici.

Tuttavia, l'abbandono dell'esecuzione seriale per abbracciare l'interleaving (l'alternanza delle azioni) introduce un problema architetturale profondo: l'esecuzione concorrente priva di regolamentazione genera inevitabilmente delle anomalie distruttive, che alterano la semantica dei dati e che, di conseguenza, devono essere rigorosamente governate dallo scheduler del DBMS.

4.2 Tassonomia delle Anomalie di Concorrenza

Le anomalie causate dalla concorrenza non controllata si manifestano in diverse forme fenomenologiche, a seconda della natura delle operazioni (letture o scritture) che si accavallano nel tempo.

4.2.1 Perdita di Aggiornamento (Lost Update)

La perdita di aggiornamento si verifica quando due transazioni accedono simultaneamente al medesimo dato, ne leggono il valore iniziale e calcolano un aggiornamento sovrascrivendosi a vicenda.

Si consideri l'esempio didattico di due prelevamenti eseguiti contemporaneamente da un medesimo conto bancario da due cointestatari (es. Paperina e Paperino). Lo stato iniziale della base di dati riporta un saldo pari a 1000.

1. Il primo terminale interroga il sistema: "Quanto c'è sul conto?", ottenendo in lettura il valore 1000.
2. Nello stesso istante, il secondo terminale interroga il sistema: "Quanto c'è sul conto?", ottenendo anch'esso in lettura il valore 1000.
3. Il primo utente decide di prelevare 200, calcolando localmente il nuovo saldo ($1000 - 200 = 800$) e comandando la scrittura del valore 800.
4. Il secondo utente decide di prelevare 100, calcolando localmente il nuovo saldo basandosi sulla *sua* lettura iniziale ($1000 - 100 = 900$) e comandando la scrittura del valore 900.

Il risultato finale impresso sulla memoria di massa sarà 900. Un intero aggiornamento (il prelievo di 200) viene letteralmente perso, svanito nel nulla. Se le due

operazioni fossero state collocate dal sistema in un piano di esecuzione seriale (tutta la prima transazione e, solo dopo il suo commit, l'inizio della seconda), il calcolo avrebbe restituito il risultato algebricamente corretto, ossia 700 ($1000 - 200 - 100$).

Formalizzando questo scenario, si osserva un'esecuzione concorrente disastrosa tra la transazione t_1 e la transazione t_2 sulla variabile x :

$$\begin{aligned} t_1 &: r_1(x) \\ t_2 &: r_2(x) \\ t_1 &: x = x - 200 \\ t_2 &: x = x - 100 \\ t_1 &: w_1(x) \\ t_1 &: \text{commit} \\ t_2 &: w_2(x) \\ t_2 &: \text{commit} \end{aligned}$$

Questa alternanza inaccettabile delle azioni produce una sovrascrittura cieca. Dal punto di vista sistemistico, questa anomalia è classificata come un conflitto di tipo **W-W (Write-Write)**, in quanto il danno si materializza per colpa di due scritture concorrenti non isolate.

4.2.2 Lettura Sporca (Dirty Read o Lettura prima del Commit)

La lettura sporca è un'anomalia che compromette radicalmente il principio di isolamento, permettendo a una transazione di accedere a dati temporanei generati da un'altra transazione non ancora conclusa, e che potrebbe non concludersi mai.

Si ponga il caso in cui vi siano 1000 unità sul conto. Viene avviata un'operazione di versamento di un assegno pari a 10.000 unità. Il sistema esegue l'aggiornamento e il nuovo saldo provvisorio in memoria diviene 11.000. In questo preciso frangente, un secondo utente esegue un'interrogazione per conoscere il saldo e legge con esultanza il valore 11.000 ("Abbiamo 11.000, evviva!!"). Tuttavia, poco dopo il sistema bancario rileva che l'assegno depositato è falso; la prima transazione non può concludersi positivamente ed esegue un abort, annullando l'operazione. Il saldo effettivo nel database viene ripristinato a 1000. L'utente due ha di fatto letto un dato profondamente sbagliato, un "fantasma" temporaneo.

La schematizzazione formale illustra la cronologia dell'errore:

$$\begin{aligned} t_1 &: r_1(x) \\ t_1 &: x = x + 11000 \\ t_1 &: w_1(x) \\ t_2 &: r_2(x) \quad \text{— (Lettura dello stato intermedio)} \\ t_1 &: \text{abort} \end{aligned}$$

La transazione t_2 ha catturato uno stato "sporco", non definitivo, che è stato successivamente cancellato dall'abort della transazione creatrice t_1 . La gravità

risiede nel fatto che t_2 potrebbe utilizzare questo dato fallace per assumere decisioni algoritmiche o per comunicare informazioni inconsistenti all'esterno del DBMS. Questa patologia è catalogata come un'anomalia derivante da conflitti **R-W** (**Read-Write**) o **W-W** culminanti in un *abort*.

4.2.3 Lettura Inconsistente e Aggiornamento Fantasma (Ghost Update)

Questo pattern anomalo si presenta quando una transazione esegue calcoli su un set di dati correlati, mentre una seconda transazione ne altera una parte modificando la coerenza logica dell'intero set.

Si immagini la necessità di calcolare il patrimonio totale di un utente distribuito su due conti: il conto A (contenente 1000) e il conto B (contenente 1000). La somma logica del patrimonio in questo stato coerente è banalmente 2000.

1. Una transazione di lettura interroga il conto A e acquisisce il valore 1000.
2. Nel frattempo, una seconda transazione si intromette ed esegue uno spostamento di fondi (un bonifico interno di 500) dal conto A al conto B . La transazione decurta A (che diventa 500), incrementa B (che diventa 1500) e chiude validamente con un commit.
3. La prima transazione, ignara, prosegue il suo calcolo interrogando ora il conto B e acquisendo il valore aggiornato di 1500.
4. Il risultato finale della lettura affermerà falsamente: "Ehi, abbiamo 2500!" ($1000 + 1500$).

Formalmente, si ha un aggiornamento fantasma o lettura inconsistente con la base di dati:

$$\begin{aligned}
 t_1 &: r_1(y) \quad (\text{legge } 1000) \\
 t_2 &: r_2(y) \\
 t_2 &: y = y - 500 \\
 t_2 &: r_2(z) \\
 t_2 &: z = z + 500 \\
 t_2 &: w_2(y) \\
 t_2 &: w_2(z) \\
 t_2 &: \text{commit} \\
 t_1 &: r_1(z) \quad (\text{legge } 1500) \\
 t_1 &: s = y + z
 \end{aligned}$$

La prima transazione (t_1) restituisce $s = 2500$, ma tale somma non è mai stata una realtà matematica nel database: t_1 percepisce un'illusione causata dallo sfasamento temporale, una fotografia di uno stato non esistente e logicamente incoerente. Tale fenomeno è causato da un conflitto **R-W**.

Esiste anche una manifestazione ancora più elementare e diretta di lettura inconsistente (nota in letteratura come *unrepeatable read*), che avviene sul medesimo singolo oggetto:

$$\begin{aligned} t_1 &: r_1(x) \\ t_2 &: r_2(x) \\ t_2 &: x = x + 1 \\ t_2 &: w_2(x) \\ t_2 &: \text{commit} \\ t_1 &: r_1(x) \end{aligned}$$

In questa sequenza, t_1 legge semplicemente due valori diversi per la stessa variabile x all'interno dello stesso ciclo transazionale. Sebbene sembri un caso di scuola, questa inconsistenza è fatale nelle elaborazioni reali, ad esempio durante l'esecuzione di un join implementato mediante l'algoritmo *nested-loop*, in cui il motore relazionale è costretto a scansionare e leggere più volte i record della medesima relazione per incrociarli. Un mutamento intermedio corromperebbe irrimediabilmente il prodotto cartesiano del join.

4.2.4 Inserimento Fantasma (Phantom)

L'inserimento fantasma rappresenta una variazione dell'inconsistenza R-W, che non si accanisce su tuple esistenti, ma che altera il risultato delle interrogazioni che si basano su conteggi o predicati di ricerca.

Considerando il dominio di un sistema di prenotazione voli:

1. La transazione t_1 esegue una query aggregata per "contare i posti disponibili" su un volo, ottenendo un certo numero.
2. Mentre t_1 sta ancora elaborando la sua logica, si introduce la transazione t_2 , la quale interviene sulla flotta sostituendo l'aeromobile con uno più grande; essa pertanto esegue l'operazione "aggiungi nuovi posti" inserendo fisicamente nuovi record nel database, per poi concludere con un commit.
3. Se la transazione t_1 , per esigenze di conferma, ripete l'operazione "conta i posti disponibili", ricaverà magicamente un conteggio numerico differente e incoerente rispetto al primo.

L'inconsistenza deriva qui da un conflitto **R-W su un dato "nuovo"** (appunto, un fantasma apparso nel dominio della query).

4.3 Riepilogo e Classificazione

La teoria del controllo di concorrenza sintetizza le patologie descritte associandole ai pattern di conflitto generanti tra letture (Read, R) e scritture (Write, W):

- **Perdita di aggiornamento (Lost Update):** È generata da un puro incrocio in cui due transazioni scrivono simultaneamente basandosi su letture obsolete, manifestando un conflitto del tipo **W-W**.
- **Lettura sporca (Dirty Read):** Scaturisce da una transazione che legge i residui non consolidati di un'altra. È un conflitto riconducibile allo schema **R-W** (o talvolta **W-W**) che degenera a causa di un **abort** della transazione modificatrice.
- **Letture inconsistenti (Inconsistent Read / Unrepeatable Read):** Generano viste matematicamente false di dati modificati ma esistenti, e sono figlie di un incrocio asincrono **R-W**.
- **Aggiornamento fantasma (Ghost Update):** Alterazione asincrona della coerenza di un insieme di tuple preesistenti indotta da un conflitto **R-W**.
- **Inserimento fantasma (Phantom Read):** Inconsistenza aggregativa provocata dall'apparizione o scomparsa di record, innescata da un conflitto **R-W** su dato "nuovo".

Capitolo 5

Il Controllo di Concorrenza: Modelli e Anomalie Avanzate

5.1 Riepilogo delle Anomalie Concorrenziali

Per inquadrare sistematicamente i problemi derivanti da un'esecuzione concorrente non sorvegliata, è utile riepilogare le principali anomalie in funzione delle tipologie di conflitto tra letture (Read, R) e scritture (Write, W). Queste anomalie costituiscono la base formale su cui si erige la necessità di stabilire regole di isolamento:

- **Perdita di aggiornamento:** Caratterizzata da una sovrascrittura distruttiva simultanea, riconducibile a un conflitto **W-W**.
- **Lettura sporca:** Avviene quando una transazione legge un dato alterato da una seconda transazione che successivamente fallisce. Tale anomalia è classificabile come un conflitto **R-W (o W-W) con abort**.
- **Lecture inconsistenti:** La lettura di un medesimo dato in istanti di tempo successivi restituisce valori differenti a causa di aggiornamenti intermedi; conflitto di base **R-W**.
- **Aggiornamento fantasma:** Modifica della coerenza complessiva di un insieme di dati letti, indotta da un'interferenza esterna asincrona **R-W**.
- **Inserimento fantasma:** Un'alterazione del conteggio o del dominio di una query generata da un conflitto **R-W su un dato "nuovo"**.

5.2 Livelli di Isolamento e Implicazioni Pratiche (SQL e JDBC)

Per calare la teoria in scenari d'uso industriali, occorre chiedersi sempre se tutte queste anomalie ci debbano *realmente* preoccupare con la stessa intensità. Si ponga ad esempio l'esigenza di una catena di supermercati che, appoggiandosi su una base di dati clienti provvista di tessera fedeltà (tabella `Clienti(Codice, Negozio,`

Punti)), debba calcolare sistematicamente statistiche aggregate. La transazione si concretizzerebbe in una query massiva:

```
SELECT Negozio, count(*) as numClienti,  
sum(Punti) as totalePunti,  
sum(Punti) / count(*) as mediaPunti  
FROM Clienti  
GROUP BY Negozio
```

La gestione completa del controllo della concorrenza (al suo massimo livello di rigore) ha un impatto architetturale enormemente costoso e degrada le performance. La filosofia gestionale moderna postula che, laddove il massimo rigore non sia strettamente necessario ai fini operativi o di business, è lecito e doveroso rinunciare temporaneamente. (Nota bene: questo ammorbidimento è concesso *solo* per le transazioni di pura lettura, mai per quelle che comportano delle scritture attive).

Per garantire questa flessibilità programmatica, lo standard **SQL:1999** (e parallelamente l'interfaccia JDBC) introduce il concetto modulare dei **Livelli di Isolamento**. In questo paradigma:

- Le transazioni possono essere dichiarate esplicitamente come **read-only** (a indicare che non possono effettuare operazioni di scrittura, agevolando lo scheduler).
- Il livello di isolamento può essere modulato e scelto arbitrariamente per ogni singola transazione, al fine di bilanciare esattezza formale e velocità d'esecuzione.

I quattro livelli di isolamento standard, elencati in ordine crescente di rigore (dal più tollerante al più restrittivo), stabiliscono quali anomalie il sistema si impegna a prevenire attivamente:

1. **READ UNCOMMITTED**: È il livello meno protettivo. Permette lo sviluppo di letture sporche, letture inconsistenti, aggiornamenti fantasma e inserimenti fantasma.
2. **READ COMMITTED**: Evita che si verifichino letture sporche (il sistema attende il commit della controparte prima di far leggere), ma tollera l'occorrenza di letture inconsistenti, aggiornamenti fantasma e inserimenti fantasma.
3. **REPEATABLE READ**: Evita la comparsa di tutte le anomalie di stato interno sui dati già esistenti, bloccando quindi le letture sporche, inconsistenti e gli aggiornamenti fantasma. Tuttavia, è ancora strutturalmente vulnerabile all'anomalia degli inserimenti fantasma.
4. **SERIALIZABLE**: È il livello assoluto. Evita l'occorrenza di tutte le anomalie in toto, garantendo un'emulazione perfetta di un comportamento seriale isolato.

Come si può evincere formalmente, ogni livello di isolamento è concepito in modo cumulativo: esso garantisce l'assenza delle anomalie associate ai livelli che lo precedono gerarchicamente nell'elenco e, contestualmente, accetta scientemente il rischio che

si manifestino le anomalie imputabili ai livelli che lo seguono. *Nota cruciale sulle Scritture:* Si potrebbe supporre che la "perdita di aggiornamento" (Lost Update, conflitto W-W) decada al variare di questi livelli. Nella realtà dei DBMS, la perdita di aggiornamento costituisce un'anomalia di tale gravità semantica che dovrebbe essere eversamente evitata a livello di motore. Nella pratica, per scongiurare qualsiasi corruzione W-W e ottenere la corretta sovrascrittura, è prescritta l'imposizione tassativa dell'uso esclusivo del livello **serializable** per tutte quelle transazioni che operano operazioni di manipolazione e che *scrivono* dati.

5.2.1 Esempi Applicativi dei Livelli di Isolamento (Caso d'esame)

Un test fondamentale per verificare la comprensione dei livelli è calare l'astrazione SQL in contesti decisionali reali, esaminando la query del supermercato sotto varie circostanze di contorno:

- **Caso 1:** L'operazione statistica è eseguita mentre, parallelamente, vengono inseriti alcuni nuovi clienti. La finalità dell'utente che lancia la query è acquisire informazioni approssimate e ragionevolmente indicative sugli andamenti. Considerando che i nuovi dati inseriti sono "pochi", il rischio è incorrere in *inserimenti fantasma*. Tuttavia, essendo la finalità esplorativa e macroscopica, un conteggio parzialmente sfasato per eccesso o per difetto non invalida la ratio del calcolo. Il livello più congruo ed efficiente è **READ COMMITTED** (o eventualmente Read Uncommitted).
- **Caso 2:** L'operazione statistica è in esecuzione mentre si stanno massivamente modificando i valori dei punti di *tutti* i clienti a causa di un algoritmo (in cui si sa per certo che si inseriranno temporaneamente valori errati di ordini di grandezza, corretti poi a fine blocco). La finalità è sempre indicativa. In questo frangente, la lettura di un dato non definitivo ("sporco") produrrebbe somme medie aberranti (es. milioni di punti per un negozio), vanificando il senso di una media indicativa. È imperativo neutralizzare la lettura sporca. Il livello appropriato si innalza a **READ COMMITTED**.
- **Caso 3:** Si esegue l'operazione per proclamare i primi tre negozi vincitori, determinando l'assegnazione di premi economici. Fortunatamente, ci si trova in un momento di "fermo" in cui non c'è concorrenza (non vi sono aggiornamenti in corso). Poiché il calcolo è cruciale ma lo stato è statico, non vi è alcun rischio anomalo imminente. È perciò saggio sfruttare l'occasione richiedendo il livello **READ UNCOMMITTED**, beneficiando della massima velocità e assenza di over-head sui lucchetti, consoni in un ambiente esente da scritture confliggenti.
- **Caso 4:** L'operazione sancisce i tre negozi da premiare (calcolo delicatissimo) mentre un'altra utenza sta inserendo alcuni nuovi clienti (potenziali concorrenti in tempo reale). Essendo in gioco la proclamazione di una classifica vincolante (e non una media approssimata), leggere un numero clienti diverso da quello

finale per via di una variazione invisibile genererebbe *inserimenti fantasma* inaccettabili, portando all'annuncio di un vincitore falsato. Va arginata ogni singola minima potenziale interferenza. Il livello richiesto obbligatoriamente è il massimo, ovvero **SERIALIZABLE**.

- **Caso 5:** Stessa finalità di premiazione di estremo rigore (come il caso 4), ma questa volta le operazioni parallele non consistono in inserimenti nuovi, bensì in massicce revisioni aritmetiche sui valori esistenti di punti di tutti i clienti. L'esigenza è calcolare la somma perfetta sulle tuple già presenti, bloccando alla radice aggiornamenti fantasma e letture inconsistenti provocate dall'algoritmo altrui. Poiché si agisce su domini esistenti, il livello idoneo a "congelare" lo stato del dataset attuale contro le derive intermedie è **REPEATABLE READ**.

5.3 Teoria e Formazione dello Schedule

Mettendo temporaneamente da parte i dettagli operativi del Gestore del buffer o dell'affidabilità, e concentrandoci unicamente sull'interfaccia tra il modulo "Gestore delle transazioni" (che emette **begin**, **commit**, **abort**) e il "Gestore della concorrenza" (che controlla, inibisce e riordina le istruzioni **read** e **write** interrogando la "Tabella dei lock"), si definisce la base formale e "teorica" del controllo concorrentiale.

In questo modello analitico purificato, una **Transazione** viene privata di tutta la sua semantica logica e di programmazione ("senza ulteriore semantica"). Essa è ridotta unicamente ed esclusivamente a una sequenza ordinata di istruzioni di lettura (r) e scrittura (w) operanti su specifiche variabili, seguite irrimediabilmente da una conclusione (**commit** c o **abort** a). *Notazione standard:* Ogni transazione è contraddistinta da un identificatore numerico univoco posto a pedice. Esempio di una transazione singola:

$$t_1 : r_1(x)r_1(y)w_1(x)w_1(y)c_1$$

Quando più transazioni vengono inviate contemporaneamente al sistema, le loro operazioni si fondono nel tempo in un'unica successione temporale complessiva. Questa sequenza cronologica globale e intrecciata di operazioni di input/output che il DBMS si accinge a processare prende il nome tecnico di **Schedule**.

Per poter formulare agevolmente teorie algebriche sugli schedule, si introduce una vitale ipotesi semplificativa preliminare: nello studio delle sequenze, si sceglie di ignorare aprioristicamente l'esistenza e la traccia di tutte le transazioni che finiscono per andare in **abort**. Poiché il fallimento di una transazione e il suo conseguente disfacimento vengono assorbiti dal gestore dell'affidabilità annullandone gli effetti pratici, la modellazione matematica di un sistema concorrentiale sano assume che a influire in via definitiva sul bilancio siano soltanto le transazioni di successo (in sostanza, non scriviamo le dichiarazioni di **commit** o **abort** esplicite e omettiamo le ritirate). Ne consegue che l'oggetto di analisi è puramente la **commit-proiezione** degli schedule reali.

Esempio di uno schedule intrecciato:

$$S : r_1(x)r_2(z)w_1(x)w_2(z)$$

Esso nasce dall'intersezione concorrente delle transazioni primarie: $t_1 : r_1(x)w_1(x)$
 $t_2 : r_2(z)w_2(z)$

5.4 Obiettivi dello Scheduler: Serialità e Serializzabilità

Il macro-obiettivo ingegneristico per l'architettura di un controllore di concorrenza (o Scheduler) è permettere il parallelismo pur evitando chirurgicamente le anomalie. Il fondamento dell'assenza di anomalie risiede nel concetto aureo dello **Schedule Seriale**. Si definisce Seriale uno schedule nel quale le istruzioni costituenti le diverse transazioni non si sovrappongono mai e non si mischiano: una transazione è servita interamente dall'inizio alla fine, separatamente e prima che la successiva possa avviare la sua prima istruzione. Esempio Seriale:

$$S_2 : r_0(x)r_0(y)w_0(x)r_1(y)r_1(x)w_1(y)r_2(x)r_2(y)r_2(z)w_2(z)$$

Essendo le esecuzioni temporalmente incapsulate e mutualmente esclusive, la teoria dimostra in modo incontrovertibile che **uno schedule seriale non presenta né può mai presentare anomalie**. Purtroppo, come già sancito, forzare la serialità annienta le prestazioni del server.

Lo scheduler deve pertanto ricercare il compromesso logico: deve ammettere interleaving, ma a condizione imperativa che lo schedule intrecciato risultante appartenga alla speciale classe matematica degli **Schedule Serializzabili**. Uno schedule misto è Serializzabile se, pur eseguendo le operazioni in un ordine temporalmente confuso e accavallato, è garantito matematicamente che esso produca lo **stesso identico risultato logico e persistente** che si otterrebbe elaborando uno schedule perfettamente seriale costruito sulle stesse medesime transazioni. Affermare che un risultato è lo "stesso" richiede di definire criteri insindacabili di equivalenza formale tra schedule. Il ruolo meccanico del Controllore di concorrenza è per l'appunto fungere da filtro ad altissima velocità: un sistema reattivo che accetta fluidamente, rifiuta o posticipa (riordina) le operazioni atomiche richieste ai database, mantenendo rigido l'ordine interno delle azioni previsto dal programmatore della singola transazione, ma scartando ogni intreccio malevolo e *ammettendo il passaggio verso il disco ai soli schedule serializzabili*.

5.5 Modello VSR: La View-Serializzabilità

L'idea base risiede nell'individuare la classe formale degli schedule serializzabili e di farvi appartenere solo istanze sicure, col vincolo di poter verificare la correttezza a run-time e in tempi ristrettissimi. Il primo modello teorico forte e concettualmente impeccabile sviluppato in ambito accademico è la cosiddetta **View-Serializzabilità**.

Per enunciare l'equivalenza secondo questo modello, occorre fornire due definizioni strutturali preliminari sugli attributi delle operazioni appartenenti a un qualsiasi schedule S :

1. **Scrittura finale (Final Write):** Un'operazione di scrittura $w_i(x)$ all'interno dello schedule S è elevata al rango di "scrittura finale" se e solo se essa rappresenta cronologicamente l'*ultima* scrittura eseguita sul particolare oggetto x durante tutta la durata dell'intero schedule S . Questa definizione fissa il parametro responsabile per lo stato definitivo della base di dati in memoria di massa alla chiusura di tutti i lavori.
2. **Relazione Legge-da (Reads-From):** Un'istruzione di lettura $r_i(x)$ instaurata dalla transazione i "legge-da" un'istruzione di scrittura $w_j(x)$ prodotta dalla transazione j nello schedule S se e solo se il valore numerico pescato ed effettivamente incamerato da $r_i(x)$ corrisponde al valore preciso depositato da $w_j(x)$. Affinché questo legame fisico sia conclamato, devono verificarsi tassativamente e contemporaneamente due condizioni posizionali: primo, $w_j(x)$ deve precedere temporalmente $r_i(x)$ nel flusso S ; secondo, nel segmento di tempo compreso fra l'esecuzione di $w_j(x)$ e la lettura di $r_i(x)$, non deve esserci mai l'intervento di nessun'altra istruzione aliena $w_k(x)$ (di un'ipotetica transazione interposta k) volta ad alterare furtivamente la variabile.

Esempio esplicativo della relazione:

$$S_2 : r_2(x) \ w_0(x) \ r_1(x) \ w_2(x) \ w_2(z)$$

Analizzando la catena: - La lettura $r_1(x)$ è situata successivamente all'istruzione $w_0(x)$ e non vi sono alterazioni interposte. Ne deduciamo che $r_1(x)$ "legge da" $w_0(x)$. - Le *scritture finali* sancite per chiudere lo schedule sono la $w_2(x)$ per la variabile x , e la $w_2(z)$ per la variabile z .

5.5.1 Definizione e Verifica dell'Equivalenza VSR

Avendo stabilito queste metriche, la teoria postula il principio di equivalenza visiva (View-Equivalence). Due distinti schedule intrecciati S_i ed S_j (che processano nativamente lo stesso blocco di transazioni senza alterare l'ordine sequenziale impartito all'interno delle singole componenti) sono definiti **view-equivalenti** (notazione formale $S_i \approx_V S_j$) se e solo se i due flussi, una volta portati a termine, manifestano congiuntamente:

1. Le ****stesse medesime scritture finali**** per ogni variabile. Questo postula l'uguaglianza materiale e garantisce che lo stato persistente della base di dati sia perfettamente identico a prescindere dall'ordine.
2. La ****stessa medesima relazione legge-da**** per ogni transazione coinvolta. Questo assicura che in ambedue gli schedule non cambi la catena della dipendenza informativa e l'"influenza" reciproca: ogni transazione nutrirà i propri algoritmi col medesimo valore originario e percepirà lo stesso identico contesto logico, preservando l'isolamento percettivo.

Per sillogismo, un qualsiasi schedule S è decretato **view-serializzabile** (e ammesso nell'insieme aureo ****VSR****) se esiste in letteratura logica un qualsivoglia "schedule seriale" che si dimostri matematicamente view-equivalente ad esso.

Se valutassimo un disastroso caso di perdita di aggiornamento:

$$S_7 : r_1(x)r_2(x)w_1(x)w_2(x)$$

Ci si domanda se S_7 sia view-serializzabile. La risposta del modello è categoricamente "NO". Esaminando il tracciato, la relazione *legge-da* è concettualmente vuota o neutra (non esistono legami incrociati di scrittura verso l'altro utente prima della lettura). Al contrario, se si provasse a costringere l'esecuzione in uno dei due possibili schedule seriali obbligatori (eseguire tutto il ramo t_1 e successivamente tutto il ramo t_2), il secondo ramo sarebbe strutturalmente forzato a instaurare una coppia dipendente e leggere il dato partorito dal primo. Le relazioni "legge-da" tra lo schedule sfuso e le sue controparti seriali non corrisponderebbero mai, sancendo l'inammissibilità di S_7 .

5.5.2 Il Dramma della Complessità e l'Inutilizzabilità Pratica

Sul piano puramente assiomatico, l'insieme degli schedule view-serializzabili (VSR) è la traduzione matematica perfetta del concetto di correttezza concorrente assoluta. Il problema dirimente sorge prepotente nell'istante in cui si pretende che uno scheduler implementi questo algoritmo di controllo a livello sistemistico.

A livello di Teoria della Complessità Computazionale, effettuare la mera verifica di view-equivalenza se si dispongono "a vista" i due schedule chiusi S_i ed S_j costituisce un'operazione risolvibile in tempo *polinomiale*. Eppure, il vero problema architetturale risiede nel fatto che un motore database non dispone mai dello schedule seriale "di riferimento" a priori; esso deve decidere in frazioni di secondo sull'accettabilità o meno di un flusso in entrata. Affrontare la domanda aperta "Decidere se un dato schedule S è implicitamente view-serializzabile?" esige il calcolo e la verifica combinatoria della view-equivalenza di S contro *tutte* le possibili innumerevoli permutazioni seriali costruibili sulle transazioni pendenti. La scienza informatica dimostra che questo problema di astrazione appartiene alla ristretta classe di complessità dei problemi ****NP-completi****.

Un problema NP-completo è, per sua natura asintotica, talmente dispendioso a livello algoritmico che la CPU dello scheduler si saturerebbe nel vano tentativo di calcolarne l'esito formale. La conclusione perentoria e inappellabile dell'ingegneria del software è che il paradigma VSR ****non è e non sarà mai utilizzabile nella pratica industriale****. Il progetto di progettare un'architettura in grado di trovare istantaneamente sul server "tutti e soli" gli schedule serializzabili basandosi sulla visuale è condannato: è oggettivamente troppo costoso.

5.6 Il Compromesso Architetture e i Limiti Superiori

Dinanzi all'impraticabilità dell'algoritmo perfetto (il set totale degli Schedule Serializzabili coperti da VSR), sorge la necessità di approntare un correttivo di salvaguardia ("Che facciamo?").

L'intuizione formidabile che getta le basi della prassi odierna consiste nel non voler inseguire caparbiamente l'insieme VSR completo. Si preferisce piuttosto individuare, all'interno del gigantesco universo di tutti gli schedule mischiati, una sotto-classe di comportamenti che possieda una "proprietà strutturale" nettamente più restrittiva della view-serializzabilità, fungendo da "condizione sufficiente" di salvaguardia, ma che sia dotata del pregio inestimabile di essere analizzabile e calcolabile a run-time dalla macchina in tempi computazionali estremamente facili e irrisori.

Questa filosofia accetta esplicitamente una perdita logica: si decide preventivamente e lucidamente di "rinunciare" alla validazione e al salvataggio di alcuni schedule intricati ma tecnicamente sani (che sarebbero potuti essere serializzabili secondo VSR), al solo scopo di sapere con matematica e tempestiva certezza e rapidità che tutti i blocchi che effettivamente riusciranno a varcare il filtro imposto dal DBMS saranno garantitamente corretti e scevri da anomalie.

Il percorso accademico e industriale per raggiungere tale snellimento procede in due step:

1. Definizione di un'altra e più restrittiva proprietà analitica "teorica": la **Conflict-serializzabilità**.
2. Implementazione successiva di protocolli fisici e meccaniche di locking utilizzabili operativamente a livello pratico, le quali inducono il rispetto di tale restrizione, specificamente il modello **2PL (Two-Phase Locking)** o i più recenti approcci **Multiversione**.

5.7 Il Modello CSR: La Conflict-Serializzabilità

La scorciatoia strutturale su cui poggia il pragmatismo risiede nell'analisi mirata e superficiale dei soli blocchi di istruzioni pericolosi. Per poter modellare tale teoria si introduce preliminarmente il concetto nodale di istruzioni "in conflitto".

In un ambiente isolato, si palesa inequivocabilmente che un'operazione atomica a_i si trova in aperto ****conflitto**** con un'altra operazione interposta a_j se e solo se sussistono contemporaneamente tre precondizioni fatali:

1. La diversità genitoriale: Deve valere rigidamente la regola $i \neq j$. Le due operazioni contrapposte devono per forza appartenere a istanze di transazioni intrinsecamente diverse e scollegate.
2. La comunanza di bersaglio: Le due operazioni a_i ed a_j devono fisicamente tentare di manipolare o afferrare nel proprio spettro logico il medesimo "stesso oggetto" o variabile del database (es. entrambe mirano alla tupla identificativa x).
3. La manipolazione di stato: Almeno una delle due operazioni che entrano in rotta di collisione deve consistere in un'operazione di alterazione e modifica, ossia una *scrittura* attiva (Write).

In base a questi fondamenti, la teoria enuclea e restringe il perimetro a due uniche categorie strutturali di conflitto che il controllore deve monitorare incessantemente:

- **Il conflitto Read-Write** (incrociabile simmetricamente come rw o wr): Si genera quando un'utenza cerca di estrarre un valore che un'altra sta repentinamente alterando, innescando lo spettro della lettura sporca o fantasma.
- **Il conflitto Write-Write** (ww): Si genera quando due utenze collidono per il dominio di una risorsa, con l'obiettivo comune di sovrascriverla senza una coordinazione sequenziale, innescando il rischio supremo della perdita di aggiornamento.

5.7.1 Definizione e Verifica dell'Equivalenza CSR

Rintracciata e mappata la ragnatela dei conflitti puri, la giurisprudenza teorica per superare il dogma dell'NP-completezza decreta il postulato della "Conflict-Equivalenza" (Equivalenza per Conflitti).

Date le medesime transazioni, incapsulate conservativamente col proprio ordine iterativo, gli schedule paralleli S_i ed S_j sono valutati e battezzati come ****conflict-equivalenti**** (in simboli $S_i \approx_C S_j$) a patto e soltanto se *ogni singola coppia identificata di operazioni in conflitto* mantenga testardamente e rigidamente in ambedue i due differenti schedule considerati "il medesimo ordine" e orientamento temporale di prevaricazione. Se il lucchetto rw nel primo schedule era vinto da una lettura preliminare a discapito della scrittura sfidante, e lo era anche in un'ulteriore sequenza esplorata, la natura dell'intersezione resta invariata e pacifica.

Traslando il ragionamento logico alla definizione seriale si asserisce: Uno schedule intrecciato S in analisi è categorizzato e promosso come ****conflict-serializzabile**** se il DBMS è in grado di reperire idealmente uno "schedule seriale" puro che risulti, per sua intrinseca struttura, rigorosamente ***conflict-equivalente*** ad esso. Questo preziosissimo e ristretto insieme elitario di pattern sicuri è denominato acronicamente ****CSR**** (l'insieme degli schedule conflict-serializzabili).

5.7.2 La Relazione Gerarchica: CSR e VSR a confronto

Il rapporto insiemistico che si viene a instaurare tra la visione perfetta e quella filtrata dai conflitti si enuncia nel teorema cardine dell'efficienza: **Ogni schedule certificato come conflict-serializzabile (CSR) è imperativamente, matematicamente e logicamente view-serializzabile (VSR). La transizione contraria non è invece necessariamente vera e garantita.**

In termini geometrici, la sfera ristretta dei CSR è interamente iscritta e contenuta nel vasto universo protetto dei VSR. L'algoritmo CSR, analizzando solo coppie di ostilità, ignora o censura pattern innocui ma elaborati che il VSR promuoverebbe.

Il controesempio lampante della "non necessità" (cioè dell'esistenza di uno scarto non coperto da CSR) si rinviene nell'analisi dello schedule:

$$S : r_1(x)w_2(x)w_1(x)w_3(x)$$

Dal punto di vista dell'onnisciente view-serializzabilità, lo schedule sfuso S risulta sbalorditivamente view-equivalente allo schedule puro e seriale S_{ser} : $r_1(x)w_1(x)w_2(x)w_3(x)$. A prescindere da come w_1 e w_2 abbiano faticato reciprocamente, i domini finali di

base dati coincidono. Eppure, sottoponendo il medesimo schedule S all'istruttoria severa e semplificata del controllore di Conflict-Serializzabilità, esso è marchiato drasticamente come ****non conflict-serializzabile****. L'algoritmo CSR traccia un grafo e rintraccia il susseguirsi cieco di due distinte manovre: il conflitto innescato dalla coppia $r_1(x)$ seguita da $w_2(x)$ (fissando temporaneamente $t_1 \rightarrow t_2$), e la speculare invasione causata da $w_2(x)$ spiazzata da $w_1(x)$ (invertendo il senso vettoriale con $t_2 \rightarrow t_1$). La presenza di questa ambiguità o "ciclo" d'ordine, generante per definizione un disallineamento insanabile rispetto a qualsiasi possibile linearità imposta di uno schedule in fila indiana, causa l'irrevocabile rigetto preventivo dello schedule CSR, ancorché, nella sua sostanza profonda e asintotica, l'operazione non fosse di per sé malevola, certificando come la sufficienza algoritmica paghi lo scotto della rinuncia calcolata a discapito della completezza.

Capitolo 6

Formalizzazione della Concorrenza: Dai Grafi ai Lock

6.1 Dimostrazione: Conflict-Serializzabilità implica View-Serializzabilità

Nei capitoli precedenti è stato enunciato un teorema cruciale della teoria della concorrenza: *Ogni schedule conflict-serializzabile (CSR) è anche view-serializzabile (VSR)*. Tuttavia, si è ribadito che il percorso inverso non è necessariamente vero, come dimostrato dall'esistenza di controesempi (es. lo schedule $S : r_1(x)w_2(x)w_1(x)w_3(x)$, il quale è VSR in quanto equivalente alla sua controparte seriale, ma rigettato da CSR a causa dei cicli di conflitto $w_2 \rightarrow w_1$ intersecati con $r_1 \rightarrow w_2$).

Per solidificare questa gerarchia insiemistica, è doveroso dimostrare la "sufficienza", ovvero fornire una prova rigorosa del perché l'appartenenza all'insieme CSR garantisca inoppugnabilmente l'appartenenza all'insieme VSR.

Rivediamo le definizioni strutturali:

- **CSR:** Afferma l'esistenza di uno schedule seriale che risulti *conflict-equivalente* ($\approx_C$) allo schedule in esame.
- **VSR:** Afferma l'esistenza di uno schedule seriale che risulti *view-equivalente* ($\approx_V$) allo schedule in esame.

Per dimostrare inequivocabilmente che CSR implica VSR, il passaggio matematico sufficiente consiste nel dimostrare che la stessa proprietà di conflict-equivalenza ($\approx_C$) implica invariabilmente la proprietà di view-equivalenza ($\approx_V$). In altri termini, occorre provare che se si prendono due schedule qualsiasi S_1 e S_2 e si assume per ipotesi che siano conflict-equivalenti ($S_1 \approx_C S_2$), da tale postulato deriverà per forza logica che essi sono anche view-equivalenti ($S_1 \approx_V S_2$).

Procediamo analizzando i due requisiti pilastro della view-equivalenza, servendoci della dimostrazione per assurdo:

1. **Requisito: Stesse scritture finali.** Assumiamo per assurdo che S_1 e S_2 siano conflict-equivalenti, ma *non* possiedano le stesse scritture finali. Se così non fosse, significherebbe che esisterebbero all'interno dei due schedule due

specifiche scritture, orientate su uno *stesso identico oggetto*, che si presentano posizionate in un ordine cronologico inverso o diverso l'una rispetto all'altra. Ma la definizione stessa impone che due operazioni di scrittura (Write-Write) volte su uno stesso oggetto generino istituzionalmente un **conflitto**. Poiché l'ipotesi di partenza certifica che S_1 e S_2 sono conflict-equivalenti, essi per definizione di $\approx_C$ devono obbligatoriamente mantenere il medesimo identico ordine per *tutte* le coppie in conflitto. Questo genera una contraddizione insanabile: due schedule non potrebbero in nessun caso dirsi conflict-equivalenti se presentassero le scritture finali sfasate. Pertanto, il requisito è sempre garantito.

2. **Requisito: Stessa relazione "legge-da" (Reads-From).** Assumiamo per assurdo che S_1 e S_2 siano conflict-equivalenti, ma la relazione "legge-da" *non* sia preservata. Se così non fosse, significherebbe che nel passaggio da uno schedule all'altro l'informazione captata da una determinata lettura sia cambiata. Ciò potrebbe avvenire solo se delle scritture (su uno stesso oggetto) avessero alterato il loro ordine reciproco prima della lettura, oppure se le coppie di lettura-scrittura (sempre su uno stesso oggetto) si trovassero collocate in un ordine diverso. In entrambi i casi si configurerebbe l'inversione posizionale di un conflitto manifesto (un conflitto W-W nel primo caso, o un conflitto R-W/W-R nel secondo). E ancora, se si presentasse l'inversione di una coppia in conflitto, per definizione stessa crollerebbe l'ipotesi di $\approx_C$. La $\approx_C$ sarebbe violata. La contraddizione chiude la dimostrazione: l'inviolabilità dell'ordine dei conflitti blindava e "congela" l'intero tessuto delle letture e delle scritture finali.

La dimostrazione sancisce inequivocabilmente che la conflict-serializzabilità è una condizione logica più stretta, rigorosa e sufficiente per ammettere l'inclusione di un elemento nel grande insieme della view-serializzabilità.

6.2 Il Grafo dei Conflitti e l'Aciclicità

Sebbene l'impianto teorico del CSR ci abbia liberato dall'onere NP-completo di dover tentare tutte le permutazioni della view-equivalenza, l'ingegneria del software richiede un meccanismo computazionale deterministico e algoritmico per validare uno schedule CSR. Questo strumento risolutivo prende il nome di **Grafo dei Conflitti**.

Il grafo dei conflitti è un digrafo (un grafo matematico orientato) che si costruisce astraendo le relazioni temporali di uno schedule in ingresso. La sua mappatura avviene secondo due direttive precise:

- **Nodi:** Si genera un nodo geometrico univoco per ogni singola transazione t_i coinvolta attivamente nello schedule.
- **Archi:** Si traccia un arco orientato unidirezionale che parte dal nodo della transazione t_i e punta al nodo della transazione t_j (con $i \neq j$) se e solo se sussiste almeno un "conflitto" materiale (di tipo R-W, W-R, o W-W) tra un'azione a_i pertinente a t_i e un'azione a_j pertinente a t_j , **con l'aggiunta della condizione fondamentale che l'azione a_i preceda cronologicamente l'azione a_j**

all'interno del flusso temporale dello schedule. La freccia rappresenta dunque il vettore del tempo intersecato all'ostilità.

L'utilità suprema della costruzione del grafo è codificata in un teorema risolutivo:
Teorema: *Uno schedule S risiede e appartiene all'insieme CSR se e solo se il suo corrispondente Grafo dei Conflitti risulta matematicamente ACICLICO (privo di cicli chiusi o loop orientati).*

6.2.1 Dimostrazione del Teorema dell'Aciclicità

La dimostrazione del teorema si regge su un lemma propedeutico vitale: *Due schedule che sono conflict-equivalenti tra di loro ($S_1 \approx_C S_2$) producono e possiedono inequivocabilmente lo stesso identico Grafo dei Conflitti.*

Dimostrazione del Lemma: Per assurdo, si supponga che S_1 e S_2 siano equivalenti ($\approx_C$), ma i loro grafi generati differiscano. Ne conseguirebbe che nel grafo di uno dei due schedule (supponiamo in S_1) dovrebbe esistere tracciato un arco orientato che risulta del tutto assente nel grafo dell'altro (S_2). Siccome ogni arco tracciato è l'espressione diretta e biunivoca di due azioni che in S_1 si trovano in conflitto e in uno specifico ordine cronologico, il fatto che tale arco non compaia o sia invertito nel grafo di S_2 implicherebbe che quelle identiche due azioni in conflitto si trovano nel flusso di S_2 disposte in un ordine ribaltato o diverso. Ma se l'ordine di due azioni in conflitto si inverte, per definizione, la conflict-equivalenza $\approx_C$ si frantuma. La deduzione crea una contraddizione logica insanabile: due schedule equivalenti per conflitti condividono invariabilmente la medesima topologia.

Risolto il lemma, si procede a dimostrare la bi-implicazione del teorema principale (se e solo se).

Parte 1: Se S è in CSR, allora il suo grafo è aciclico. Per definizione costituzionale di CSR, se lo schedule sfuso S appartiene alla categoria CSR, significa che esso deve poter risultare conflict-equivalente ($\approx_C$) a un qualche ipotetico schedule perfettamente e formalmente seriale (denominato convenzionalmente S_0). In virtù del lemma appena provato, lo schedule intrecciato S e lo schedule in fila indiana S_0 condividono il medesimo grafo. Osservando esclusivamente lo schedule puro S_0 (essendo seriale, le sue transazioni t_i avvengono per intero ed in stretta sequenza una dopo l'altra senza accavallamenti), si evince che in S_0 possono nascere conflitti diretti tra un'azione della transazione t_i e un'azione della transazione t_j esclusivamente se t_i è stata servita "prima" di t_j ($i < j$ nell'ordine logico di esecuzione). È materialmente precluso che vi siano archi che tornano indietro nel tempo. La presenza formale di un ciclo orientato in un grafo necessita in modo imprescindibile l'esistenza di almeno un arco di ritorno che vada da un nodo j maggiore verso un nodo i minore ($i > j$). Poiché nello schedule seriale S_0 i vettori viaggiano solo in un'unica direzione, il suo grafo è intrinsecamente e perentoriamente aciclico. Essendo il grafo di S l'esatta copia carbone del grafo di S_0 , anche il grafo di S è inconfutabilmente aciclico.

Parte 2: Se il grafo di S è aciclico, allora S è in CSR. Partendo dalla constatazione che il grafo orientato generato da S risulta aciclico (cioè è un DAG - Directed Acyclic Graph), le leggi della teoria dei grafi assicurano che per un simile reticolo esiste sempre la possibilità di computare ed estrarre un cosiddetto "Ordinamento Topologico". Un ordinamento topologico consiste in una "numerazione

logica" o ridisposizione lineare di tutti i nodi, tale per cui l'intero grafo contenga ed esibisca esclusivamente frecce dirette dal basso verso l'alto (esclusivamente archi orientati da i verso j rispettando costantemente la condizione $i < j$). Costruendo e assemblando artificialmente uno schedule seriale le cui singole transazioni siano fisicamente accodate prestando cieca obbedienza all'ordinamento topologico calcolato, si ottiene una sequenza lineare che risulta per forza di cose "equivalente" al caotico schedule originale S . L'equivalenza sussiste perché in tale architettura tutti i conflitti (i, j) rispetteranno puntualmente e uniformemente la legge vettoriale $i < j$, non contravvenendo a nessun ordine. Si decreta quindi che esiste ed è ricavabile uno schedule seriale equivalente ad S . Il corollario è definitivo: lo schedule intrecciato S merita di stare in CSR.

6.3 Limiti Operativi della Conflict-Serializzabilità nella Pratica DBMS

L'approdo alla conflict-serializzabilità e allo strumento dell'aciclicità tramite grafi topologici costituisce un immenso passo avanti per l'accademia informatica. Il problema NP-completo della view-equivalenza è stato brillantemente declassato. Con il ricorso ad opportune strutture dati algoritmiche (costruzione matriciale e visita in profondità), la costruzione e l'analisi di aciclicità per un grafo di conflitti si tramuta in un'operazione risolvibile con una banalissima e scalabile "complessità lineare".

Nonostante questo successo matematico, la cruda e fredda prassi dell'ingegneria dei sistemi rivela che **anche la conflict-serializzabilità (applicata in modo nudo e diretto) è, in pratica, del tutto inutilizzabile per un motore relazionale reale.**

Il motivo di questa bocciatura sistemistica non è da ascrivere alla lentezza della matematica, bensì ad alcune incolmabili assunzioni e all'architettura temporale del motore ("Però..."):

1. **L'ignoranza del futuro e la natura "incrementale":** L'algoritmo del grafo dei conflitti risulterebbe magnificamente efficiente *se* lo scheduler potesse disporre a priori e conoscere fin dal principio dell'elaborazione l'intera sceneggiatura completa delle operazioni dello schedule (il grafo precostruito). Questa onniscienza non si verifica mai nella vita reale di un server. Lo scheduler è un "guardiano" cieco che opera e valuta le direttive in maniera strettamente "incrementale": riceve una singola richiesta e deve decidere in millisecondi se eseguirla subito, posticiparla o bloccarla. Diventa fisicamente impraticabile (per ragioni di congestione e latenza) l'idea di mantenere persistentemente in memoria viva il grafo, aggiornarlo inserendo dinamicamente nuovi nodi per ogni millisecondo, e lanciare la routine di verifica topologica dell'aciclicità ad ogni singola impercettibile richiesta di input/output.
2. **L'illusione della Commit-Proiezione:** Come accennato nel preambolo, l'intera dottrina algebrica della CSR si fonda tacitamente sulla comoda ipotesi riduttiva della "commit-proiezione", ossia l'assunto irrealistico che nessuna transazione vada in errore o venga annullata (abortita). Ma "le transazioni possono

fallire". Il fallimento asincrono di una transazione, che sfonda nel campo del controllore dell'affidabilità, manda in frantumi la validazione effettuata staticamente dal grafo. Ad esempio, se si trascura la possibilità di fallimento, un sistema CSR potrebbe ammettere incroci temporalmente leciti ma che, a fronte di un **abort** improvviso, esporrebbero inesorabilmente e drammaticamente il sistema alla piaga delle "letture sporche" (se t_i legge i dati di t_k , e t_k fallisce prima del commit dopo che t_i ha già divulgato le informazioni).

La conclusione tecnologica e industriale è inappellabile: l'approccio astratto dei grafi topologici viene relegato all'analisi. In pratica, "si utilizzano tecniche" protocollari ingegnerizzate (hardware e software) che, imponendo vincoli operativi rigidi a livello di risorse, fungono da "guardrail" e **garantiscono organicamente e meccanicamente la conflict-serializzabilità come risultato derivato, senza dover in alcun modo costruire o calcolare il grafo on-the-fly, e svincolandosi per di più dalla falsa ipotesi della commit-proiezione**. Le due grandi famiglie tecniche utilizzate per questo scopo nei RDBMS planetari sono il **2PL (Two-Phase Locking)** e il protocollo timestamp (implementazione del meccanismo "multiversion").

6.4 Il Paradigma Fisico: Locking e Incompatibilità

L'essenza filosofica per incatenare le anomalie e giungere a una schedulazione pulita risiede nell'impiego massivo di "Lock" (lucchetti temporanei). Il "Principio" fondante stabilisce che l'utilizzo di una risorsa (l'accesso ai dati o ad oggetti) deve essere strettamente e gelosamente controllato dal sistema al fine primario di evitare gli aggiornamenti indesiderati e caotici. Questa supervisione introduce leggi di mutua esclusione basate su postulati prestabiliti di compatibilità:

- L'atto di leggere una risorsa è strutturalmente e categoricamente "incompatibile" con l'atto di scriverci sopra. Una sovrapposizione genererebbe un abominio logico.
- Di contro, diverse letture pure (Read-Read), eseguite contemporaneamente da più utenti e che si limitano a consultare, sono del tutto "compatibili" e tollerate tra di loro, in quanto non modificano il tessuto del database.

Il motore transazionale declina questi vincoli imponendo l'obbligo contrattuale secondo cui **ogni singola operazione (DML) debba sempre e per forza essere preceduta nel tempo da una formale e preventiva "richiesta di lock"**. Questa richiesta è lo strumento per evitare l'intersecarsi caotico o la contestualità nociva di letture e alterazioni attive:

- **Lock in Lettura:** Ogni comando applicativo di lettura è tassativamente preceduto dal comando `r_lock` (identificativo di un lock di tipo "condiviso" da più transazioni affini e non invasive) e deve essere concluso formalmente con un rilascio `unlock`.

- **Lock in Scrittura:** Ogni comando volto a impartire una scrittura è tassativamente preceduto dal prepotente comando `w_lock` (identificativo di un lock di tipo "esclusivo", che può essere assegnato a una e una sola transazione dispotica) ed è seguito in coda dall'`unlock` per lo sgombero del privilegio.

Sorge la casistica ibrida: qualora una singola transazione necessiti dapprima di leggere un oggetto e, contestualmente nel prosieguo del calcolo, scriverlo, il DBMS offre due vie: o si richiede per prudenza fin da subito un pesante `w_lock` esclusivo, o, con maggiore scaltrezza programmatica, si acquisisce prima un leggero `r_lock` condiviso e, in prossimità della modifica imminente, si avanza una supplica per un "lock upgrade" verso il livello esclusivo.

6.4.1 Tavola dei Conflitti e Gestione delle Attese (Lock Manager)

Il modulo esecutore per eccellenza preposto all'applicazione pratica è il "Lock Manager". Esso agisce come un impiegato che riceve incessanti richieste d'uso dalle transazioni, e, in base al suo registro e alla compatibilità assiomatica incrociata, delibera un rigido verdetto di accettazione ("accoglie") o respingimento ("rifiuta").

Queste decisioni binarie non sono estemporanee, ma formalizzate nella ineluttabile "**Tavola dei Conflitti**". La matrice contrappone lo stato attuale di impiego in cui verte la risorsa desiderata contro la natura specifica della nuova richiesta inoltrata dal client:

Richiesta ↓ / Stato →	free (libera)	r_locked (in lettura)	w_locked (in scr
r_lock (condiviso)	OK / diviene r_locked	OK / resta r_locked	NO / resta w_lo
w_lock (esclusivo)	OK / diviene w_locked	NO / resta r_locked	NO / resta w_lo
unlock	(gestione anomala)	OK / dipende (contatore)	OK / diviene f

Il Lock Manager traccia tutto orchestrando silenziosamente una complessa e massiccia "tabella dei lock" allocata in memoria temporanea RAM. Nel caso dei lock condivisi (`r_locked`), poiché sono concessi in multiproprietà, il gestore attiva e mantiene aggiornato un "contatore" matematico che enumera con esattezza il numero dei "lettori" simultanei; il vincolo di prigionia sulla risorsa è considerato finalmente estinto, rilasciato ed `unlockato solo` nel frangente in cui l'operazione di decremento porta il contatore della folla di lettori esattamente a scendere fino a zero.

Il Meccanismo del Timeout e della Coda: Se il verdetto generato dalla tavola risulta essere negativo ("NO", la risorsa è occupata o concessa in blocco), la transazione richiedente non viene terminata e gettata via, ma viene formalmente e bruscamente arrestata e "posta in attesa" in una coda invisibile (sospensione o sleep).

Il periodo di quarantena nella coda non è tuttavia infinito. Ad ogni richiesta inevasa di lucchetto, l'architettura relazionale applica in automatico una timer-bomba: il cosiddetto "Timeout" (un parametro di configurazione indicante il tempo soglia e massimo di attesa). Il fluire del timer stabilisce due ipotetici scenari per la transazione paralizzata:

1. La risorsa contesa improvvisamente si libera o va in `unlock` da parte degli occupanti storici, prima dello scoccare inesorabile del timeout. Il lock manager si risveglia e la transazione "può acquisire" il privilegio di lucchetto tanto agognato e proseguire il calcolo applicativo (è specificato il termine "può" anziché "acquisisce certamente", in quanto il sistema potrebbe dover dirimere priorità, considerando che nella medesima coda ci potrebbero essere decine di "altre transazioni in attesa" assemblate simultaneamente per l'accaparramento).
2. Il tempo limite e cronometrico di scadenza del "timeout" scocca senza che la liberazione si sia materializzata in modo propizio. Il gestore, in quanto implacabile regolatore di congestione, deduce e decreta il fallimento, e la transazione in coda viene impietosamente stroncata (abortita). Qualora fosse strategica, essa potrebbe eventualmente "essere rilanciata" da zero dal codice, seppur questa prassi di retry incondizionato resti vincolata a un'esclusiva "scelta applicativa" demandata all'intelligenza del client di programmazione.

6.5 Il Fallimento del Lock Puro e la Genesi del Protocollo 2PL

La dotazione e disseminazione dei lock descritta precedentemente non possiede, singolarmente presa, la proprietà alchemica di risanare le anomalie. Un utilizzo naif di blocchi `w_lock` e rilasci `unlock`, seppur proteggendo gli accessi millimetrici e localizzati alle celle, non argina il cancro del Lost Update.

"I lock da soli non bastano". Si riproduca lo schedule infetto della perdita di aggiornamento, adornandolo ora, fittiziamente, con i soli lock elementari:

$$\begin{aligned}
 t_1 &: \text{rlock}_1(x) \\
 t_1 &: r_1(x) \\
 t_1 &: x = x - 1000 \\
 t_1 &: \text{unlock}_1(x) \\
 t_2 &: \text{rlock}_2(x) \\
 t_2 &: r_2(x) \\
 t_2 &: x = x - 1000 \\
 t_2 &: \text{unlock}_2(x) \\
 t_1 &: \text{wlock}_1(x) \\
 t_1 &: w_1(x) \\
 t_1 &: \text{unlock}_1(x) \\
 t_1 &: \text{commit} \\
 t_2 &: \text{wlock}_2(x) \\
 t_2 &: w_2(x) \\
 t_2 &: \text{unlock}_2(x) \\
 t_2 &: \text{commit}
 \end{aligned}$$

Seppur il Lock Manager abbia governato l'alternanza assecondando le richieste (avendo t_1 e t_2 educatamente rilasciato i rispettivi lucchetti **rlock** non appena terminata la lettura locale, e avendo rispettato diligentemente il turno durante la richiesta del **wlock** non interscambiabile), l'architettura logica ha fallito la sua pretesa e "il problema resta". Non è cambiato nulla. La lettura asincrona sfalsa i calcoli che produrranno una scrittura omicida e disgiunta in un punto di non ritorno, divorando la transazione avversaria.

Per tramutare il modesto strumento del lock in un'infallibile armatura di serializzabilità, il sistema ingegneristico globale innalza il **Locking a Due Fasi (2PL - Two Phase Locking)**. Il 2PL erige il suo protocollo non solo richiedendo in modo ossessivo la protezione con lock prima di tutte le manipolazioni (letture/scritture), ma introducendo d'imperio un dogma monolitico e comportamentale, un "vincolo sulle richieste e i rilasci". Il postulato assoluto e inossidabile del 2PL statuisce che: **una transazione, dopo aver iniziato le pratiche di rilascio cedendo o abbandonando un primissimo lock pre-acquisito, diviene categoricamente non abilitata ad acquisirne di nuovi o ulteriori.**

Da tale restrizione discende il fatto che l'intera esistenza della transazione debba spaccarsi fisicamente e visivamente in "due fasi" non sovrapponibili e non mescolabili:

- **Prima Fase (Fase Espansiva / Acquisizione):** La transazione opera acquisendo, accaparrandosi e monopolizzando a cascata tutti i lucchetti relazionali di cui il suo calcolo potrebbe aver bisogno. Lo stato della transazione "cresce" in possesso.
- **Seconda Fase (Fase di Ritiro / Rilascio):** Al momento in cui il processo espleta un qualsiasi comando di rilascio **unlock**, la fase di crescita è chiusa irreversibilmente. La transazione scivola nel baratro della seconda fase, e, da quel nanosecondo, deve ritirarsi, spogliarsi di poteri e restringere progressivamente e docilmente le sue pretese o, semplicemente, scappare senza più intercedere col gestore, ma mai domandare nuovi presidi.

Questo protocollo comportamentale garantisce meccanicamente la celeberrima conflict-serializzabilità. Il 2PL incarna il modello della "condizione sufficiente": esso certifica irrevocabilmente che **ogni e qualsiasi schedule ammesso all'esecuzione dal rigore del 2PL è, per definizione e con matematica certezza, conflict-serializzabile (CSR).**

6.5.1 2PL vs CSR: L'Inclusione Insiemistica

Al pari del rapporto tra CSR e VSR, l'insiemistica definisce una demarcazione: *Ogni schedule certificato 2PL è in CSR, ma non necessariamente o obbligatoriamente è vero il processo viceversa.* Il controesempio per testare la "non necessità" del protocollo 2PL si materializza analizzando:

$$S_{viola} : r_1(x) \ w_1(x) \ r_2(x) \ w_2(x) \ r_3(y) \ w_1(y)$$

Se sottoponessimo tale sequenza S_{viola} a un banale solutore o grafo dei conflitti appureremmo con serenità che esso, sotto l'esclusivo profilo matematico, si inquadra

e si qualifica come sanamente e aciclicamente *Conflict-serializzabile*. Tuttavia, nonostante la purezza dell'aciclicità, un vero e proprio "scheduler 2PL hardware/software NON lo può produrre né elaborare mai". Perché uno scheduler impazzirebbe bloccandolo sul nascere? Osservando le propaggini del calcolo per la transazione t_1 , notiamo che in apertura t_1 effettua $r_1(x)$ e $w_1(x)$, e poi cede momentaneamente la scena a t_2 . Avendo t_2 letto e scritto x (attraverso le azioni $r_2(x)$ e $w_2(x)$), vuol dire per logica ferrea che la transazione madre t_1 aveva proceduto al "rilascio e cessione dei lock su x " permettendo il subentro di t_2 . Ed ecco consumarsi il tabù del dogma 2PL: essendosi denudata di un lucchetto su x , t_1 è formalmente precipitata nella fase irrevocabile di "rilascio". Purtroppo, a fine corsa dello schedule, subentra imprevedibilmente un'istruzione $w_1(y)$: la transazione t_1 tenta subdolamente di "acquisire un nuovo strato lock" esclusivo verso l'oggetto y . Questa richiesta post-rinuncia infrange in maniera rovinosa il 2PL ed il controllore rigetterebbe l'intero blocco abortendo e stroncando t_1 (Viola 2PL).

6.5.2 Il Limite Mortale: 2PL e Stallo (Deadlock)

L'impiego pervasivo della prima fase esplosiva e accumulatrice di lucchetti indotta dal Two-Phase Locking nasconde un'insidia algoritmica mortale. Il protezionismo estremo genera il fenomeno delle "**attese incrociate**": un imbuto logico dove due o plurime transazioni isolate detengono gelosamente e in modo asincrono una risorsa (rifiutandosi dogmaticamente di rilasciarla per non entrare anzitempo nella Fase due del 2PL), e, simultaneamente, anelano e "aspettano passivamente l'acquisizione in blocco della risorsa precedentemente e fatalmente detenuta e bloccata dall'altra concorrente".

Questo stallo perpetuo o paralisi inestricabile è denominato **Deadlock**. Esempio d'intercettazione disastrosa:

$$\begin{array}{lcl} t_1 : r(x) & \text{e} & w(y) \\ t_2 : r(y) & \text{e} & w(x) \end{array}$$

Avviando questo schedule con un parser: t_1 si prenota con `r_lock1(x)` ed egualmente ma in differita t_2 avanza `r_lock2(y)`. t_1 invoca il `w_lock1(y)` per scrivere su y , subendo l'esito avverso dal Manager per l'occupazione preesistente di t_2 . La transazione t_1 è spedita nella coda dell'attesa ("timeout"). Inconcepibilmente anche t_2 procede alla chiamata `w_lock2(x)`, cozzando contro l'egemonia preesistente di t_1 . Anche t_2 sosta in attesa. t_1 aspetta in ginocchio che t_2 rilasci, ma t_2 aspetta altrettanto in ginocchio e con ostinazione lo sblocco di t_1 . L'impasse si tramuta in blocco silente di server: un cataclisma architetturale.

Il DBMS, appurata l'anomalia, interviene chirurgicamente avvalendosi delle tre macrotecniche per la "Gestione dello Stallo" (ove lo stallo non è altro che un diabolico ciclo geometrico impresso in un grafo ausiliario, il "grafo delle attese", in cui il nodo ricalca la transazione oppressa, mentre l'arco funge da traccia dell'attesa):

1. **Timeout**: Come precedentemente illustrato, lo scadere asettico dell'orologio miete la vita di una delle due concorrenti. Il problema infame del timeout è la calibratura del "trade-off" per la scelta e l'estensione dell'intervallo ideale

(intervalli brevi decimano transazioni altrimenti fluide, intervalli spropositati infliggono tempi biblici agli utenti incatenati).

2. **Rilevamento strutturato (Detection):** La mente diagnostica del motore esegue periodicamente asettiche elaborazioni in background (grafi ciclici) a "ricerca di cicli" nel grafo delle attese pendenti, per snidare e colpire i loop bloccati.
3. **Prevenzione profilattica dello stallo (Prevention):** Strategia draconiana attuata sopprimendo a priori in forma preventiva l'"uccisione" e decapitazione rapida di eventuali transazioni tacciate di essere potenzialmente "sospette", ancor prima di palesare la chiusura dell'anello. Metodo brutale, soggetto ad un noto margine d'errore (può facilmente "esagerare" sprecando buone iterazioni).

6.6 Locking a Due Fasi Stretto (S2PL) e Fine dell'Utopia

Un'ulteriore complicazione, affrontata dai grandi calcolatori che impiegano 2PL, ricade nel rischio delle "Lettture Sporche", non scongiurate dal rilascio immediato o asincrono. Si ricordi il monito che la "commit-proiezione" non sussiste, le transazioni purtroppo falliscono ("rimuoviamo l'ipotesi di commit-proiezione"). Se t_1 cede un lock in corso d'opera (legittimamente concesso nella sua sacra Fase 2 di rilascio imposta dal protocollo 2PL) per via di una sbloccante elaborazione completata e, dopo pochi cicli clock, la sfortunata va drammaticamente incontro a un tracollo o fallimento irreversibile causato da altre anomalie logiche periferiche o da timeout, la cessione prematura del lock provocherà l'anomalia aberrante se, nel medesimo frattempo, una fortunata t_2 aveva agguantato o "andata in commit" proprio quei frammenti e dati falsi scritti poc'anzi da t_1 (e veicolando e dischiudendo, orrore degli orrori, la comunicazione errata all'universo e agli I/O "all'esterno").

Come risolvere l'equazione per evitare la piaga delle letture sporche indotte dai rimasugli? La contromisura dogmatica è l'impedimento radicale per ogni client: "impedendo il commit e la perversa divulgazione di comunicazione all'esterno" di qualsivoglia dato in pendenza, finché la transazione garante che ha partorito quei parametri non giunga e non sia andata effettivamente ed integralmente nel paradiso del commit.

La soluzione e derivazione algoritmica "semplice e diretta" che scongiura la nefandezza del dato fantasma si riassume nell'assunto lapidario "non fare o imporre mai di leggere e assorbire dati sporchi". Questa preghiera e ingiunzione programmatica si formalizza architetturealmente nell'estensione del protocollo principe, battezzando ed originando lo strumento del **Locking a due fasi stretto (Strict 2PL o S2PL)**. Il paradigma S2PL recepisce il dictat base del progenitore (il binomio fase uno/fase due) per innestarvi un fendente limitativo di spaventosa rigidità: **"i lock (specie in stesura esclusiva o scrittoria) possono inequivocabilmente essere liberati o rilasciati SOLO, SOLTANTO ED ESCLUSIVAMENTE ad opera chiusa, ovvero in maniera contemporanea o susseguente al consolidamento dello shutdown o approdo finale imposto per commit o abort"**.

La stratificazione gerarchica delle proprietà del rigore dei grafi, culminante nell'anello S2PL, riproduce la seguente matrisca insiemistica: L'universo totale di tutti i possibili mischiati **Schedule** abbraccia ed ingloba il reame protetto ma incalcolabile per tempi CPU, ovvero lo **Schedule VSR**. Il campo VSR, come acclarato, abbraccia ed ospita le file controllabili colla teoria del grafo dei conflitti (**Schedule CSR**). All'interno o per sineddoche all'intorno della sfera matematica CSR germoglia e attecchisce l'insieme ingegneristico di programmazione in hardware denominato **Schedule 2PL**. Quest'ultimo incuba rigidamente al proprio interno la cruna dell'ago perfezionata dei sistemi attuali contro l'infezione della ghost-read, lo **Schedule S2PL**. Infine, l'occhio cieco ristretto ed impervio dell'immacolato alveo senza incroci o parallelismi: lo specchio puro degli **Schedule Seriali** chiude il diagramma.

6.7 Integrazione Complessa: Granularità, Isolamento Reale e Lock di Predicato

Non vi è reale comprensione informatica se il modello teorico 2PL Stretto non è integrato nell'ecosistema globale di persistenza delle moderne architetture RDBMS.

La scalabilità di un DBMS pretende che il Lock non sia limitato a un'istanza molecolare, bensì esteso con "diversi livelli di granularità e gerarchie". Un nodo lock in produzione ingegneristica non incatena genericamente "la lettera x", bensì può agire progressivamente o estensivamente sull'astrazione strutturata di: *un'intera immensa base di dati, un plico tabelle interconnesse, frammenti contigui di volume, raggruppamenti multipli di blocchi disco magnetici sino a degenerare nella finezza chirurgica imposta dalla singola minuscola riga o "ennuple" registrate nella tabella*. L'esistenza di siffatta "gerarchia fra le varie granularità" è il volano per arginare le carenze hardware: "Perché?" ci si espande? Semplice pragmatismo di performance.

L'applicazione e il bloccaggio condotto con "granularità grossa" (e.g. blocco di una tabella intiera) possiede lo spregevole difetto di immobilizzare inibitoriamente moltissime preziose sotto-risorse (saturando lo scheduler), tuttavia la "granularità fine" (sfruttata ad oltranza bloccando record per record a colpi decuplicati di timeout) sversa e richiede computazionalmente al Lock Manager in CPU l'allocazione massiva e il calcolo ingestibile di una galassia e miriade incalcolabile di innumerevoli e piccoli "molti lock", rischiando di annientare la memoria vitale in stallo e tracking index. I sistemi eludono la morsa del caos ingegnerizzando dinamici e reattivi "meccanismi di escalation". Ad esempio logico: lo scheduler in back end, allorquando monitora che le chiamate sfociano e "se ho accumulato o asservito a lock o troppi lock mirati per una pletora estesa di ennuple adiacenti per una data singola transazione, autonomamente recide i blocchetti individuali per compiere o passare fluidamente verso l'istituzione e promulgazione globale di un lock univoco di maglia più tozza, che imprigioni l'intera macro-relazione, agevolando l'albero interno di log".

6.7.1 Implementazione Pratica dell'Isolamento e Inserimento Fantasma

Il banco di prova ultimo per il controllore 2PL risiede nella trasposizione e nell'attivazione a runtime dei Livelli SQL:1999 summenzionati (Read Uncommitted, Read Committed, Repeatable Read, Serializable).

Una preconditione inaudita per l'integrità strutturale: a prescindere dal livello di visibilità scelto per gli sguardi o per la consultazione utente, **sulle e per le Scritture (Write Operation) "si ha sempre l'imperioso obbligo protocollare del 2PL stretto" in qualsivoglia architettura seria e relazionale, inibendo in senso lato la commutazione del rilascio anzitempo.** (Tuttavia "nonostante ciò che dicono le favole in svariati manuali generici", tale 2PL sulle scritture non esime e non previene matematicamente l'atroce e tragica "perdita d'aggiornamento": "la sovrascrittura si evita unicamente e solo se le transazioni in scrittura assurgono ed esigono perentoriamente all'interno dello script applicativo il dispiegamento del livello massimo di isolamento logico serializzabile").

Esaminiamo la modulazione meccanica per i Lock a fine lettorio (Read):

- **read uncommitted:** L'incuria del "nessun lock in lettura", disdegnando la regola elementare e non palesando timore alcuno: così operando si permette che l'algoritmo irrompa ciecamente nei dati volatili (e quindi ignora e non calcola, né tantomeno "rispetta i lock" severi e ostili apposti e lasciati ad ammuffire dagli "altrui" processori attivi).
- **read committed:** S'impone l'apposizione cautelare dell'effimero "lock in lettura" (e, in tal modo, incrociando i binari con la tavola compatibilità, si blocca fermamente la transazione, in quanto "rispetta reverenzialmente i lock opposti in scrittura" in corso d'opera attivati). Ma, attenzione letale, il lucchetto è sganciato all'istante o rimosso "senza ottemperare la prigionia al 2PL", disinnescando o sbloccando repentinamente dopo la prelievo del valore la morsa o lock e scappando verso altra istruzione asincrona.
- **repeatable read:** Il salto nell'isolamento robusto è dettato dall'introduzione dogmatica del famigerato "2PL stretto, imposto a sorpresa e severamente anche per le esecuzioni in fiera lettura, corredandolo o abbinandolo a costrizioni lock applicati perennemente sui dati puntuali e sulle tuple prelevate".

Ma il castello e l'architettura 2PL non possono contenere il baratro metafisico dello spaventoso **"Inserimento Fantasma"** o delle anomalie indotte dalle interrogazioni spaziali. Si riprenda il sistema di prenotazione: se la query impartita conta massivamente "i posti disponibili per il volo X" ricorrendo alla decodifica di un costrutto aggregativo `select ... from ... dove condizione parametrica: volo=X`, un lock 2PL convenzionale, apposto stoicamente sulle ennuple esistenti e bloccate con valore X, non sortirà preclusione o scudo alcuno per il fatto inaudito che un "aggiungi o un aereo più capiente" sfoci per inserire e far piovere materialmente dal nulla nuove tuple o record inediti estranei al reame del lock precedente. "I lock visti finora non bastano, perché materializzano o ci sono fioriture di record nuovi, mai incatenati da un check".

Il rimedio architetturale ultimo del massimo livello formale **serializable** trascende e devia dal controllo di tuple: non è sufficiente un 2PL Stretto, occorre innestarvi in combinazione sinergica e fatale il concetto assoluto di "Lock di Predicato". Il controllore decreta un congelamento algoritmico o "Lock di predicato innestato brutalmente sulla condizione astratta `volto=X`". Il controllore, applicando tale regola preclusiva, "impedisce materialmente e fisicamente l'accesso sconsiderato per qualsiasi operazione mutante e di scrittura" e su "tutti, perennemente i dati incogniti che astrattamente soddisfano logicamente le clausole o il predicato matematico e relazionale usato o sfruttato originariamente nella fase d'apertura o vettoriale di una lettura aggregata".

L'implementazione o concretizzazione algoritmica di queste astrazioni è devastante e ondivaga: "Come si implementano operativamente i lock di predicato nel disco fisico?"

- Nel **Caso peggiore** (assenza del file strutturato B-Tree), la clausola si propaga e impantana la granularità bloccando l'incursione di lock mostruoso "sull'intera relazione (tabella) e dominio", bloccando il traffico dell'infrastruttura.
- Se tuttavia e fortunatamente "esiste un indice strutturale, b-tree, o cluster orientato sugli specifici attributi relazionali menzionati e setacciati nella condizione **where**", il lock mutante intercetterà i settori e si aggancerà in modo snello e letale, ancorandosi "sull'indice stesso".

Capitolo 7

Controllo di Concorrenza Avanzato: Timestamp e Multiversione

7.1 Il Controllo di Concorrenza basato su Timestamp (TS)

Il protocollo 2PL, pur essendo granitico e dominante nella prima era dei database relazionali, soffre dell'intrinseco limite del pessimismo e della propensione fisiologica al blocco tramite dead-lock. L'ingegneria informatica ha pertanto elaborato una "tecnica alternativa al 2PL", fondata su un paradigma radicalmente opposto: il controllo di concorrenza basato su **Timestamp (TS)**.

Il Timestamp è formalmente definito come un identificatore univoco e monotonicamente crescente che stabilisce e "definisce un ordinamento totale sugli eventi di un sistema" distribuito o centralizzato. L'intuizione logica alla base di questo approccio è associare il concetto di isolamento non all'acquisizione di lucchetti spaziali, bensì a una collocazione temporale pura: ad ogni singola transazione, nel momento in cui essa nasce, viene "stampigliato" un timestamp che ne "rappresenta l'istante di inizio". (Questa intuizione ricorda, con un parallelismo cinematografico, le logiche della timeline affrontate in Ritorno al Futuro e il "Paradosso del nonno", in cui gli eventi passati non possono essere alterati retroattivamente senza generare un'incongruenza o distruggere il presente).

Lo scheduler governato dal protocollo TS non mette in coda nessuno a priori, ma funge da guardiano temporale: uno schedule intrecciato viene "accettato" e condotto a termine se, e solo se, esso "riflette (nei conflitti R-W, W-R, W-W) l'ordinamento seriale delle transazioni indotto dai timestamp" nativi. In altre parole, se la transazione più vecchia accede a un dato che una transazione più giovane ha già furtivamente sovrascritto, il sistema rileva un anacronismo (il paradosso). Essendo basato sull'ordine dei conflitti, la teoria certifica che "ogni schedule accettato da TS è anche CSR (Conflict-Serializzabile)".

7.1.1 Meccanica ed Etichettatura degli Oggetti

Per poter rilevare gli anacronismi, il protocollo TS non necessita di mantenere un grafo centrale, ma delega la memoria del tempo ai dati stessi. Per ogni singolo

oggetto x della base di dati, il sistema alloca e aggiorna continuamente due variabili di stato interne (metadata):

- $RTM(x)$ (*Read Timestamp*): Rappresenta il timestamp della transazione più "giovane" (quella col valore numerico di timestamp più alto, arrivata per ultima) che ha coronato con successo l'operazione di lettura del dato x .
- $WTM(x)$ (*Write Timestamp*): Rappresenta il timestamp della transazione più "giovane" che ha effettivamente alterato e scritto il dato x .

Lo scheduler riceve a raffica richieste di lettura $r_t(x)$ e scrittura $w_t(x)$, dove l'indice t indica il timestamp nativo della transazione richiedente.

Gestione delle Letture $r_t(x)$: Quando la transazione con timestamp t desidera leggere l'oggetto x :

- Se si verifica la disuguaglianza $t < WTM(x)$: Significa che la transazione t (vecchia) sta cercando di leggere un dato che è già stato rimpiazzato o sovrascritto nel "futuro" da una transazione più giovane. La lettura di un dato fantasma comprometterebbe la serialità. La richiesta viene brutalmente "respinta e la transazione viene uccisa" (abort immediato).
- Altrimenti ($t \geq WTM(x)$): La richiesta è cronologicamente valida. Viene "accolta". Il sistema aggiorna lo storico dell'oggetto ponendo $RTM(x)$ "uguale al maggiore fra $RTM(x)$ e t ".

Gestione delle Scritture $w_t(x)$: Quando la transazione con timestamp t desidera scrivere sull'oggetto x :

- Se $t < WTM(x)$ oppure $t < RTM(x)$: Significa che la transazione t sta tentando di sovrascrivere un dato che una transazione più giovane ha già scritto in modo definitivo (Late Write, scatenando W-W) oppure che una transazione più giovane ha già letto assorbendone il valore logico per i propri calcoli (scatenando un anacronismo in lettura R-W). In entrambi i frangenti la richiesta è fatale e viene "respinta e la transazione viene uccisa".
- Altrimenti: La richiesta temporale è lecita, "viene accolta" e la metavariabile $WTM(x)$ è posta insindacabilmente "uguale a t ".

Per funzionare in purezza nei sistemi reali e aggirare le vulnerabilità del fallimento di transazioni non completate ("senza ipotesi di commit-proiezione"), le primitive di scrittura del Timestamp dovrebbero essere protette "fino al commit" delegando la stesura materiale alla memoria a un modulo simile al "2PL sulle scritture". Tuttavia, per sfuggire all'ingombro del lock, "in pratica si usa però una variante".

7.1.2 La Variante "Thomas Write Rule" (Scritture Ignorate)

Analizzando il rigetto per le scritture, si intuisce un'ottimizzazione. Se una scrittura t giunge in netto ritardo fisiologico rispetto al record $WTM(x)$ (ossia $t < WTM(x)$), ma magicamente e fortunatamente nessuno nel futuro ha ancora letto o fatto affidamento su quel dato superato (ovvero non sussiste $t < RTM(x)$), la vecchia scrittura morta potrebbe "semplicemente essere ignorata" senza scatenare il massacro dell'abort. La regola di ammissione viene dolcemente modificata in:

- Se $t < RTM(x)$: La "richiesta è respinta e la transazione viene uccisa" (L'anacronismo di lettura resta letale).
- Se $t < WTM(x)$ e $t \geq RTM(x)$: La "richiesta viene ignorata". La variabile x manterrà il valore più aggiornato scritto dal futuro, giacché "nessuno ha avuto necessità di leggere quel dato" intermedio e la transazione t può proseguire illesa.
- Altrimenti: La "richiesta viene accolta e $WTM(x)$ è posto uguale a t ".

7.1.3 Esempio di Esecuzione TS

Si tracci l'evoluzione per un oggetto x avente stato iniziale $RTM(x) = 7$ e $WTM(x) = 4$.

1. Arriva $r_6(x)$: $6 > 4(WTM)$. Esito: "ok". RTM resta 7 ($\max(7, 6)$).
2. Arriva $r_8(x)$: $8 > 4$. Esito: "ok". Il nuovo valore diviene $RTM(x) = 8$.
3. Arriva $r_9(x)$: $9 > 4$. Esito: "ok". Il nuovo valore diviene $RTM(x) = 9$.
4. Arriva $w_8(x)$: Il controllo della scrittura valuta $t = 8$. Si ha $8 < 9(RTM)$. Esito catastrofico: "no, t_8 uccisa" (ha tentato di alterare il passato già letto da 9).
5. Arriva $w_{12}(x)$: $12 > 9(RTM)$ e $12 > 4(WTM)$. Esito: "ok". Il nuovo valore fissa $WTM(x) = 12$.
6. Arriva $r_{10}(x)$: $10 < 12(WTM)$. Esito fatale: "no, t_{10} uccisa".
7. Arriva $w_{11}(x)$: Controllo: $11 > 9(RTM)$, ma $11 < 12(WTM)$. Esito biforcuto: secondo il metodo TS "base" la transazione t_{11} viene inesorabilmente "uccisa"; se applicassimo la "variante" (Thomas), la scrittura verrebbe puramente e silenziosamente "ignorata" salvando la vita a t_{11} .

7.2 Confronto di Architettura: 2PL vs TS

Dal punto di vista insiemistico e logico, le due supreme architetture di schedulazione (2PL e TS) sono dichiaratamente "incomparabili". Nessuna delle due fagocita interamente l'altra, poiché coprono porzioni intersecate ma disgiunte dello spazio dei conflitti. L'incomparabilità è dimostrata dalla casistica:

- Esistono "Schedule in TS ma non in 2PL": es. $r_1(x)w_1(x)r_2(x)w_2(x)r_0(y)w_1(y)$ (TS la accetta per successione temporale intatta, ma 2PL la rigetta per il tentativo illegittimo di $w_1(y)$ in Fase 2).
- Esistono "Schedule in 2PL ma non in TS" (con i pedici intesi come identificatori nativi timestamp): es. $r_2(x)w_2(x)r_1(x)w_1(x)$. Un 2PL la accetta serenamente tramite turnazione con lock; il TS impazzisce e uccide t_1 poiché, essendo anagraficamente 1 (più vecchia di 2), si presenta a bussare allo sportello con colpevole ritardo cronologico incappando nel blocco fatale $1 < WTM = 2$.
- Esistono ovviamente "Schedule in TS e in 2PL": la confort zone dell'intersezione (es. $r_1(x)r_2(y)w_2(y)w_1(x)r_2(x)w_2(x)$).

A livello filosofico e di gestione sistemistica, l'incomparabilità si traduce in due approcci comportamentali opposti:

- **2PL è intrinsecamente "pessimista"**: Presume il disastro e gioca d'anticipo, ergendo barricate e "ponendo le transazioni in attesa" cautelativa. L'effetto collaterale è la paralisi lenta e l'induzione latente del fenomeno di deadlock, innescato dallo stallo incrociato dei rami in attesa.
- **TS è intrinsecamente "ottimista"**: Non blocca mai la coda. Fa fluire e "va avanti" ad ogni costo; se poi il calcolo cronologico rivela un'intrusione asincrona a posteriori, funge da mietitore e "uccide le transazioni" senza pietà (le quali dovranno essere intercettate dal client e "possono essere rilanciate"). Benché il deadlock sia minimizzato ("TS anche, ma meno perché non ha i lock in lettura"), lo spreco di cicli CPU per calcoli scartati è alto.

Il dilemma per l'ingegnere dei dati ("Le ripartenze sono più o meno costose delle attese?") ha delineato la storia dell'Information Technology: "Originariamente", data la ristrettezza delle CPU e la penuria di RAM, abortire e ritentare era folle, quindi "quasi tutti i sistemi preferivano il 2PL". "Ora", con l'avvento dei cluster paralleli ad altissima efficienza e connettività, "si stanno diffondendo" massivamente quelli a fondamento temporale, specificamente le varianti "con timestamp e multiversioni". Il compromesso d'oro nei big-player commerciali è divenuto la tecnica ibrida: "usano 2PL per le transazioni che scrivono" (arginando gli omicidi futili) "e multiversioni per le read-only" (garantendo consultazioni analitiche immediate e non bloccanti).

7.3 Controllo Multiversione (MVCC)

L'impalcatura del Timestamp puro è afflitta dal sanguinoso dazio degli abort in lettura. Riprendendo il tracciato precedente dove giungeva in coda la richiesta $r_{10}(x)$, essa veniva condannata a morte impietosamente dall'anacronismo $10 < 12(WTM)$. "È proprio necessario uccidere t_{10} ? Potremmo fare qualcos'altro?" L'uovo di Colombo dell'ingegneria moderna risponde in maniera geniale: anziché sopprimere l'interrogazione, basterebbe avere l'accortezza di fornirle in pasto e fargli "Leggere il valore precedente!" all'aggiornamento.

Nasce così il **Controllo di Concorrenza Multiversioni (MVCC)**. L'"Idea base" sovverte il principio monolitico del dato singolo: "le scritture" distruttive vengono abolite. Esse non sovrascrivono più la cella in loco, bensì "generano nuove copie" fisiche del medesimo record, espanse lungo la linea del tempo; di converso, le interrogazioni e "le letture leggono la copia "valida" per la transazione" richiedente.

In un siffatto ecosistema, "ogni scrittura genera una nuova copia con un WTM (Write Timestamp) proprio", comportando che, "in generale, ci sono più versioni di un oggetto, con timestamp di scrittura diversi". Le transazioni possono coesistere sfogliando le diverse fotografie temporali. (Mentre permarrà la metavariabile unica $RTM(x)$ per l'oggetto, per rilevare gli storici).

7.3.1 Meccanica della Variante Multiversione Pura

Il motore gestisce l'arsenale delle copie parallele con il seguente regolamento:

- **Gestione della Lettura** $r_t(x)$: È il trionfo dell'ottimismo; in un ambiente MVCC puro, una transazione di lettura "è sempre accettata" a prescindere dal ritardo cronologico. Lo scheduler si erge ad orologiaio e provvede a sfogliare il carosello delle versioni, selezionando unicamente e servendo la singola specifica copia x_k che gode delle seguenti prerogative:
 - Se il timestamp in ingresso t risulta clamorosamente e numericamente "maggiore di tutti i timestamp delle copie" esistenti nel database, si deduce che è una richiesta d'avanguardia e "allora viene scelta l'ultima versione" disponibile.
 - Altrimenti, lo scheduler naviga a ritroso e "si sceglie la versione k tale che $WTM_k(x) < t < WTM_{k+1}(x)$ ", offrendo alla transazione lenta l'esatta istantanea temporale del dato prima che esso venisse alterato dal passo $k + 1$.
- **Gestione della Scrittura** $w_t(x)$: Essendo creatrice di cronologia, mantiene un perimetro di rigidità. Se $t < RTM(x)$ (ossia la transazione sta provando sadicamente a inserire nel passato un dato per rimpiazzare una fotografia già estratta in via definitiva da RTM), l'inserimento anacronistico non è collocabile e "la richiesta è rifiutata". "Altrimenti", nell'andamento sano, "una nuova versione di x viene generata e stoccata, marchiata permanentemente con $WTM(x) = t$ ".

Il database non può accatastare versioni obsolete all'infinito per via dello Storage. L'architettura prevede il processo di garbage collection interna (il "VACUUM"): "Versioni precedenti vengono eliminate" e falciate dai cluster fisici del disco non appena lo scheduler si avvede che "nessuno è più interessato a leggerle" (ovvero il timestamp di tutte le transazioni vive e operative risulta superiore e staccato rispetto a quello delle copie decrepite).

Un caveat per l'ingegnere in aula: questa affascinante dinamica purista del timestamp e "Attenzione: non è proprio questa la tecnica che si utilizza in pratica!" a livello nativo e puro. I colossi, come ad esempio PostgreSQL, ne fondono il DNA "in una variante".

7.4 Multiversione MVCC in PostgreSQL (Analisi e Anomalie)

La mappatura delle teorie accademiche incontra compromessi radicali nei "Controllo di Concorrenza in DBMS reali":

- Piattaforme storiche come **DB2** adottano come architettura blindata e fondante il "sostanzialmente 2PL stretto" su tutti i fronti (sebbene annegato in una galassia di dettagli interni d'ottimizzazione).
- L'ammiraglia open source **PostgreSQL** scommette anima e core sul motore "multiversione, con una condizione aggiuntiva".

La piaga comune di questi due colossi è permettere l'impostazione di "diversi livelli di isolamento anche per le transazioni che scrivono", configurazione pericolosissima e lascia che "è cosa che non si dovrebbe fare" mai se si vuole evitare in radice l'abominio della Lost Update.

7.4.1 Livelli di Isolamento PostgreSQL implementati su Multiversion

L'unione dell'SQL con MVCC in Postgres partorisce dei comportamenti altamente idiosincratici e distintivi:

1. **Read Committed (Default Postgres):** (Nota: In Postgres, il livello nudo "Read Uncommitted" a norma SQL "non è materialmente implementato", ed invocandolo esso retrocede tacitamente e "si comporta nello stesso modo" tutelato del Read Committed). In questo livello base, lo snapshot per le Letture opera magicamente "su dati in commit al momento della lettura". In termini MVCC significa che ogni singola istruzione **SELECT** avulsa esegue un ricalcolo dello specchio e "vede solo dati andati in commit prima dell'inizio della SELECT stessa" (non dell'avvio della transazione macro, ma della query specifica in fieri), sommandovi per ovvietà contestuale "gli aggiornamenti della transazione in cui è inserita" per preservare la coerenza interna di ramo. Di contro, le manipolazioni e "Scritture" restano asservite al "2PL stretto"; ciò significa che richiedono un lock monolitico pesante che "li tiene fino al commit; e rispetta obbedientemente i lock altrui" restando pietrificate nelle code di attesa (timeout).
2. **Repeatable Read:** Eleva enormemente la stabilità dello specchio. Qui le "Letture multiversion" sono fissate e congelate indissolubilmente "sui dati andati in commit all'inizio della transazione", e non ricalcolate per ogni select. La fotografia panoramica del DB è perenne e invariabile per l'intero arco vitale, sommandovi solo i delta interni della "transazione in cui è inserita". Sulle "Scritture" (governate da 2PL Stretto), il motore MVCC impone il feroce "Rispetto delle versioni": il controllore statuisce che "una transazione T non può mai, sotto nessuna giustificazione, avere lock o modificare proattivamente

dati che siano stati proditoriamente modificati da un'altra aliena transazione T' in un'epoca temporale *dopo* l'avvio della stessa T". Se si tenta tale sovrascrittura spaziotemporale errata, PostgreSQL innesca un fatal error denominato *serialization failure*. (Nota bene ingegneristica per questo scudo: Il DB in architettura MVCC "considera come inizio della transazione non l'istante vuoto del comando BEGIN TRANSACTION, ma quello effettivo della *prima lettura o scrittura della transazione*" ove lo snapshot primario viene instradato e istanziato).

3. **Serializable:** L'olimpo isolativo. Reitera perfettamente le prerogative del Repeatable Read per lo specchio immacolato (Lecture multiversion "su dati in commit all'inizio della transazione" più gli update di sessione) e reitera il feroce "rispetto delle versioni" per il blocco di W-W ("una transazione T non può modificare dati modificati da T' dopo l'avvio di T"). Ma l'aggiunta salvifica per sconfiggere i fantasmi R-W e le inconsistenze incrociate asincrone è l'introduzione attiva della "Verifica di cicli di conflitti lettura/scrittura" ("read/write dependencies among transactions"). Introdotta e resa operativa solo tardivamente a partire dalla "versione 9.1" di PostgreSQL, questa funzionalità non paralizza brutalmente l'infrastruttura con il temuto ed esoso index-locking per predicati astratti; al contrario, essa è "Implementata subdolamente con una variante di lock di predicato". Questo algoritmo invisibile "ricorda e traccia minuziosamente l'ordine delle operazioni e le dipendenze, *senza fisicamente bloccare* i rami in esecuzione". La resa dei conti si sposta drammaticamente in coda: essa viene "verificata aspramente all'atto esatto del commit". Se lo scheduler detecta una "write skew" distruttiva o se deduce retroattivamente che "più transazioni hanno scritto" e generato incroci R-W incostituzionali capaci di violare lo spartito di uno scheduler seriale emulato, innesca a freddo l'aborto espiatorio, abortendo il commit senza l'uso preventivo di blocchi d'attesa pesanti.

7.4.2 Esercitazioni e Dinamica Postgres VS 2PL (Scenari a confronto)

"L'esercizio e la progettualità sono determinanti per studiare e sperimentare il comportamento di uno o più DBMS (Postgres in primis) nella gestione dei livelli d'isolamento". Omettiamo di descrivere i compiti esplorativi JDBC, per addentrarci nei calcoli d'esame cruciali (come gli Esempi 1bis, 1quater e "27/05/2019"), che confrontano la tenuta del MVCC Postgresiano contro un paradigma 2PL liscio.

Si valuti il famigerato **Scenario "Esempio 1bis": Perdita di Aggiornamento**.

```

 $t_1$  : Start (livello isolamento XXX)
 $t_2$  : Start (livello isolamento XXX)
 $t_1$  : Read( $x$ )
 $t_2$  : Read( $x$ )
 $t_1$  :  $x = x + 100$ 
 $t_1$  : Write( $x$ ) → Commit
 $t_2$  :  $x = x + 1$ 
 $t_2$  : Write( $x$ ) → Commit

```

Cosa avviene con il rigore del 2PL ("a mano" e in DB2)? Se questo schema viene calato in un ecosistema 2PL (simulando "Esempio 1bis con 2PL" o "Esempio 1quater con 2PL" con isolamento "Serializable"), le prime letture incrociate passano grazie a `r_lock` compatibili. Il tracollo in 2PL avviene quando si affacciano le scritte. Quando t_1 emette `Write( $x$ )` pretende e chiede disperatamente la mutazione in lucchetto esclusivo `w_lock`, negato perché t_2 ha in pancia un lucchetto di lettura. t_1 si blocca in attesa. Disgraziatamente t_2 farà lo stesso gioco, chiedendo `Write( $x$ )` e l'esclusiva. Le due sorelle formano "attese incrociate": si infrangono nello stallo. *Confronto: Il 2PL "va in stallo" (Deadlock).*

Cosa avviene con MVCC (Postgres)? Se eseguito su terminali Postgres (con tabelle `conti`), imponendo il massimo vigore (`isolation level serializable` per entrambe o persino `repeatable read`), le due letture asincrone scorrono felici ed estrapolano dalle memorie la medesima istantanea intatta (es. *Saldo* = 0). Quando t_1 sferre l'update e chiama `Write( $x + 100$ )` e contestualmente chiude con `commit`, tutto fluisce con successo. L'anomalia emerge deflagrando al passaggio di t_2 . Quando t_2 , forte della sua obsoleta fotografia iniziale, emette `Write( $x + 1$ )`, il muro del MVCC interviene. Entra in gioco la clausola ferrea del MVCC sui livelli superiori: "una transazione non può modificare dati che siano stati modificati da un'altra transazione T" dopo l'avvio della transazione stessa". Siccome lo snapshot di t_2 era vecchio e l'oggetto x è stato deturpato asincronamente da t_1 , Postgres impietosamente emette un abort di sicurezza. *Confronto: MVCC (Postgres) non va mai in stallo e "gestisce la situazione, abortendo una transazione" ("troppo ottimista, ha fatto avviare entrambe le transazioni e non può portarle a termine, almeno non entrambe").*

"Nei casi Serializable e Repeatable Read, la seconda transazione fallisce, perché il dato è stato modificato dalla prima, dopo l'inizio della seconda".

Tuttavia, un cedimento strutturale avviene e dilania l'MVCC Postgresiano **se l'operatore abbassa negligenemente l'isolamento a livello READ COMMITTED ("Esempio 1-RC bis")**. Sotto RC, la perentoria salvaguardia "rispetto delle versioni" cade. Quando t_1 esegue `commit` su $x = 100$, il sistema concede luce verde. Quando t_2 tenta l'assalto con $x = 1$ (frutto della lettura sporca su base ricalcolata), Postgres, seppur costringendo t_2 temporaneamente al blocco dietro il 2PL stretto della scrittura della controparte in attesa del `commit`, la lascia poi sfogare ciecamente alla riapertura. Postgres sovrascrive, validando l'ultimo arrivato. La "Perdita di aggiornamento" ha materialmente distrutto i 100 di t_1 .

7.4.3 Esercizio d'Esame 27/05/2019: Le Inconsistenze Occulte

Il tranello ultimo del MVCC e dell'interlocking senza lucchetti si manifesta nei domini in cui la semantica applicativa collega letture e scritture incrociate in modi imprevedibili (la cosiddetta *Write Skew*).

Si valuti il formidabile esame ("Scenario 2, esecuzione concorrente") ove intervengono conteggi trasversali su Postgres: Una tabella **impiegati** (**matricola**, **cognome**, **conteggio**) con tuple preesistenti (101 Rossi 1, 102 Bruni 2). Due transazioni t_1 e t_2 operano simultaneamente e perversamente per creare dei cloni incrementati:

t_1 : select count(*) into R1 e insert into impiegati (101, Rossi, R1.C + 1)
 t_2 : select count(*) into R2 e insert into impiegati (102, Bruni, R2.C + 1)

- Se la classe isolativa è imposta per decreto a **SERIALIZABLE**, lo scheduler implementato post versione 9.1 si destreggia magistralmente. Riconosce per calcolo algoritmico che le transazioni soffrono di "cicli di conflitti lettura/scrittura" interpolati asincronamente sulle macrostrutture, deducendo e capendo "che le letture influenzano le scritture" reciproche. Al momento del varco temporale per il commit della seconda sorella ritardataria, il verificatore impone il castigo e "la seconda transazione viene abortita".
- Se l'operatore declassa ingenuamente il profilo richiedendo il **REPEATABLE READ** (che nell'assioma teorico basta e avanza a precludere gli aggiornamenti fantasma su valori stabili preesistenti), l'architettura MVCC Postgresiana si infossa disastrosamente. Il protocollo RR blocca solo le sovrascritture su stessi record, ma qui si sta operando su *Insert* per record vergini (matricole diversificate 101/102). Postgres, ignorando che l'Insert di t_2 perverte ontologicamente il computo logico del **count** per t_1 e viceversa, dichiara il semaforo verde per entrambe le iterazioni.

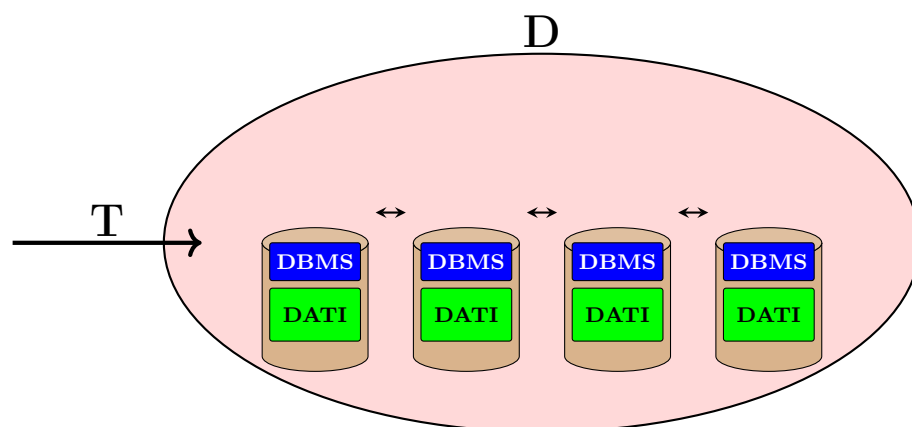
"Entrambe le transazioni vengono accettate", e il disastro logico si compie, con un risultato finale (valori spuri del conteggio logico) che una schedulazione seriale perfetta non avrebbe mai e in alcun modo potuto sdoganare, giustificando la sentenza formale che "il controllo di concorrenza ignora la semantica delle operazioni".

Capitolo 8

Gestione delle Transazioni nelle Basi di Dati Distribuite

8.1 Architetture e Tipologie di Transazioni Distribuite

Il mondo reale dei sistemi informativi complessi richiede di trascendere l'isolamento del singolo calcolatore. L'architettura evolve verso il concetto di **Base di dati distribuita**. In un tale paradigma, il patrimonio dei "dati e il controllo sono distribuiti su server (nodi) diversi, coordinati fra loro". L'insieme omogeneo forma un dominio logico (indicato con D) entro il quale le operazioni della transazione T in ingresso "sono per quanto possibile delegate ai vari nodi" per favorire il parallelismo e la vicinanza geografica o logica al dato.



Le strategie per l'implementazione pratica di queste "Architetture per basi di dati distribuite" si basano su due tecniche cardinali (spesso sovrapposte):

- **Partizionamento (sharding)**: Consiste nella "suddivisione dei dati fra vari nodi". Una gigantesca tabella, ad esempio, viene segmentata orizzontalmente in *chunk* o porzioni assegnate a macchine indipendenti per bilanciare il carico I/O.

- **Replicazione:** Consiste nel mantenere gli "stessi dati su più nodi" per ridondanza, fault-tolerance o per avvicinare i record in lettura agli utenti dislocati nel globo.

A livello applicativo, i linguaggi come SQL si adattano alla topologia di rete gestendo due ordini di interazioni:

1. **Transazione Remota:** Avviene quando un client esegue un programma che necessita di "Interagire con una base di dati nel contesto di un'altra". In SQL (standard Oracle/dblink), la sintassi esplicita la deviazione verso un nodo esterno tramite l'operatore @. Esempio:

```
UPDATE scott.dept@sales.us.americas.acme_auto.com  
SET loc = 'NEW YORK' WHERE deptno = 10;
```

2. **Transazione Distribuita su più basi di dati (o su base frammentata):** Avviene quando l'applicativo mira a "Interagire con più basi di dati contemporaneamente" o frammentate all'interno di una singola unità logica di lavoro ("La distribuzione è spesso trasparente per l'utilizzatore"). In questo caso la transazione spara direttive a tabelle diverse gestite da motori paralleli, per poi imporre una chiusura globale:

```
UPDATE scott.dept SET loc = 'NEW YORK' WHERE deptno = 10;  
UPDATE scott.emp SET deptno = 11 WHERE deptno = 10;  
COMMIT;
```

8.2 Tecnologia, Consistenza e Durabilità nei Sistemi Distribuiti

Focalizzandosi prettamente sui "contesti con il solo sharding" puro (l'analisi della replicazione introduce logiche differenti), l'architettura a rete dimostra un'intrinseca modularità: "la distribuzione non influenza consistenza e durabilità". Questo postulato si declina per i singoli gestori atomici:

- **Consistenza Locale:** "La consistenza non dipende dalla distribuzione: i vincoli di integrità sono locali". Poiché "la tecnologia DBMS attuale non gestisce vincoli distribuiti" incrociati o foreign key tra server fisici disgiunti, il gestore di integrità di ogni nodo si limita a vigilare asetticamente sulla conformità del proprio frammento locale di dati.
- **Durabilità Locale:** Analogamente, "La durabilità non dipende dalla distribuzione: ogni sistema garantisce la durabilità locale utilizzando meccanismi di recupero locali". Ciascun DBMS partecipante scrive diligentemente il proprio "log", stacca il proprio "checkpoint" e crea il proprio "dump" su dischi fisici locali.

Ciò che risulta drammaticamente ed esclusivamente impattato (e che rappresenta l'"Aspetto rilevante") dalla natura distribuita è la triade composta da:

1. ****Concorrenza**** (e isolamento relazionale).
2. ****Affidabilità**** (esclusivamente intesa come tutela dell'atomicità "tutto o niente" a livello macroscopico).
3. ****Ottimizzazione delle interrogazioni**** (efficienza del piano di esecuzione di rete).

8.3 Controllo di Concorrenza su Nodi Plurimi

Nel contesto distribuito, l'isolamento diviene un'astrazione fragile. "In teoria: La serializzabilità locale non è sufficiente per la serializzabilità globale".

Si esamini il seguente "Esempio, due transazioni t_1 e t_2 su due nodi A e B":

$$t_1 : r_{1A}(x) \ w_{1A}(x) \ r_{1B}(y) \ w_{1B}(y)$$

$$t_2 : r_{2B}(y) \ w_{2B}(y) \ r_{2A}(x) \ w_{2A}(x)$$

Se i due nodi fossero svincolati l'uno dall'altro, ciascuno schedulerebbe autonomamente i propri task. Potrebbero emergere i seguenti schedule fisici "Sui nodi":

$$\text{Nodo A} : r_{1A}(x) \ w_{1A}(x) \ r_{2A}(x) \ w_{2A}(x)$$

$$\text{Nodo B} : r_{2B}(y) \ w_{2B}(y) \ r_{1B}(y) \ w_{1B}(y)$$

Analizzandoli separatamente, il nodo A garantisce l'isolamento locale serializzando t_1 prima di t_2 (Conflict-equivalente a $t_1 \rightarrow t_2$). Il nodo B agisce altrettanto lecitamente sul suo frammento y , ma serializza t_2 prima di t_1 (Conflict-equivalente a $t_2 \rightarrow t_1$). Sebbene i due DB siano singolarmente perfetti e privi di lock falliti, il collante logico intersistemico è spezzato: l'ordine dei conflitti R-W si è capovolto. Il sistema non possiede serializzabilità globale, esponendo la business logic ad anomalie gravissime.

"Però", nella prassi architetturale, il disastro si sventa facilmente. Il rispetto del protocollo S2PL (Strict Two-Phase Locking) salva il paradigma: "se i lock in scrittura sono mantenuti fino al commit, che è globale (cioè coinvolge tutti i nodi) la serializzabilità è garantita dall'ordinamento dei commit e dalle letture delle versioni appropriate". Inibendo il rilascio di un lucchetto su A finché B non ha pattuito la chiusura collettiva, lo scheduler forza in stallo (o a risoluzione topologica corretta) ogni tentativo di inversione cronologica asincrona.

8.4 Guasti Sistemici e l'Atomicità Globale

Il problema monumentale della base di dati distribuita emerge durante il **commit**. Per tutelare l'atomicità ("tutto o niente") della transazione globale, un fallimento in un singolo frammento allocato a Tokyo deve potersi riversare, annullando retroattivamente e inesorabilmente i calcoli di successo già elaborati nel frammento allocato a Roma.

A peggiorare le probabilità di successo intervengono i "Guasti nei sistemi distribuiti":

- **Guasti sui singoli nodi (locali):** Equiparabili al "caso centralizzato" (crash di sistema locale, blackout).
- **Perdite di messaggi:** L'instabilità del cavo di rete corrompe i pacchetti TCP/IP. Le perdite "lasciano i protocolli in stato di incertezza". Per prassi, "I messaggi sono seguiti da conferme (ack)". Purtroppo, nell'alveo dell'affidabilità "Si possono perdere gli uni come gli altri" generando loop di retrying o false deduzioni di down.
- **Partizionamento della rete:** Uno switch principale cade e la rete si spacca in due emisferi ciechi tra di loro, pur mantenendo i nodi interni vivi e vitali.

Per governare questo caos comunicativo si elegge un paradigma rigido ad "Una decisione (commit o abort) per tutti i partecipanti ad una transazione".

Questa concertazione è "Simile ad un contratto" notarile civile: "Due o più parti si impegnano e una terza parte (il mediatore o il notaio) ratifica" la validità formale dell'atto. Gli attori sul palcoscenico informatico assumono nomenclature speculari:

- **I partecipanti:** Nodi foglia che detengono la frazione dei dati. Assumono il ruolo di *resource manager* (RM).
- **Il coordinatore:** Il nodo centrale (spesso eletto tra uno stesso dei partecipanti, non necessariamente un server a parte) che orchestra i turni. Assume il ruolo di *transaction manager* (TM).

8.5 Il Protocollo di Commit a Due Fasi (2PC)

Il meccanismo sincronico universale atto a preservare l'atomicità diffusa è il **Protocollo di commit a due fasi**. Esso ricalca la cerimonia nuziale: prima si interpellano le parti individualmente, si acquisisce l'eventuale assenso vincolante e, solo a seguito dell'unanimità, si proclama l'atto pubblico o l'annullamento.

Esistono "Due fasi, preparazione e conferma, separate da una decisione" irrevocabile e centrale.

- **Fase di preparazione:** Il TM interroga la platea lanciando il quesito "siete disponibili al commit?".
- **Momento di decisione:** "se tutti d'accordo 'ok' altrimenti 'lasciamo perdere'".
- **Fase di conferma:** Consta della "comunicazione a tutti, con conferma di ricezione" dell'esito, per forzare l'allineamento dei dischi.

Questo balletto diplomatico si regge integralmente "con scambio (rapido) di messaggi fra coordinatore e partecipanti con scritture nei log e con conferme (ack)" ad ogni singolo step per garantirsi la persistenza logica a fronte dei guasti di rete.

8.5.1 Dizionari di Log: TM vs RM

Il rigore protocollare si evince dai vocaboli esatti che i manager incedono e marchiano in memoria stabile.

Record nel log del coordinatore (TM):

- **prepare**: Marcatore primordiale. "Contiene l'identificatore della transazione e dei partecipanti". Costituisce il promemoria fisico, "serve a ricordare di avere chiesto disponibilità".
- **global commit** o **global abort**: Rappresentano l'istante cogente della "decisione".
- **complete**: Atto di chiusura che segna la "conclusione del protocollo" e permette di scartare i file in ram.

Record nei log dei partecipanti (RM): Oltre alle tracce abituali "come già visto per la gestione tradizionale: begin, insert, delete, e update oltre a commit e abort", viene forgiato un record rivoluzionario:

- **ready**: È la cambiale e la "conferma di avere dato disponibilità (irrevocabile) a partecipare al commit; contiene riferimenti alla transazione e al coordinatore". Questo record viene scritto dal frammento *solamente* "dopo avere verificato che, dal punto di vista del partecipante, la transazione può andare in commit (ha le risorse, soddisfa i vincoli,...)". L'atto di firma per l'RM è un suicidio tattico temporaneo: "dopo aver scritto ready il partecipante perde la propria autonomia decisionale sulla transazione". A prescindere da blackout futuri, per lui quel pacchetto di record è congelato in pendenza perpetua.

8.5.2 Evoluzione Completa delle Due Fasi

Prima Fase del Protocollo (Preparazione):

1. "Il coordinatore scrive **prepare** (indicando transazione e partecipanti) nel log e poi invia un messaggio di **prepare** ai partecipanti. Fissa un timeout per le risposte". (Sveglia 1).
2. "Ogni singolo partecipante che riceve il messaggio di **prepare**" valuta la sua coscienza logica.
3. "se è in grado di partecipare al commit, scrive nel proprio log un record di **ready** e poi trasmette al coordinatore un messaggio di **ready** per comunicare la propria disponibilità".
4. "altrimenti, può inviare un messaggio di **not-ready** (oppure niente)", rifugiandosi nel silenzio.
5. "Il coordinatore riceve risposte fino al timeout".
6. "se ha ricevuto *tutte* le risposte e sono *tutte positive* scrive un record **global commit** nel proprio log". L'atomicità in positivo è sancita.

7. "altrimenti (almeno una risposta mancante o negativa) scrive un record di `global abort`". La catena si frantuma per salvaguardare l'insieme.

Seconda Fase del Protocollo (Conferma e Propagazione):

1. "Il coordinatore trasmette la decisione (globale) ai partecipanti che hanno risposto `ready` e a quelli la cui risposta non è arrivata, fissando un nuovo `time-out`". (Sveglia 2).
2. "Ogni partecipante che riceve il messaggio, scrive la decisione (record di `commit` o `abort`) nel proprio log, invia un `ack` al coordinatore e implementa la decisione con le opportune scritture" (rilasciando lucchetti 2PL ecc.).
3. "Il coordinatore riceve i messaggi di `ack` dai partecipanti".
4. "se allo scadere del `time-out` ne manca qualcuno (fra coloro che avevano risposto `ready` o non avevano risposto), invia loro di nuovo la decisione". (Insiste fino alla sfinimento).
5. "quando tutti hanno risposto scrive un record di `complete` nel log". Protocollo estinto.

8.6 Stati Anomali: La Finestra di Incertezza e il Recovery Distribuito

Il nervo scoperto ed il punto debole intrinseco del 2PC risiede nello stato liminale del RM. "Un partecipante in uno stato di `ready`" ha letteralmente e volontariamente "perso la propria autonomia". Esso "attende la decisione del coordinatore" centrale. Tuttavia, "se ha acquisito lock (e questo accade quasi sempre) le risorse relative sono bloccate" a tempo indeterminato e "è lasciato in uno stato incerto in caso di fallimento del coordinatore".

Questo lasso di tempo psicologico e formale si definisce ****Finestra di incertezza****. Corrisponde a "l'intervallo tra la scrittura del `ready` e la ricezione della decisione". Durante questi secondi (o ore, in caso di crash prolungato del nodo leader), "il partecipante resta in attesa della decisione e non può fare niente". Non può dedurre un `abort` arbitrario (potrebbe innescare un'anomalia globale se il leader aveva già ratificato il `global commit` su altri nodi), né tantomeno forzare il salvataggio. Resta congelato.

8.6.1 Recupero Post-Crash (Recovery del TM e dell'RM)

I "Guasti (Crash, perdite di messaggi di notifica o di `ack`, partizionamenti)" vengono sterilizzati ed incapsulati col dogma pessimista: "sono gestiti tutti nello stesso modo: se non si riceve conferma si considera fallita la fase". Conseguenze immediate per l'orchestratore TM: "se è la prima, (di solito) si abortisce; se è la seconda, si ripete la seconda fase". ("Nota bene: la decisione è presa fra la prima e la seconda fase, quindi la seconda fase non può modificarla", è solo uno sforzo di propagazione e notifica).

Al risveglio dopo un blackout severo o un boot a freddo, i motori eseguono lo scanning dei propri archivi fisici stabili:

****Recovery (ripristino) di un partecipante (RM):**** L'agente isolato sonda "Per ogni transazione" pendente.

- "se l'ultimo record nel log relativo ad essa è un **commit**, le sue azioni debbono essere confermate (redo)". La catena locale è conclusa.
- "se l'ultimo record è un'azione o un **abort**, allora la transazione deve abortire, con undo delle sue azioni". Significa che il partecipante è "caduto" prima ancora di spedire la dichiarazione di ready. Essendo mancata la sua risposta al giro primario, il TM avrà dedotto l'aborto per l'intero grappolo. Procedere all'undo locale è perfettamente coerente col diktat globale.
- "se l'ultimo record è un **ready**, il partecipante non ha informazioni sull'esito della transazione e ha bisogno di notizie dal coordinatore". Il partecipante si trova bloccato nella maledetta finestra. Non vi è algoritmo locale salvifico: "può aspettare o chiederle". Interrogherà il master al suo risveglio.

****Recovery (ripristino) del coordinatore (TM):**** Rialzatosi dalle proprie ceneri, il leader valuta la sua coda "Per ogni transazione" in base all'ultimo fiato vitale apposto nel log.

- "se l'ultimo record è **complete**, non è necessario fare niente (la decisione è stata presa e tutti sono stati informati e hanno recepito l'informazione)". Pratica evasa.
- "se l'ultimo record è una decisione (**global commit/abort**), allora la decisione è stata presa e non può essere cambiata, ma non è sicuro che tutti siano informati e allora va ripetuta la seconda fase". Il TM spara massivamente nel brodo primordiale i solleciti fino a esaurire tutti gli ack dei dispersi.
- "se l'ultimo record è **prepare**, ci possono essere partecipanti in attesa della decisione" sparsi nell'etere. Ma non essendo approdato al global commit prima della morte, "la soluzione più semplice è global abort" (cancellare tutto l'albero d'impegno), o in alternativa (se si presume che i dati siano vivi), "è possibile ripetere la prima fase" rilanciando le interrogazioni.